

PYTHON NOTATKI

Laboratorium, rok akademicki 2025/2026, semestr zimowy
Wydział Fizyki, Astronomii i Informatyki Stosowanej UJ

AUTOR

Witold Przygoda

Notatki nie aspirują do roli kompletnego kursu języka Python,
ale mogą pomóc w systematycznej pracy podczas kursu.
Dokument jest ciągle aktualizowany, zmieniany i poprawiany,
w miarę rozwoju kursu oraz pojawiających się pomysłów.
Zapisane informacje mogą zawierać błędy lub być nieaktualne.
Za powstałe szkody wynikłe z używania tych notatek
autor nie ponosi odpowiedzialności.

Aktualizacja: 15 sierpnia 2026, 21:52

Spis treści

1. Instalacja i środowisko pracy	4
Instalacja klasyczna	4
Pip – zarządzanie pakietami.....	4
Wirtualne środowisko pracy venv.....	6
Konfigurowanie narzędzi do pracy	7
Notebook.....	9
Google Colab	10
Python w przeglądarce.....	10
Narzędzia AI.....	11
2. Konsola.....	11
3. Nazwy i typy w języku Python	15
Informacje o typach i konwersje.....	15
Obiekty a pamięć.....	17
Typy proste.....	17
Obiekty niemodyfikowalne, identyfikacja, usuwanie	22
Operatory, wyrażenia kontrolne, pętle	23
4. Typy złożone	39
LIST	39
TUPLE.....	51
DICT	53
SET	56
FROZENSET	57
5. Funkcje	58
Dekorator	62
Wyrażenia lambda.....	64
Map	67
Filter	68
Reduce.....	68
6. Przestrzeń nazw	69
7. Moduły i pakiety	75
Pakiety w Pythonie.....	79
8. Klasy	83
Protokół deskryptorów	95
MRO (Method Resolution Order)	101

METAPROGRAMOWANIE.....	108
Projektowanie obiektowe	117
9. DODATKI.....	121
Print	121
Tkinter	127

1. INSTALACJA I ŚRODOWISKO PRACY

Istnieją różne implementacje Pythona, my będziemy używać klasyczną (CPython), napisaną w języku C. Oznacza to, że kod źródłowy jest najpierw przetwarzany na drzewo AST (Abstract Syntax Trees <https://docs.python.org/3/library/ast.html>). Drzewo AST jest kompilowane do kodu bajtowego (zapisywanego w tymczasowych plikach *.pyc, *.pyo – można je podejrzeć za pomocą modułu dis <https://docs.python.org/3/library/dis.html>). Kolejne fragmenty bytecode'u są tworzone lub czytane (z plików) i wysyłane do wykonania przez wirtualną maszynę (PVM), która dokonuje ostatecznej konwersji na kod maszynowy i go uruchamia na konkretnym sprzęcie. Maszyna wirtualna jest ostatnim etapem interpretera Pythona.

Sam pakiet z interpreterem Python może zostać zainstalowany niejako przy okazji z innymi produktami, np. IDE PyCharm, czy popularnym pakietem Anaconda. Na komputerze może być zainstalowane wiele wersji. Wraz z instalacją Pythona pod Windows (opis <https://realpython.com/installing-python/>), instaluje się **py** launcher, który pomaga w zarządzaniu różnymi wersjami, powiązaniu plików .py z interpreterem:

<https://www.infoworld.com/article/2265603/how-to-use-pythons-py-launcher-for-windows.html>. Można skonfigurować w swoim projekcie izolowane wirtualne środowisko pracy, z wybraną wersją (opis poniżej). Zapewne zauważymy, że różne IDE tworzą sobie właśnie takie wirtualne środowiska (venv), zawierające konkretny binarny program interpretera Python.

INSTALACJA KLASYCZNA

Na stronie <https://www.python.org/downloads/> wybrać właściwy pakiet. Na ten moment w wersji 3.12.7, raczej nie poleca się instalować natychmiast najnowszych wersji (np. 3.13.0), zanim nie *upewnimy się*, czy interesujące nas moduły również działają bez niespodzianek. Podczas instalacji pod Windows zaznaczyć opcję Add PYTHON 3.12 to PATH

Po zainstalowaniu otworzymy okienko (terminal), pod Windows zalecana jest aplikacja PowerShell. Po instalacji sprawdzimy:

```
PS C:\> python --version
Python 3.12.7
PS C:\> py -0p
-V:3.12 *          C:\Python\python.exe
```

Do linii poleceń interpretera możemy zatem wejść poprzez **python** lub **py**, pojawi się znaczek:

```
>>>
```

Z interpretera wychodzimy poprzez **quit()** lub szybciej Ctrl-Z (i Enter).

PIP – ZARZĄDZANIE PAKIETAMI

Zazwyczaj istnieje konieczność doinstalowania pakietów. Służy do tego program **pip**. Python często informuje o tym, że pip ma nowszą wersję i zachęca do aktualizacji. Możemy na przykład zainstalować rozszerzenie (An enhanced Interactive Python) o nazwie **ipython**, które ma numerację kolejnych komend. <https://ipython.org/>

➤ pip install ipython

Po zainstalowaniu:

```
PS C:\> ipython
Python 3.12.7 (tags/v3.12.7:0b05ead, Oct 1 2024, 03:06:41) [MSC v.1941 64 bit (AMD64)]
Type 'copyright', 'credits' or 'license' for more information
IPython 8.21.0 -- An enhanced Interactive Python. Type '?' for help.

In [1]: print("hello")
hello

In [2]:
```

pip to oficjalny menedżer pakietów Pythona, który pozwala instalować, aktualizować oraz usuwać biblioteki Pythonowe. **pip** pobiera pakiety z Python Package Index (PyPI), czyli największego repozytorium oprogramowania open source dla Pythona <https://pypi.org/>. Kilka przydatnych komend (na przykładzie pakietu pygame).

pip show pygame – sprawdzenie czy jest zainstalowane pygame, wersja i inne informacje

pip install pygame==2.6.0 – zainstalowanie konkretnej wersji modułu, tutaj 2.6.0

pip uninstall pygame – usunięcie pygame

pip index versions pygame – sprawdzenie dostępnych wersji

pip install --upgrade pygame – aktualizacja do najnowszej wersji

Czasem, jeśli coś się „popsuje” (zdarza się tak, gdy ręcznie manipulujemy zainstalowanymi wersjami pythona), można wymusić reinstalację pakietu:

pip install --force-reinstall pygame

albo dodatkowo z opcją wymuszającą pobranie pakietu (niekorzystania z cache):

pip install --force-reinstall --no-cache-dir pygame

Na koniec wspomnijmy jeszcze: **pip show pygame** (wypisze informacje) oraz **pip list** (wylistuje wszystkie zainstalowane pakiety). Podobną informację, ale sformatowaną pod kątem zainstalowania grupy pakietów, osiągniemy poleceniem: **pip freeze**. Najczęściej wynik zapisuje się (przekierowując strumień standardowy) do pliku o nazwie `requirements.txt`:

➤ pip freeze > requirements.txt

Gdy będziemy pisać projekt, wymagający użycia określonych bibliotek, albo nawet jeśli tak będzie w przypadku realizacji któregoś z zadań, warto załączyć do niego plik wymagań – czyli stworzyć plik `requirements.txt`. Przykładowo, podstawowy zestaw do obliczeń numerycznych, manipulacji danymi oraz wizualizacji w Pythonie, to pakiety, które można zapisać w pliku wymaganych:

numpy==1.26.4

pandas==2.2.3

matplotlib==3.9.2

Można podać samą nazwę pakietu, można podać też maskę głównej wersji (np. 1.* dla numpy, żeby nie zainstalowała się wersja 2.x), można podać minimalną wymaganą wersję, jak i maksymalną. Ilustruje to poniższy przykład:

numpy==1.*

pandas>=1.0.0,<2.0.0

matplotlib>=3.3.0

scipy # dowolna wersja

Instalowanie pakietów: **pip install -r requirements.txt**

Można dodać opcję aktualizacji (`-U` lub `--upgrade`). Odinstalowanie pakietów wraz z domyślnym potwierdzeniem:

```
pip uninstall -r requirements.txt -y
```

Omówienie pip znajdziemy pod adresem <https://realpython.com/what-is-pip/>

Alternatywne rozwiązania dla pip, to menadżer Conda <https://conda.org/>, najczęściej wykorzystywany wraz z <https://www.anaconda.com/> a także system Poetry <https://python-poetry.org/>.

WIRTUALNE ŚRODOWISKO PRACY VENV

Pracując z różnymi projektami, warto utworzyć dla nich izolowane środowisko. Python posiada wbudowane narzędzie `venv` do tworzenia wirtualnych środowisk, które pozwalają na izolowanie zależności projektu. Dzięki temu każda aplikacja może mieć swoje własne, oddzielne pakiety, co zapobiega konfliktom między różnymi wersjami bibliotek.

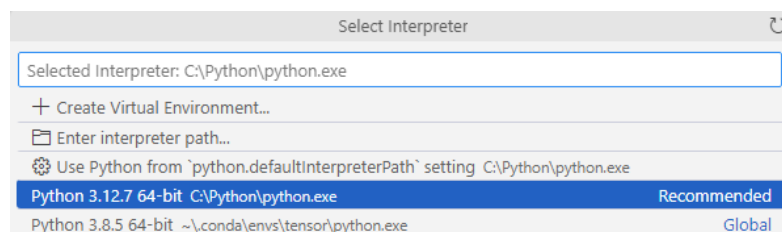
Utworzenie lokalnej konfiguracji:

```
python -m venv nazwa_srodowiska (można użyć „launher” czyli: py -m venv nazwa).
```

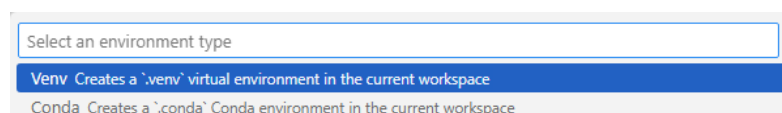
`nazwa_srodowiska` to katalog, który zostanie utworzony. Bardzo często praktykuje się: `py -m venv venv\` ukośnik na końcu nie jest potrzebny, ale podkreśla, że ostatnia nazwa dotyczy tworzonego katalogu.

Aby korzystać z wirtualnego środowiska, trzeba je aktywować. Kroki aktywacji różnią się w zależności od systemu operacyjnego. Pod Windows: `venv\Scripts\activate`, pod Linux: `source venv/bin/activate`. Po aktywacji nazwa wirtualnego środowiska pojawiają się przed znakiem zachęty (np. `(venv)`). Oznacza to, że jesteśmy teraz w środowisku `venv` i wszystkie zainstalowane pakiety będą ograniczone tylko do tego środowiska. Kiedy środowisko jest aktywowane, można zainstalować pakiety używając `pip`. Aby wyjść z wirtualnego środowiska, użyj polecenia: `deactivate`. Po dezaktywacji wrócisz do swojego globalnego środowiska Pythona, a pakiety z `venv` nie będą już dostępne. `venv` jest łatwe w użyciu i bardzo przydatne, szczególnie podczas pracy nad wieloma projektami jednocześnie, aby uniknąć konfliktów między pakietami. Więcej szczegółowych informacji: <https://realpython.com/python-virtual-environments-a-primer/>

Visual Studio Code pozwala na utworzenie wirtualnego środowiska przy okazji wyboru interpretera. Po skrócie klawiszowym `Ctrl-Shift-P`, wpisujemy `Python: Select Interpreter`, zobaczmy:



Wyberzmy `Create Virtual Environment`, a następnie typ (np. `.venv`), po wyborze którego zostanie lokalnie utworzony podkatalog `.venv` wraz z wirtualnym środowiskiem dla danego projektu.



KONFIGUROWANIE NARZĘDZI DO PRACY

Zasadniczo można pisać program w Pythonie w prostym, interaktywnym środowisku programowania, w powłoce (konsoli), czyli realizując **REPL** (ang. **read-eval-print loop**) – użytkownik może wprowadzać polecenia, które zostaną wykonane, a ich wynik wypisany na ekran. Nie jest to wydajny sposób pracy. Do pisania kodu potrzebny jest edytor, lub lepiej, środowisko pracy wspomagające proces tworzenia kodu (IDE - integrated development environment). Można wybrać według własnych preferencji (niektórzy, znając i używając wcześniej, trwają przy edytorach vim lub emacs). Niektórzy używają załączonego w pythonowym pakiecie instalacyjnym IDLE. Bardzo dużo możliwości dają lekkie edytory Sublime Text <https://www.sublimetext.com/> lub Atom <https://atom.io/>. Z drugiej strony w pełni wyposażony pakiet IDE dedykowany dla Pythona to PyCharm <https://www.jetbrains.com/pycharm/> (istnieje licencja akademicka). Całkowicie darmowym produktem jest Spyder <https://www.spyder-ide.org/> używane w analizie danych, naukach przyrodniczych oraz inżynierii, głównie ze względu na swoją integrację z popularnymi bibliotekami Pythona, takimi jak NumPy, SciPy czy Matplotlib.

Rekomendacja

Bardzo wiele osób używa obecnie Visual Studio Code (firmy Microsoft – ale jest to produkt wieloplatformowy), ze względu na wydajne działanie, dużą elastyczność, konfigurowalność oraz ciągłe wsparcie i aktualizacje. Za pomocą wybranych dodatków można szybko skonfigurować VSC do wydajnej pracy z Pythonem. Oto jak to zrobić.

Instalujemy VSC <https://code.visualstudio.com/download>

Uruchamiamy i dodajemy rozszerzenia Extensions (Ctrl-Shift-X): Python (Intellisense od Microsoft), następnie wybieramy View > Command Palette (Ctrl-Shift-P): wpisujemy Python i z listy wybieramy Python: Select linter, można tu poeksperymentować, teraz wybierzmy **pylint**. Linter to moduł sprawdzający i podpowiadający składnię. Pylint, jak zobaczymy, instaluje się inaczej niż typowe dodatki VSC, bo poprzez wywołanie komendy (zrobi się automatycznie):

➤ "C:/Program Files/Python/python.exe" -m pip install -U pylint

Utwórzmy plik main.py (w jakimś podkatalogu, który też można utworzyć) i wpisząmy celowo błędną składnię (z punktu widzenia Python3), na przykład:

```
print "hello wrong"
```

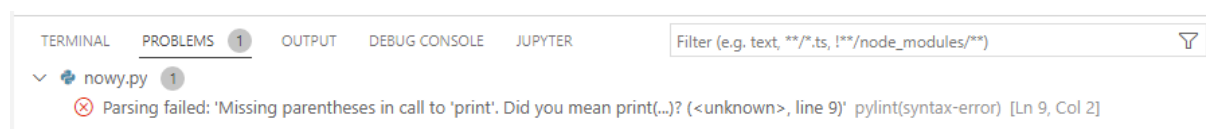
Powinien się pojawić problem oraz podpowiedź (może wyglądać, jeśli będzie więcej linterów):

```
Parsing failed: 'Missing parentheses in call to 'print'. Did you mean print(...)? (<unknown>, line 2)' pylint(syntax-error)
```

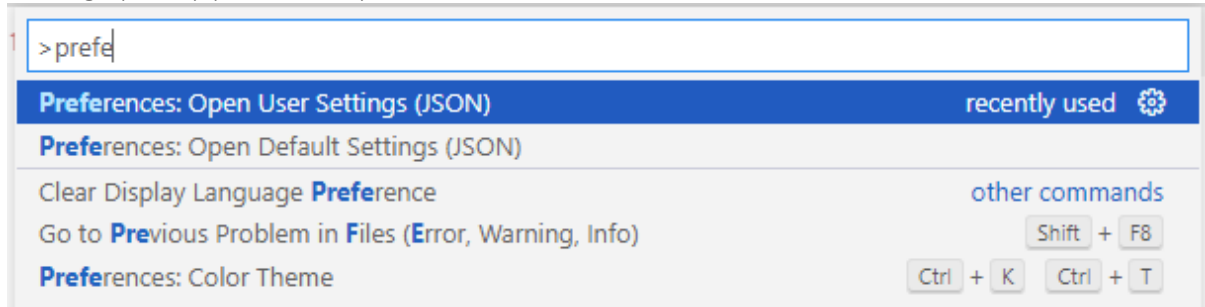
[View Problem](#) No quick fixes available

```
print "hello wrong"
```

Albo (Ctrl-Shift-M) zakładka Problems:



Może być tak, że zobaczymy dwie „porady”, również z innego Lintera – Pylance. Powiedzmy, że chcemy go usunąć. Prosta operację (udać się do rozszerzeń, odszukać Pylance – i można np. dezaktywować, albo odinstalować) trzeba jednak uzupełnić wpisem w pliku konfiguracyjnym VSC. W tym celu idziemy do Command Palette (Ctrl-Shift-P) i szukamy Preferences: Open User Settings (JSON) (nie Default):



Otwieramy plik settings.json do edycji i np. na końcu dopisujemy (jeśli nie ma):

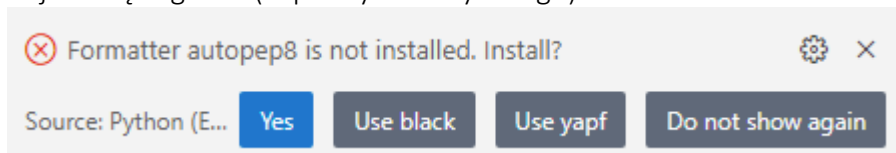
```
"python.languageServer": "None"
```

Po przeładowaniu powinniśmy widzieć podpowiedź tylko z Pylint.

W języku Python szereg opisów jak ma działać oraz propozycji usprawnień zawarta jest w tak zwanych PEPs (Python Enhancement Proposals) <https://www.python.org/dev/peps/>. Jeden z najbardziej znanych, to PEP 8 -- Style Guide for Python Code, <https://www.python.org/dev/peps/pep-0008/> czyli jak powinien być formatowany kod Pythona. VSC posiada dodatek wspomagający właściwe formatowanie kodu. W naszym pliku napiszmy teraz np.

```
x=0
```

a następnie w Command Palette (Ctrl-Shift-P) Format Document With i wybierzmy Python. Pojawi się sugestia (w prawym dolnym rogu):



aby zainstalować Formatter autopep8 – zatwierdzamy i instalujemy, również w tym przypadku wywołana będzie zewnętrzna komenda:

```
➤ "C:/Program Files/Python/python.exe" -m pip install -U autopep8
```

Teraz możemy wykonać formatowanie – albo idąc do Format Document jak wyżej, albo lepiej, korzystając ze skrótu klawiszowego: Shift-Alt-F. W tym przypadku zobaczymy:

```
x = 0
```

ponieważ w PEP8 jest rekomendacja, aby przed i za operatorem były spacje. Nie należy formatowania mylić z poprawianiem błędów – do tego celu służy linter (wcześniej opisany pylint). Jeśli chcemy, aby formatowanie nastąpiło automatycznie podczas zapisu pliku, można w File > Preferences > Settings (albo skrót Ctrl-,) wyszukać formatOnSave i zaznaczyć opcję Editor: Format On Save.

Założmy, że mamy z powrotem w pliku prostą instrukcję:

```
print("pierwszy program")
```


Pylint, w zależności od ustawień, ostrzeże nas: Missing module docstring (czyli że nie ma w naszym pliku – module żadnego opisu komentarza). Jeśli ostrzeżenia nas irytują, można je wyłączyć. W tym celu idziemy do pliku settings.json (jak poprzednio) i dodajmy linię:

```
"python.linting.pylintArgs": ["--errors-only"]
```

Notatka: można odczytać konkretny kod komunikatu, żeby się zorientować o co linterowi chodzi... w terminalu piszemy:

➤ `pylint --help-msg=missing-module-docstring`

Mamy zatem plik z instrukcją print i chcemy go uruchomić z VSC. W tym celu dodamy jeszcze jedno rozszerzenie, idziemy do Extensions i szukamy "code runner", wybieramy ten z pomarańczową ikonką .run i instalujemy. Od tej pory uruchomienie kodu to: Ctrl-Alt-N (a zatrzymanie Ctrl-Alt-M). W terminalu VSC zobaczymy coś typu:

```

TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE
[Running] python -u "c:\tmp\PYTHON\main.py"
pierwszy program

[Done] exited with code=0 in 0.367 seconds

```

Jeśli ta sekcja na dole nam się przypadkiem zamknie, to: Terminal > New Terminal (lub skrót klawiszowy Ctrl `). W ten sposób mamy przygotowane środowisko pracy w VSC.

NOTEBOOK

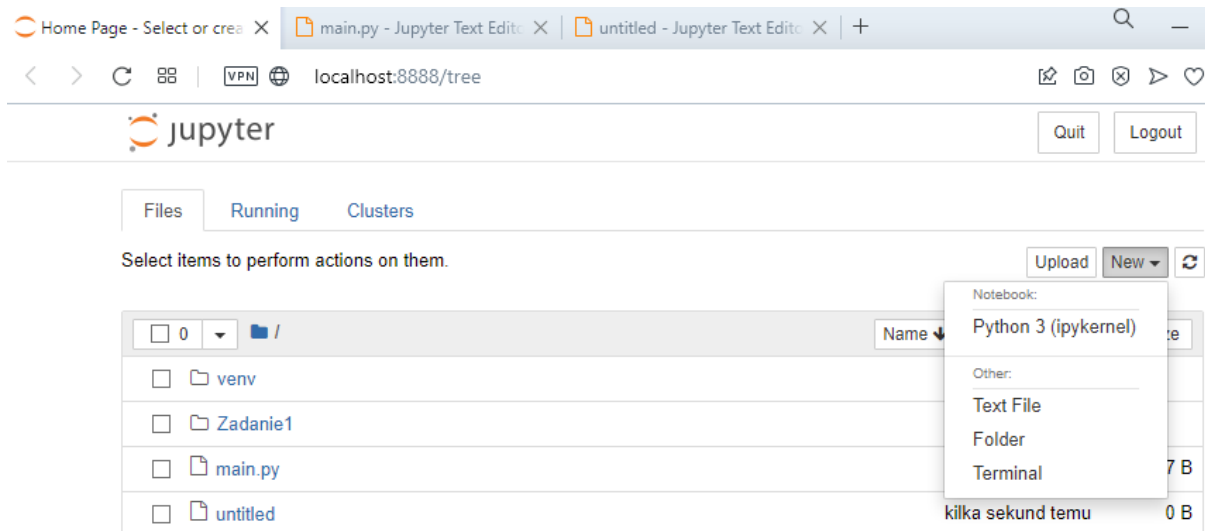
Innym bardzo popularnym rozwiązaniem rozwijania kodu w Pythonie jest Jupyter Notebook <https://jupyter.org/>, który ma formę serwera uruchamianego na naszym komputerze, a edycja najczęściej odbywa się w przeglądarce internetowej. Jeśli ktoś decyduje się na taką strategię używania Pythona, to zazwyczaj wykonuje instalację poprzez pakiet Anaconda <https://www.anaconda.com/products/individual>, który ma własną strategię zarządzania pakietami o nazwie Conda <https://docs.conda.io/en/latest/>. Ponieważ mamy zainstalowany Python w klasycznym podejściu, możemy w ten sposób zainstalować Jupyter Notebook:

➤ `pip install notebook`

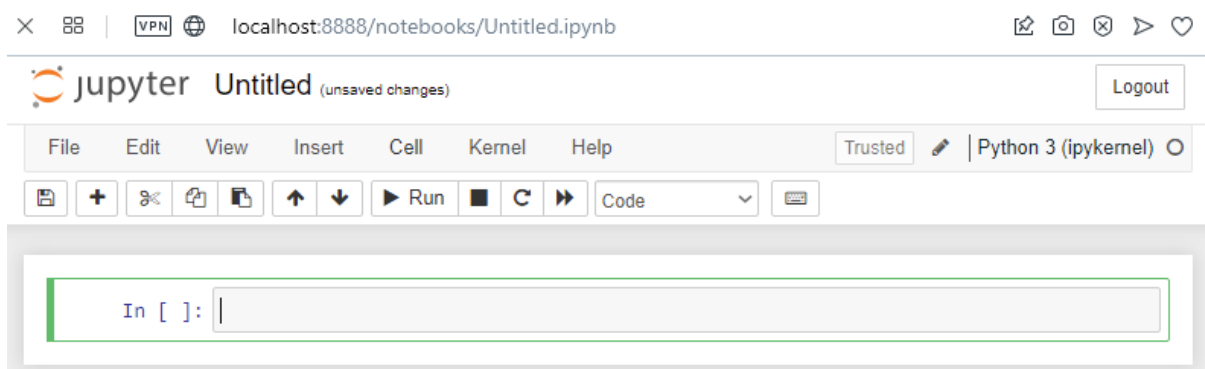
co skutkuje zainstalowaniem pokażnej liczby pakietów. Następnie, również w terminalu, uruchamiamy serwer:

➤ `jupyter notebook`

Powinna otworzyć się domyślna przeglądarka, jeśli nie, to trzeba otworzyć samemu i w polu adresowym wkleić podany na końcu w terminalu adres. Wtedy zobaczymy:



Jak widać na rozwiniętym menu, można rozpocząć nowy Notebook:



W kolejnych polach można pisać kod Pythona lub dokumentację, oraz uruchamiać (Run). Takie podejście jest bardzo popularne w szeroko pojętej analizie i przetwarzaniu danych. Notebook można zapisać (pliki z rozszerzeniem .ipynb).

GOOGLE COLAB

Jupyter Notebook w chmurze to właśnie Google Colab (skrót od "Colaboratory") <https://colab.research.google.com/>. Jako że jest to narzędzie oparte na chmurze, użytkownicy nie muszą (ale mogą) instalować ani konfigurować żadnego oprogramowania na swoich komputerach. Pliki zapisywane są na dysku Google Drive, z możliwością zespołowej pracy. Google Colab oferuje dostęp do GPU (Graphics Processing Unit) i TPU (Tensor Processing Unit), więc jest pomocny w rozwijaniu kodu z dziedziny ML. Oczywiście istnieją płatne plany, które dają dostęp do większej mocy obliczeniowej czy mocniejszych kart graficznych. Google Colab obsługuje Python 2 i Python 3 oraz wiele popularnych bibliotek Pythona, wspiera importowanie notebooków z GitHub oraz zapisywanie ich bezpośrednio na GitHub.

PYTHON W PRZEGLĄDARCE

Istnieje szereg serwisów, które umożliwiają pisanie i testowanie programów Pythona wprost w przeglądarce. Jest to wygodne rozwiązanie w przypadku, gdy akurat nie dysponujemy komputerem z odpowiednio zainstalowanym programem i środowiskiem. Dzięki temu możemy się uczyć praktycznie „zawsze i wszędzie”.

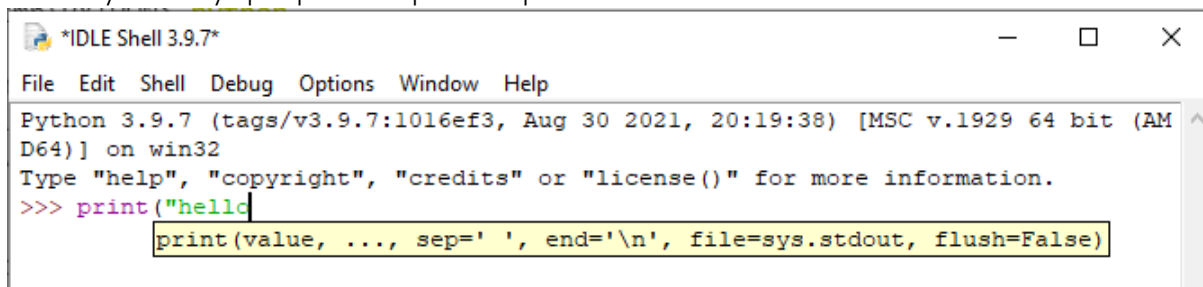
Serwisy warte zainteresowania: Brython <https://brython.info/tests/editor.html?lang=en>
 Sculpt <https://skulpt.org/> Pyodide <https://pyodide.org/en/latest/console.html> oraz Replit <https://replit.com/>

NARZĘDZIA AI

Temat używania narzędzi AI do pisania kodu, a przede wszystkim w nauce, jest bardzo dynamiczny. Powszechną wiedzą jest, że narzędzia typu ChatGPT bezproblemowo wygenerują dowolny kod, który np. będzie rozwiązaniem zadania, czy wręcz gotowym kodem projektu. W ten sposób jednak niczego nie zyskamy i nie nauczymy się. Nie oznacza to, że mamy takich narzędzi czy pomocy unikać – wręcz przeciwnie. Należy skorzystać z faktu, że serwis GitHub oferuje darmowe licencje akademickie, w tym dla GitHub Copilot. Wszelkie informacje znajdziemy na stronie <https://github.com/education>. Bardzo polecam lekturę <https://realpython.com/github-copilot-python/>, znajdziemy tam opis instalacji w różnych IDE, skrótów klawiszowych i sposobów używania Copilot. Zapewne ten paragraf będzie poszerzony w swoim czasie, aby przekazać ciekawe i użyteczne wskazówki.

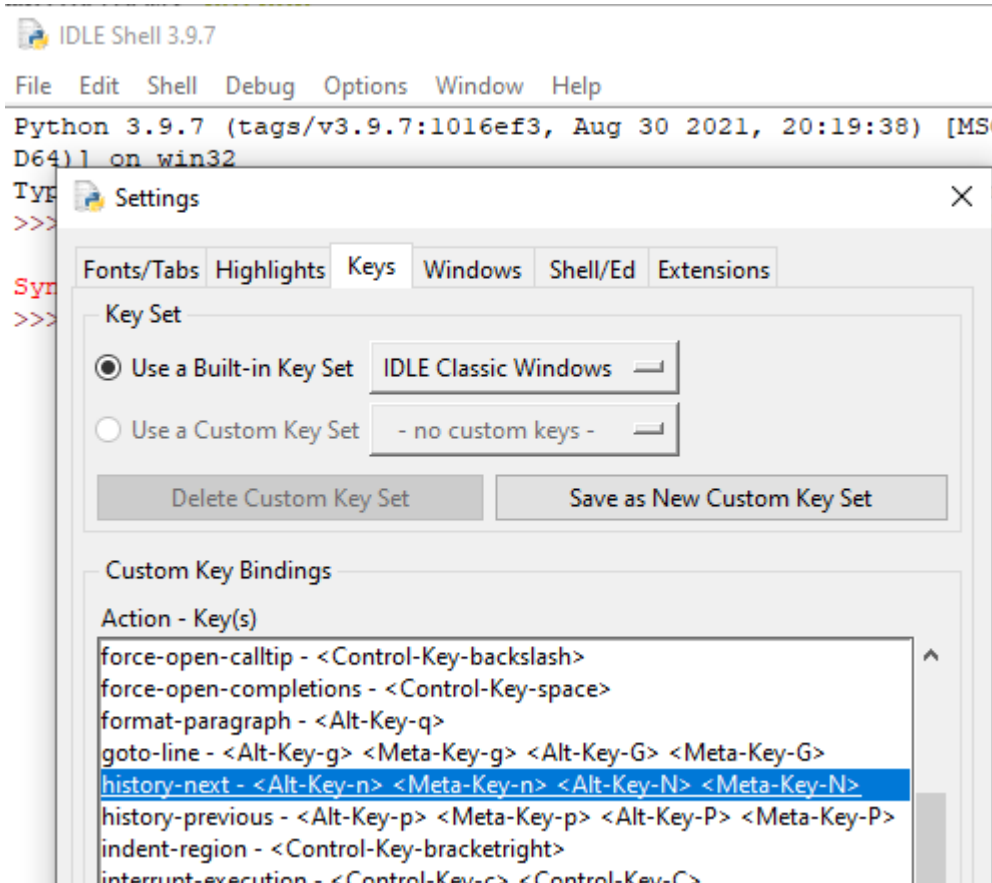
2. KONSOLA

Działania w interpreterze Pythona można prowadzić w dowolnym terminalu, zależnie od systemu operacyjnego. Przede wszystkim program interpretera powinien być widzialny (takie komendy, jak **whereis python** czy **which python**, pokazują co i gdzie jest zainstalowane). Python można uruchomić w oknie cmd Windows (niewygodne), w PowerShell Windows, albo np. w programie IDLE (jeśli instalowało się Python jako cały pakiet). Warto przećwiczyć wykonanie kilku komend, aby się oswoić. Nawet prosta funkcja print ma kilka opcji, np. w IDLE możemy zobaczyć podpowiedź podczas pisania:



```
Python 3.9.7 (tags/v3.9.7:1016ef3, Aug 30 2021, 20:19:38) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> print("hello
      print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)
```

Uwaga: IDLE ma niekoniecznie intuicyjny zestaw skrótów klawiszowych, który można przededefiniować lub się ich nauczyć. Przykładowo, poprzednia (kolejna) wykonana instrukcja to Alt-n (Alt-p).



Spróbujmy:

```
>>> print("hello world"*2, ' ', 123, 'tez string', sep=",", end=" ||KONIEC\n")
```

```
hello worldhello world, ,123,tez string ||KONIEC
```

```
>>> print('hello world', end=', '); print("yeah")
```

```
hello world, yeah
```

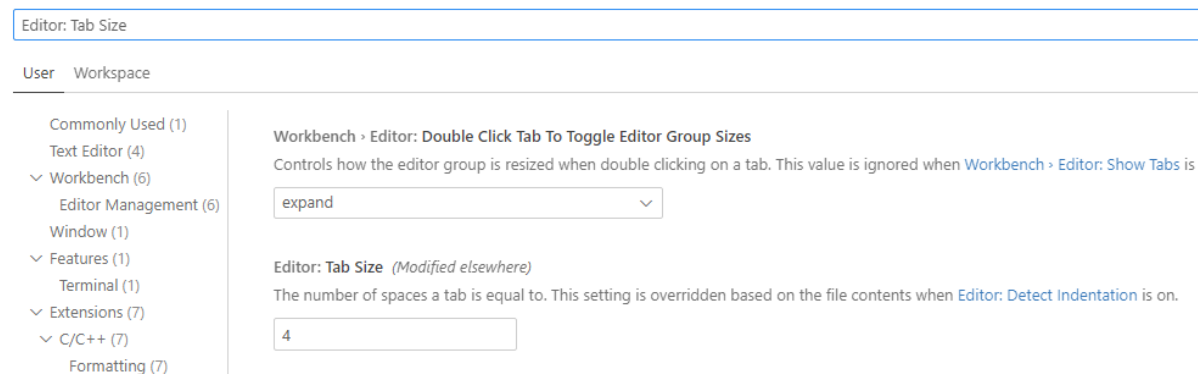
Z powyższych przykładów widać, że argumenty funkcji mogą mieć postać argumentów pozycyjnych bez nazwy, nazywanych (**sep**, **end**) – te można pisać w dowolnej kolejności (spróbuj zamienić end z sep), że może być separator **;** jeśli kolejna funkcja jest w tej samej linii.

Obiekty w Pythonie tworzy się dynamicznie, bez deklaracji typów, są zbudowane na podstawie wielkości inicjalizujących. Przykładowo, typ logiczny **bool** ma dwie wartości opisane jako **True**, oraz **False** (z wielkich liter), gdzie True umownie ma wartość 1, ale też wszystko, co nie jest zerem, obiektem pustym lub None, jest traktowane w kontekście logiki jako True.

```
>>> logika = True
>>> if logika:
    print("to jest prawda")
```

➔ Słowo kluczowe **None** służy do definiowania wartości null lub braku wartości. None to nie to samo co 0, False lub pusty ciąg. None jest własnym typem danych (NoneType).

Formatowanie kodu w Pythonie odbywa się za pomocą : (dwukropka) oraz odpowiednich wcięć. Do zrobienia wcięcia zaleca się użycie tabulacji, a nie spacji. Z drugiej strony warto tak skonfigurować tabulację, że wstawi ona określoną liczbę spacji, np. 4. W Visual Studio Code (skrót: Ctrl,) wpisując „tab size” znajdziemy odpowiednie pole, w którym wstawimy odpowiednią wartość.



Nie można mieszać rodzaju wcięć. Przykład:

```
>>> a = 2
>>> if a > 1:
    print("1 spacja")
    print("2 spacje")
SyntaxError: unexpected indent
```

Nawet „na oko” dobrze sformatowany kod, jeśli ma pomieszane tabulacje i spacje, może być błędny. Żeby tego doświadczyć trzeba oczywiście najpierw zapisać plik, w którym rzeczywiście jedno wcięcie będzie wypełnione znakiem Tab, a drugie zbudowane ze spacji. Można taką sytuację uzyskać np. pisząc fragment kodu w prostym edytorze typu Notatnik (Windows) i jedno wcięcie tworząc klawiszem Tab, a drugie wstawiając 8 spacji. Optycznie wygląda identycznie. Jednak po skopiowaniu takiego kodu do interpretera (np. okienko IDLE) zobaczymy błąd niekonsystencji znaków użytych we wcięciach.

The screenshot shows a Notepad window titled '*Bez tytułu — Notatnik'. It contains the following Python code:

```
>>> if a > 1:
    print("tabulacja")
    print("8 spacji")
>>> a = 2
>>> if a > 1:
    print("tabulacja")
    print("8 spacji")
SyntaxError: inconsistent use of tabs and spaces in indentation
```

Komentarz jest na prawo od znaku #. Komentarz zapisany w wielu liniach osiąga się przez stworzenie wielolinijkowego łańcucha znakowego, bez jego przypisania. Tworzy się go za pomocą trzy razy powtórnego pojedynczego ''' lub podwójnego """ cudzysłowu.

```
>>> logika = False # tu jest komentarz
>>> # komentarz od początku
>>> '''
pierwsza linia
druga linia
'''
'\npierwsza linia\ndruga linia\n'
>>> """
albo w ten sposób
to samo
"""
'\nalbo w ten sposób\nto samo\n'
```

➔ W Visual Studio Code jeśli zaznaczymy kilka linijek kodu, to skrótem klawiszowym Ctrl-K-C (lub prościej, Ctrl-/) możemy dodać znak komentarza # lub skrótem Ctrl-K-U go usunąć w wielu liniach jednocześnie.

Zmienne można tworzyć, wykonując przypisanie wielu wartości jednocześnie, np.

```
>>> a, b, c = 1, 2, "abc"
>>> print(a,b,c)
1 2 abc
```

Konsolę można użyć jako kalkulatora. Spróbujmy:

```
>>> 2 + 2
>>> 50 - 5*6
>>> (50 - 5*6) / 4
>>> 8 / 5 # float dzielenie prawdziwe
>>> 17 // 3 # dzielenie bez reszty
>>> 17 % 3 # modulo czyli reszta z dzielenia
>>> 5 * 3 + 2
>>> 5 ** 2 # 5^2
>>> 2 ** 7 # 2^7
>>> width = 20
>>> height = 5 * 9
>>> width * height
```

Ostatnia **wyświetlona** w konsoli wartość jest dostępna również pod nazwą (znacznikiem) `_`

```
>>> 5 # spróbuj (Enter)
>>> _
>>> _ + _
```

Aby mieć dostęp do większej liczby funkcji matematycznych, trzeba skorzystać z dodatkowych modułów. Przed użyciem, moduł należy zaimportować. W tym przypadku:

```
>>> import math
>>> math.sqrt(4)
```

Spis funkcji: ➔ <https://docs.python.org/3/library/math.html> w przypadku liczb zespolonych użyjemy cmath ➔ <https://docs.python.org/3/library/cmath.html>

3. NAZWY I TYPY W JĘZYKU PYTHON

Oficjalnie nazwy zmiennych w Pythonie mogą mieć dowolną długość i mogą składać się z wielkich i małych liter (A-Z, a-z), cyfr (0-9) oraz znaku podkreślenia (_). Pierwszym znakiem nazwy zmiennej nie może być cyfra. Python ma też zestaw słów kluczowych, które są słowami zastrzeżonymi, których nie można używać jako nazw zmiennych, nazw funkcji ani żadnych innych identyfikatorów. Choć na tym etapie poznawania języka jest to całkowicie nudny katalog 33. nazw, to warto się z nimi zapoznać, ich znaczenie będziemy poznawać: and, as, assert, break, class, continue, def, del, elif, else, except, False, finally, for, from, global, if, import, in, is, lambda, None, nonlocal, not, or, pass, raise, return, True, try, while, with, yield.

Typ obiektu określany jest podczas jego tworzenia. W Pythonie mamy kilka typów prostych, oraz typy złożone (klasyfikacja ze względu na strukturę składowania – „storage”). Mogą one być modyfikowalne lub niemodyfikowalne (klasyfikacja ze względu na modyfikowalność – „update”). Dostęp do nich może być bezpośredni – liczby lub sekwencyjny – różne kontenery (klasyfikacja rodzaju dostępu – „access”). Można wykonywać konwersję (rzutowanie). Oto lista typów:

- Text Type: str
- Numeric Types: int, float, complex
- Sequence Types: list, tuple, range
- Mapping Type: dict
- Set Types: set, frozenset
- Boolean Type: bool
- Binary Types: bytes, bytearray, memoryview

INFORMACJE O TYPACH I KONWERSJE

Typ można sprawdzić za pomocą funkcji **type()** lub **isinstance(obiekt, typ)**.

```
>>> a, b, c, d, e = True, 1, 3.14, "hello", 2+3j
>>> type(a); type(b); type(c); type(d); type(e)
<class 'bool'>
<class 'int'>
<class 'float'>
<class 'str'>
<class 'complex'>
```

```
>>> isinstance(a,int) # True
>>> isinstance(b,int) # True
>>> isinstance(c,int) # False
>>> isinstance(e,str) # False
```

Dla funkcji **isinstance()** drugim argumentem może być również kolekcja typów, wymienionych w okrągłym nawiasie, oddzielonych przecinkami (formalnie – jest to krotka, czyli typ **tuple**):

```
>>> isinstance("abc", (int, float, str, bool, type(None), complex))
True
```

Rzutowanie (jawna konwersja):

```
>>> float(b)
1.0
>>> str(a)
'True'
>>> int(c)
3
>>> complex(b)
(1+0j)
>>> bool(d)
True
```

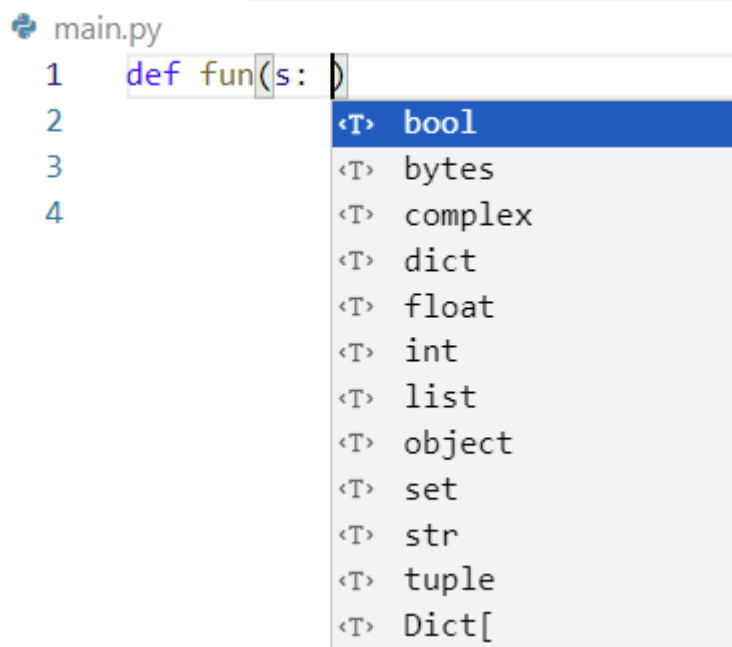
Nazwa w języku Python to swego rodzaju „uchwyt”, referencja. Generalnie nic nas nie może powstrzymać przed przypisaniem wartości czy wyrażeń różnych typów, do tej samej nazwy:

```
x = 2
x = "abcd"
```

I jest to poprawny kod Pythona. Od wersji 3.5 wprowadzono możliwość opisu typu zmiennej (dokument PEP484), nazywa się to „type hinting” albo „type annotation”. Takie prezentowanie typów nadal do niczego nie zmusza, ale pomaga w szybkim rozpoznaniu zamierzonego typu danego obiektu, według składni **nazwa: typ = <inicjalizator>**

Zatem, pisanie kodu z podaniem typu nie zmienia dynamicznego charakteru tworzenia obiektów, stanowi coś na kształt etykiety, czyli zwiększa czytelność kodu (jak również trafność podpowiedzi w IDE). Dla zainteresowanych <https://pypi.org/project/pyannotate/> lub <https://pypi.org/project/MonkeyType/>

Jeśli chcemy wsparcie dla takich opisów, można w VSC zainstalować dodatek Python Type Hint. Pisząc kod (np. definicję funkcji), otrzymamy podpowiedź:



```
main.py
1 def fun(s: )
2
3
4
```

The dropdown menu shows the following options:

- <T> bool
- <T> bytes
- <T> complex
- <T> dict
- <T> float
- <T> int
- <T> list
- <T> object
- <T> set
- <T> str
- <T> tuple
- <T> Dict[

Niemniej, jeśli chcielibyśmy widzieć w edytorze ostrzeżenia niewłaściwego używania typu obiektu, pylint tego nie robi. Z listy dostępnych linterów (Ctrl-Shift-P czyli konsola: Python: Select Linter) wybieramy **mypy**. Otrzymamy zapewne komunikat, że nie jest zainstalowany (należy wybrać Install). Instalacja zostanie wykonana za pomocą komendy typu:

➤ "C:/Program Files/Python/python.exe" -m pip install -U mypy

Wtedy, przykładowo, próba utworzenia obiektu przez przypisanie wielkości o innym niż opisany typ, zostanie przez mypy zauważona (ale nie może zostać zabroniona):

```
main.py
1  a: int = "abc"
2
3
4
```

Incompatible types in assignment (expression has type "str", variable has type "int") mypy(error)

[View Problem](#) No quick fixes available

OBIEKTY A PAMIĘĆ

Utworzone obiekty zajmują pewien obszar pamięci. Nie wchodząc w szczegóły na ten moment, wielkość obiektu prostego dla typu wbudowanego można zbadać za pomocą funkcji **sys.getsizeof()** z modułu **sys**. Najpierw zatem importujemy ten moduł, a potem używamy funkcji za pomocą zapisu `sys.jakas_funkcja()`

```
>>> import sys
>>> sys.getsizeof(1)
28
>>> sys.getsizeof("a")
50
>>> sys.getsizeof(3.14)
24
```

Alokacja i zwalnianie pamięci w Pythonie są automatyczne. Python używa dwóch strategii, zliczanie referencji (reference counting) i odśmiecacz (garbage collector). Odśmiecanie to proces, w którym interpreter zwalnia pamięć, gdy nie jest używana, aby udostępnić ją innym obiektom. Jeśli żadne odwołanie nie wskazuje na obiekt w pamięci, tj. nie jest on używany, garbage collector automatycznie usuwa ten obiekt z pamięci. Zliczanie odwołań polega na ustaleniu ile razy do obiektu odwołują się inne obiekty w systemie. Po usunięciu odwołań zmniejsza się licznik odwołań do obiektu. Gdy licznik odwołań osiągnie zero, obiekt jest zwalniany. W Pythonie wywołania metod i referencje są przechowywane w pamięci stosu, a wszystkie obiekty wartości są przechowywane na prywatnej stercie.

TYPY PROSTE

Typ bool

Typ logiczny, posiada dwa stany **True** i **False**. Choć Python pozwala na elastyczne podejście do operacji na obiektach, nie piszmy składni, która jest jakościowo fatalna i nieczytelna. Przykłady „niepoważnych” pomysłów składniowych, które formalnie działają:

```
>>> -True
-1
>>> True+1.1+True
3.1
>>> tekst="abcdef"; tekst[False]
'a'
```

Niemniej, zdarzają się w kodzie kreatywne podejścia do zmiennych logicznych:

```
>>> True or "Cokolwiek"
True
>>> False or "Cokolwiek"
'Cokolwiek'
```

Albo jeśli chcemy poprzez warunek logiczny otrzymać tekst opisujący wariant zwróconej przez funkcję wartości **None**:

```
>>> wynik = None
>>> tekst = wynik or "Brak danych"
>>> print(tekst)
Brak danych
```

Należy również wiedzieć, że język Python dla operatorów **or** lub **and**, stosuje strategię „skróconego wyrażenia” (ang. short-circuiting) lub „leniwej ewaluacji” (ang. lazy evaluation). Operator **and** działa w ten sposób, że jeśli lewa strona jest **False**, to Python nie ocenia prawej strony, ponieważ wynik całego wyrażenia musi być **False**. Jeśli lewa strona jest **True**, to Python musi sprawdzić prawą stronę, aby określić wartość wyrażenia. Zatem:

- `True and True` → zwraca wynik ostatniej operacji, czyli prawej strony.
- `False and X` → zawsze zwraca `False`, bez potrzeby sprawdzania `X`.

Operator **or** działa w ten sposób, że jeśli lewa strona jest **True**, to Python nie ocenia prawej strony, ponieważ wynik całego wyrażenia musi być **True**. W przypadku, gdy lewa strona jest **False**, Python musi sprawdzić prawą stronę. Zatem:

- `True or X` → zwraca wynik lewej strony.
- `False or X` → musi sprawdzić `X`.

Dzięki temu mechanizmowi można optymalizować kod i unikać niepotrzebnych obliczeń, co jest szczególnie przydatne, gdy obliczenia na prawej stronie są kosztowne.

Typ int

W Pythonie wartość liczby całkowitej nie jest ograniczona liczbą bajtów i może rozszerzać się do limitu dostępnej pamięci. Wszystkie typy całkowite w Python3 to po prostu `'int'`

```
>>> import sys
>>> x = 100**10000+1 # ogromna liczba
>>> sys.getsizeof(x) # rozmiar 8884
```

Domyślną podstawą dla typu całkowitego jest podstawa dziesiętna. Z odpowiednimi przedrostkami możemy wyrazić liczbę w systemie dwójkowym, ósemkowym, bądź szesnastkowym, otrzymując w konsoli wartość przeliczoną na system dziesiętny:

```
>>> 40 # dziesiętnie
40
>>> 0b101000 # dwójkowo
40
>>> 0o50 # ósemkowo
40
>>> 0x28 # szesnastkowo
40
```

Przeliczenie do tych systemów można również wykonać za pomocą wbudowanych funkcji, takich jak `bin()`, `oct()`, czy `hex()` – funkcje te dają na wyjściu typ `str`. Konwersję łańcucha znakowego na typ całkowity o określonej bazie można wykonać operatorem `int(x=0, base=10)`, gdzie `x` to wartość lub łańcuch znakowy do konwersji na podstawę `base` (0, 2-36), baza 0 oznacza interpretację ciągu dokładnie jako literał całkowity.

Typ float

Liczby zmiennoprzecinkowe są reprezentowane w sprzęcie komputerowym jako ułamki o podstawie 2 (binarne). Niestety większość ułamków dziesiętnych nie może być reprezentowana dokładnie jako ułamki binarne. W konsekwencji liczby dziesiętne zmiennoprzecinkowe są przybliżane przez binarne liczby faktycznie przechowywane w maszynie. Dla systemu dziesiętnego, $1/3$ jest przybliżane przez $0.333...$. Podobnie w systemie dwójkowym, $1/10$ jest $0.00011001100110011...$. Na większości dzisiejszych maszyn ułamki zmiennoprzecinkowe są aproksymowane za pomocą ułamka binarnego, przy czym licznik wykorzystuje pierwsze 53 bity, zaczynając od najbardziej znaczącego bitu i mianownik jako potęgę dwójki. W przypadku $1/10$ ułamek binarny to $3602879701896397 / 2^{55}$, który jest zbliżony, ale nie do końca równy prawdziwej wartości $1/10$. Nawet jeśli wydrukowany wynik wygląda jak dokładna wartość $1/10$, rzeczywista przechowywana wartość jest najbliższym możliwym do przedstawienia ułamkiem binarnym. Co więcej, dwie różne liczby mogą mieć takie samo przybliżenie.

Sprawdźmy:

```
>>> x = 0.1
>>> y = 0.10000000000000001
>>> x.as_integer_ratio() # (3602879701896397, 36028797018963968)
>>> # mianownik to 2**55
>>> y.as_integer_ratio() # (3602879701896397, 36028797018963968)
>>> format(x, '.50f') # '0.1000000000000000055511151231257827021181583404541'
```

Z tego powodu:

```
>>> .1 + .1 + .1 == .3
False
```

Zaokrąglenie argumentów nie pomaga (nie poprawiamy reprezentacji składników):

```
>>> round(.1, 1) + round(.1, 1) + round(.1, 1) == round(.3, 1)
False
```

ale zaokrąglenie wyniku działania pomaga:

```
>>> round(.1 + .1 + .1, 10) == round(.3, 10)
True
```

1/10 nie jest dokładnie reprezentowana jako ułamek binarny. Komputery 64-bitowe używają arytmetyki zmiennoprzecinkowej IEEE-754 w „podwójnej precyzji”, a to oznacza 1 bit na znak, 11 bitów na wykładnik i pozostałe 52 bity na mantysę. Liczba to: $(-1)^{\text{znak}} * 2^{\text{wykładnik}} * \text{mantysa}$. Komputer stara się przekonwertować 0.1 na najbliższy ułamek, jaki może, w postaci $J/2^{**}N$, gdzie J i N jest liczbą całkowitą.

Sytuacja z typami float mogłaby się wydawać nieciekawa, na szczęście w Pythonie mamy moduły rozszerzające, które poprawnie radzą sobie z problemem operacji na wielkościach zmiennoprzecinkowych (moduł decimal <https://docs.python.org/3/library/decimal.html>) oraz wielkościach, które można wyrazić w postaci liczb wymiernych (moduł fractions <https://docs.python.org/3/library/fractions.html>).

Typ complex

Liczby zespolone reprezentowane przez część rzeczywistą i urojoną.

```
>>> x = conjugate(2,3) # albo: x = 2 + 3j
>>> x.real # 2
>>> x.imag # 3
>>> x.conjugate() # sprzężenie liczby zespolonej
```

Typ str

Łańcuch znakowy (typ str) to dowolnej długości niezmienny ciąg znaków w ' ' lub " ", można w jego wnętrzu użyć \ do zapisania znaków specjalnych. Zwróćmy uwagę na prezentację zawartości łańcucha gdy jest on utworzony w konsoli, oraz gdy jest argumentem funkcji print:

```
>>> "Isn\t," they said.'
>>> print("Isn\t," they said.)
```

Zmienną typu str tworzymy poprzez przypisanie do niej łańcuch znakowego.

```
>>> s = 'First line.\nSecond line.' # \n znak nowej linii
>>> s # bez print(), \n jest zawarte w wyjściu
>>> print(s) # gdy print(), \n wykonuje przejście do nowej linii
```

Surowy (raw) string jest poprzedzony literą r:

```
>>> print('C:\some\name') # tu \n oznacza nową linię
>>> print(r'C:\some\name') # r powoduje brak interpretowania zawartości
```

Stringi rozciągnięte na wiele linii za pomocą """...""" lub '"...' można również wydrukować:

```
print("""\
Usage: thingy [OPTIONS]
    -h                Display this usage message
    -H hostname       Hostname to connect to
""")
```

Stringi można łączyć operatorem `+` oraz powielać operatorem `*`. Łączenie literałów można wykonać również bez operatora `+`.

```
>>> "A"*3 + 'bbb'    # 'AAAbbb'
>>> tekst = "oto"    'przykład'
```

Indeksowanie łańcucha znakowego to nie tylko zakres od 0 do $n-1$ (gdzie n to długość łańcucha), ale również ujemne indeksy, jak na przykładzie poniżej:

A	b	C	d	E	f
0	1	2	3	4	5
-6	-5	-4	-3	-2	-1

Odwołanie się do indeksu spoza zakresu jest błędem (`IndexError: string index out of range`).

Indeksowanie może się odbywać również na zasadzie podania zakresu oraz kroku (co ile pozycji), jest to bardzo wydajny sposób na selekcję wybranych części łańcucha (slicing). Na wyciętym fragmencie można stosować wszystko to, co na łańcuchu znakowym. Zasady selekcji (założmy, że tekst jest zmienną typu `str`):

`tekst[od:]` # wycinek od pozycji o indeksie `od` – do końca

`tekst[:do]` # wycinek od początku do indeksu `do` (**bez** niego)

`tekst[od:do]` # wycinek od pozycji o indeksie `od` do indeksu `do` (**wyłączając**)

`tekst[:]` # cały łańcuch tekst

`tekst[::+krok]` # wybranie co któryś (krok) znak, jeśli ujemny to idziemy od tyłu

Przykład (warto się przyjrzeć i poeksperymentować):

```
>>> tekst = "AbCdEfGhIjKlMn" # indeksy od 0 do 13, od -14 do -1
>>> tekst[3:] # 'dEfGhIjKlMn'
>>> tekst[:6] # 'AbCdEf'
>>> tekst[4:8] # 'EfGh'
>>> tekst[:2] # 'ACEGIKM'
>>> tekst[::-1] # 'nMlKjIhGfEdCbA'
>>> tekst[::-2] # 'nljhfdb'
>>> tekst[-4:-1] # 'KlM'
>>> tekst[2:-7] # 'CdEfG'
>>> tekst[-10:-1:2] # 'EGIKM'
>>> tekst[10:1:-2] # 'KIGEC'
>>> tekst[-3:-11:-2] # 'ljhf'
>>> tekst[0:10][2:4] # 'Cd'
```

Wczytanie łańcucha z urządzenia standardowego (klawiatury) wykonujemy za pomocą funkcji `input`, która (opcjonalnie) może jako argument mieć wyświetlony tekst zachęty:

```
>>> tekst = input("Jak sie nazywasz: ")
>>> Jak sie nazywasz: Jan
>>> tekst    # 'Jan'
```

Długość obiektu (liczbę elementów w kolekcji) zwraca funkcja `len()`, w tym przypadku długość łańcucha znakowego, jeśli pusty, to wynosi 0. Typ `str` umożliwia wykonanie szeregu operacji (wywołanie funkcji), które nie zmieniają oryginalnego obiektu, tylko zwracają zmodyfikowaną kopię. Katalog jest tu <https://docs.python.org/3/library/stdtypes.html#string-methods>

Kilka wybranych przykładów do sprawdzenia:

```
>>> x = "  abcd ab efgh ab "
>>> x.upper()
>>> x.lower()
>>> x.strip() # usuwa białe znaki na początku i końcu
>>> x.replace("ab","AA") # wszystkie wystąpienia, ale można na x zrobić slice
>>> x.split(" ") # rozбивa na elementy listy po separatorze - tu spacji
>>> x.find("AA") # pierwsze wystąpienie "AA", ale jako 2 i 3 parametr można podać zakres
>>> x.index("AA") # to samo co find, gdy find nie znajdzie, zwróci -1, zaś index zgłosi wyjątek
```

W Pythonie nie ma typu reprezentującego pojedynczy znak (char). Nawet jeden znak w cudzysłowie to jest typ str. Kod ASCII danego znaku w standardzie Unicode można pozyskać za pomocą funkcji `ord()`, zaś odczytać znak za pomocą funkcji `chr()` – zakres argumentów jest od 0 do 1114111. Na przykład, `ord('a')` to 97, a `chr(2620)` to znaczek czaszki ☠️ (czy się wyświetli to zależy od zastosowanego fontu).

Obszerne informacje o typie str oraz możliwościach jego formatowania <https://docs.python.org/3/library/string.html>

omówimy przy innej okazji.

OBIEKTY NIEMODYFIKOWALNE, IDENTYFIKACJA, USUWANIE

Obiekty prostych typów zaprezentowane powyżej, są niemodyfikowalne. Możemy być tym zaskoczeni, ponieważ w wielu językach programowania obiekt określonego typu powstaje i zajmuje określone miejsce (adres) w pamięci, niezależnie od tego, jaką ma zawartość. W Pythonie wszystko jest obiektem, unikatową sygnaturę obiektu można zbadać za pomocą funkcji `id()`. Przykład, zwróćmy uwagę, że po kolejnym przypisaniu innej wartości zmienia się adres, czyli powstaje kopia obiektu pod innym adresem, a nie modyfikacja zawartości obiektu, który wcześniej powstał:

```
>>> a, b, c, d, e = 1, 2.34, "abc", True, 1+2j
>>> print(id(a), id(b), id(c), id(d), id(e))
1549640165680 1549679852816 1549641109424 140709478197352 1549680549680
>>> a, b, c, d, e = 2, 3.34, "efg", False, 2+1j
>>> print(id(a), id(b), id(c), id(d), id(e))
1549640165712 1549679852240 1549681490544 140709478197384 1549680546160
```

Jak widać, konkretne nazwy (a, b, c...) wskazują raczej na adres w pamięci, w którym znajduje się ich wartość (też traktowana jako obiekt). Jeśli więc jakaś wartość nie ma dłużej dowiązana, jest przez Python automatycznie z pamięci usuwana. Przystudiujmy taki fragment:

```
>>> a = "xyz" # tworzymy obiekt a
>>> id("xyz") # sprawdzamy adres ciągu "xyz"
1549681491504
>>> id(a) # sprawdzamy adres obiektu a, jest taki sam jak "xyz"
1549681491504
>>> b = a # tworzymy obiekt b z wartością a
>>> id(b) # sprawdzamy adres obiektu b, jest taki sam jak a
1549681491504
>>> a = "aaa" # zmieniamy zawartość a
>>> id(a) # jak widać, zmienił się adres, czyli a pokazuje teraz na inny obszar pamięci
1549681490864
>>> b = a # ponownie przypisujemy wartość a do b
```

```
>>> id(b) # b ma teraz adres taki jak a (czy raczej łańcuch znakowy "aaa")
1549681490864
>>> id("xyz") # co się stało z łańcuchem znakowym "xyz"?
1549681491312 # inny adres!
```

Jak widać, poprzedni łańcuch "xyz" został usunięty, gdyż ani obiekt a, ani obiekt b, już na niego nie wskazywały, a skoro teraz pytamy, to utworzono w pamięci nowy "xyz", pod nowym adresem. Adresy (prezentowane dziesiętnie) można oczywiście rzutować poprzez `hex()`.

W przypadku niewielkich wartości typu `int`, z zakresu `[-5, 256]`, Python wykonuje pewną optymalizację – tworzy wstępnie obiekty o takiej wartości i potem ich używa. Na przykład, jeśli utworzę dwa obiekty niezależnie, o tej samej wartości 257, to mają one różne adresy:

```
>>> a = 257
>>> b = 257
>>> print(id(a), id(b))
1840771930992 1840770948976
```

Jeśli jednak to samo zrobię dla wartości 256, obiekty mają identyczny adres:

```
>>> a = 256
>>> b = 256
>>> print(id(a), id(b))
1840729581968 1840729581968
```

Można również odczytać (zrutować) wskazany adres pamięci jako obiekt Pythona, korzystając z zewnętrznego modułu `ctypes`. Ilustrujący fragment kodu:

```
import ctypes
a = "abcd" # spróbuj z innymi typami
print("Wartosc -", a)
addr_a = id(a)
print("Adres pamieci - ", addr_a)
b = ctypes.cast(addr_a, ctypes.py_object).value
print("Wartosc odczytana z pamieci - ", b)
```

Choć zarządzanie czasem życia obiektów w Pythonie jest zautomatyzowane, istnieje słowo kluczowe `del`, można w ten sposób usuwać obiekty, elementy złożonych obiektów (np. list, słowników), czy nawet definicji klas. Składnia jest prosta: `del nazwa_usuwanego_elementu`.

OPERATORY, WYRAŻENIA KONTROLNE, PĘTLE

Zanim przejdziemy do omówienia typów złożonych, szybki przegląd operatorów oraz podstawowych składni, które są powszechne w praktycznie każdym języku programowania.

Wyrażenie `if`, `else if`

Kluczową rolę grają odpowiednie wcięcia kodu. Wyrażenia `if` mogą być zagnieżdżone, ale odpowiednie głębokości wcięć muszą być zachowane.

```
x = int(input("Wpisz liczbę całkowitą: "))
if x < 0:
```

```

x = 0
print('Ujemna zmieniona na zero')
elif x == 0:
    print('Zero')
elif x == 1:
    print('Jeden')
else:
    print('Wiecej niz jeden')

```

Operator trójskładnikowy

W wielu językach programowania istnieje operator trójargumentowy, jego idea polega na tym, że spełnienie warunku logicznego pociąga za sobą zwrócenie pierwszego obiektu, a niespełnienie – drugiego obiektu. Dokładnie to samo można osiągnąć za pomocą instrukcji `if... elif`, niemniej chodzi o elegancję i zwięzłość zapisu. W Pythonie rolę taką pełni składnia według następującego porządku: `value_if_true if condition else value_if_false`. Jak widać, badany warunek jest pośrodku, po lewej przed `if` jest wartość zwracana przy spełnieniu warunku, a po prawej za `else` wartość zwracana przy niespełnieniu warunku:

```

>>> warunek = True
>>> wynik = "spełniony" if warunek else "niespełniony"
>>> print(wynik)
spełniony

```

Możliwe jest zagnieżdżanie trójskładnikowych instrukcji warunkowych, trzeba przy tym uważać, żeby zapisana logika odpowiadała rzeczywiście naszym zamiarom. Przykład z podwójnie zagnieżdżonym warunkiem `if`:

```

a = False
b = True
if a:
    print("1")
else:
    if not b:
        print("2")
    else:
        print("3") # ten warunek spełniony

```

Kod ten można zapisać skrótowo:

```

c = "1" if a else "2" if not b else "3"
print(c)

```

Inny przykład, z trzech liczb chcemy wskazać największą:

```

a, b, c = 4, 5, 1
max_abc = a if a > b and a > c else b if b > c else c
print(max_abc)

```

Nieczytelność kodu może być pogłębiona poprzez modyfikowanie obiektów, na których wyliczane są wartości logiczne. Za pomocą operatora przypisania wyrażenia `:=` (na jego temat więcej w dalszej części dokumentu) można zmieniać „w locie” wartość, czyniąc całe wyrażenie mało przejrzystym i podatnym na błędy. Dodatkowo operator `:=` wymaga zastosowania nawiasów `()` ze względu na priorytety operacji:


```
b = 1 # dla jakiej wartości b jakie a
a = b if (b:=b-1) else (b:=-10)
print(a)
```

Pętla for

Chyba najpopularniejsza struktura językowa, w wersji po pełnym zakresie jakiegoś obiektu złożonego (kontenera, tablicy), tutaj na przykładzie łańcucha znakowego.

```
a = "abcdefgh"
for i in a:
    print(i, end=" ")
# a b c d e f g h
```

Obiekt o nazwie `i` (przyzwyczajenie, możemy użyć dowolnej nazwy) w pętli, staje się na moment jednej iteracji obiektem będącym referencją do poszczególnych obiektów (tu jednoliterowych łańcuchów znakowych), każdy z nich ma swoje (chwilowe) miejsce w pamięci:

```
>>> for i in "abcd":
        print(i, id(i))

a 1840736726576
b 1840736399216
c 1840731209776
d 1840731201264
```

Pętla `for` w Pythonie posiada również wersję `for... else`, gdzie blok `else` wykonany zostanie w przypadku, gdy pętla całkowicie zakończy swoje działanie (nie zostanie przerwana np. przez `break` – ilustracja tego będzie w dalszej części). `else` powinien być w takim samym wcięciu, co `for`, na co trzeba uważać zwłaszcza, gdy dana pętla `for` jest zagnieżdżona np. wewnątrz innej pętli `for`.

```
for i in "abc":
    print(i, end=" ")
else:
    print("poprawny koniec")
# a b c poprawny koniec
```

Zazwyczaj jesteśmy przyzwyczajeni do pętli, która używa jakiejś zmiennej indeksującej. Nie jest to konieczne, ale jeśli potrzebujemy dodatkowo indeksu, najlepiej użyć wbudowaną funkcję `enumerate()`. Zwraca ona sekwencję par argumentów, oba można pobrać w pętli (jest to przykład rozpakowywania przekazanych danych, tak jak przypisanie wielu obiektom wielu różnych wartości, w jednej linii z operatorem przypisania).

```
a = "abcdefgh"
for i, j in enumerate(a):
    print(i, j, end="; ")
# 0 a; 1 b; 2 c; 3 d; 4 e; 5 f; 6 g; 7 h;
```

Funkcja `enumerate()` posiada również nazwany argument `start`, za pomocą którego możemy zdefiniować początkową wartość (różną od domyślnego 0), czyli `enumerate(a, start=1)`. A co jeśli w kodzie pętli nie wpisujemy dwóch obiektów `i, j` tylko jeden?

```
a = "abcdefgh"
for i in enumerate(a):
    print(i, end="; ")
```

Kod jest nadal całkowicie poprawny, ale obiektowi `i` będą przypisane obiekty krotki (class 'tuple'), czyli pary wartości: (0, 'a'); (1, 'b'); (2, 'c'); (3, 'd'); (4, 'e'); (5, 'f'); (6, 'g'); (7, 'h');, które można również „rozpakować” poprzez przypisanie do dwóch nazwanych obiektów, czyli np. `x, y = i` (w powyższym kodzie w pętli). Typ „tuple” omówimy później. Oczywiście wyniki kolejnych wołań `print` są przedzielone średnikiem `;` podanym jako argument nazwany `end` w funkcji.

Funkcja `enumerate()` zwraca typ iterowalny czyli taki, po którym może przejść iterator, wydobywając kolejne elementy. Przejście iteratora jest jednorazowe, raz odczytany element jest usuwany z obiektu zwróconego przez `enumerate()`. Można to tak zademonstrować:

```
>>> x = enumerate("abcd")
>>> for i in x: print(i, end=" ")

(0, 'a') (1, 'b') (2, 'c') (3, 'd')
>>> for i in x: print(i, end=" ")
```

Dokładnie jak widać powyżej, drugie wykonanie pętli `for` na obiekcie `x` nie daje żadnego wywołania `print`, gdyż `x` jest po pierwszym wykonaniu pętli `for` już pusty. Ta osobliwa własność wielkości iterowalnej sprawia, że nie da się „odpytać” ile elementów zawiera obiekt iterowalny, gdyż wymaga to przejścia przez wszystkie elementy. A takie przejście powoduje, że pierwotny obiekt iterowalny staje się pusty i drugi raz już po nim nie przejdziemy. Można oczywiście rozpakować zawartość do innego kontenera, np. listy, ale tylko raz:

```
>>> x = enumerate("abcd")
>>> len(list(x))
4
>>> len(list(x))
0
```

Za drugim razem, jak widać, rozpakowanie daje liczbę elementów 0. Nie rozpakowując całej zawartości z obiektu iterowalnego, można wykonać przejście po jego elementach (nadal jednorazowe), wykonując sumę liczby 1 tylekroć, ile jest elementów do iteracji. Sprytny zapis (o takiej składni powiemy niebawem), z użyciem wbudowanej funkcji Pythona `sum()` wygląda następująco:

```
>>> x = enumerate("abcd")
>>> sum(1 for _ in x)
4
>>> sum(1 for _ in x)
0
```

Jeszcze raz widać, że drugi przebieg daje wartość 0. Znaczek podkreślenia `_` użyty w pętli `for` to nie jest żaden tajemny trik, tylko zwykła nazwa zmiennej, którą równie dobrze możemy nazwać zwyczajowo `i`, albo `n`. Natomiast jest taki obyczaj, że znacznikiem `_` nazywamy obiekty, których zawartości de facto nie potrzebujemy. W powyższej sumie sumujemy bowiem cały czas wartość 1, ale tyle razy, ile pętla `for` wykonała iteracji, a sama pętla `for` potrzebuje jakiejś zmiennej.

Konsumowanie wielkości iterowalnej może się odbywać oczywiście na raty. W obiekcie iterowanym jest tyle elementów, ile jeszcze nie zostało odczytanych.

```
>>> x = enumerate("abcdef")
>>> for i, j in x:
    if j=="c":
        print(i,j)
        break
    else:
        print(i,j)

0 a
1 b
2 c
>>> for i, j in x:
    if j=="c":
        print(i,j)
        break
    else:
        print(i,j)

3 d
4 e
5 f
```

Generator `range`

Funkcja `range()` to typ generatora, zwracającego sekwencję liczb całkowitych. Od razu dodajmy, że w zewnętrznym module NumPy jest dostępna również wersja dla wartości zmiennoprzecinkowych. Składnia `range()` zawiera do trzech parametrów. Z jednym parametrem oznacza ile elementów (zaczynając od 0 i z krokiem 1) wygenerować: `range(5)` wyprodukuje 0, 1, 2, 3, 4, ale jeśli ich nie „rozpakujemy” to zobaczymy:

```
>>> range(5)
range(0, 5)
```

Innymi słowy jeden argument oznacza również górny zakres, zaczynając od 0. Dlaczego nie widzimy sekwencji liczb? Ponieważ (tu mądre zdanie z Wikipedii) generatory są w Pythonie mechanizmem leniwej ewaluacji funkcji, która w przeciwnym razie musiałaby zwracać obciążającą pamięć lub kosztowną w obliczaniu listę.

Z dwoma parametrami, podajemy zakres: od - do, wyłączając górną wartość z ciągu wygenerowanych liczb (z krokiem 1). Z trzema, możemy dodatkowo podać krok.

```
>>> for i in range(1,10,2):
        print(i)

1
3
5
7
9
```

Generator jest obiektem przechowującym stan, mogącym wielokrotnie wchodzić do i opuszczać ten sam dynamiczny zakres. Wywołanie generatora może być użyte zamiast listy lub innej struktury, po której elementach będziemy iterować. Za każdym razem, gdy pętla for w powyższym przykładzie potrzebuje następnego elementu, wywoływany jest generator, który daje następny element. Oczywiście da się wydobyć z generatora wszystkie elementy i albo je rozpakować do tylu obiektów, ile wartości wygeneruje generator, albo utworzyć obiekt złożony. Jeszcze nie znamy typów złożonych, ale dla listy wyglądałoby to tak: `i = list(range(5))`

Krok w generatorze **range** może być ujemny, argumenty mogą być odpowiednie (od większej wartości, do mniejszej):

```
>>> for i in range(10,1,-2):
        print(i)

10
8
6
4
2
```

Do elementów z generatora można odnieść się przez indeks, można też użyć selekcji zakresowych, o których wcześniej już napisano. Wynikiem będzie albo int:

```
>>> range(3,12)[2]
5
```

albo kolejny obiekt **range**:

```
>>> range(3,12)[2:4]
range(5, 7)
```

Można też odwrócić w ten sposób kolejność (poniżej oznacza to sekwencję od 11 do 3):

```
>>> range(3,12)[::-1]
range(11, 2, -1)
```

Kolejność można odwrócić również za pomocą jednej z wbudowanych funkcji Pythona o nazwie **reversed()**, zwraca ona obiekt typu iterowalnego (class 'range_iterator').

```
>>> for i in reversed(range(5)):
    print(i, end = " ")

4 3 2 1 0
```

Czym się różni obiekt iterowalny od obiektu generatora? Obydwa stosują strategię leniwej ewaluacji. Jednak obiekt generatora jest dostępny bez ograniczeń, możemy przez niego przechodzić wielokrotnie. Przytoczmy przykład:

```
>>> x = range(5)
>>> sum(1 for _ in x)
5
>>> sum(1 for _ in x)
5
```

Jak widać pierwsze i drugie przejście po elementach x daje taki sam wynik. Gdybyśmy jednak zastosowali **reversed()** i tym samym otrzymali obiekt iterowalny:

```
>>> x = reversed(range(5))
>>> sum(1 for _ in x)
5
>>> sum(1 for _ in x)
0
```

to od razu widzimy, że przejście przez taki obiekt może być wykonane tylko raz. Możemy sprawdzić wielkość obiektu generatora, możemy się też odwołać przez indeks (lub zakres indeksów) do dowolnej jego części – nie było to możliwe w przypadku iteratorów. Obiekt iterator może być przekazany do funkcji wbudowanej Pythona o nazwie **next()**, która zwraca (konsumuje) kolejne obiekty iteratora, aż na koniec (jeśli nie przestaniemy) zgłosi wyjątek:

```
>>> x = reversed("abc")
>>> next(x)
'c'
>>> next(x)
'b'
>>> next(x)
'a'
>>> next(x)
Traceback (most recent call last):
  File "<pyshell#313>", line 1, in <module>
    next(x)
StopIteration
```

Obiektu generatora nie da się przekazać do funkcji next().

```
>>> x = range(5)
>>> next(x)
Traceback (most recent call last):
  File "<pyshell#322>", line 1, in <module>
    next(x)
TypeError: 'range' object is not an iterator
```

Czy można reversed() wykorzystać do odwrócenia i porównania łańcucha znakowego, czy jest palindromem? Można, ale ponieważ reversed() tworzy iterator (pamiętajmy, leniwa ewaluacja), to nie można ot tak porównać:

```
>>> x = "KayaK"
>>> x == reversed(x)
False
```

Natomiast można sprytnie wykonać iterację i połączenie elementów iterowalnych za pomocą funkcji **join()**. Działa ona na obiekcie typu str, który będzie separatorem poszczególnych elementów, po których iterujemy. Przykład:

```
>>> "---".join("abcd")
'a---b---c---d'
```

Widzimy, że "---" jest separatorem, a join() skleja kolejne iterowane elementy, przedzielone właśnie łańcuchem - separatorem. Stosując pusty łańcuch znakowy jako separator, możemy w ten sposób wytworzyć łańcuch znakowy z obiektu iteratora, zatem jeszcze raz:

```
>>> x = "KayaK"
>>> x == "".join(reversed(x))
True
```

Teraz sprawdzenie zakończyło się powodzeniem, słowo **KayaK** jest palindromem. Pamiętajmy jednak, że przebiegnięcie po obiekcie iteratorze jest jednorazowe. Zatem już wiemy, dlaczego drugie wywołanie operatora == w poniższym przykładzie daje wynik False:

```
>>> x = "KayaK"
>>> y = reversed(x)
>>> x == "".join(y)
True
>>> x == "".join(y)
False
```

Sprawdzenie czy łańcuch znakowy jest palindromem, można zrobić na wiele sposobów, chyba najprościej za pomocą odpowiedniego „slicingu”:

```
>>> x = "KayaK"
>>> x == x[::-1]
True
>>> "KayaK" == "KayaK"[::-1]
True
```

Pętla while

Dopóki warunek logiczny jest spełniony, blok należący do while (czyli odpowiednio wcięty), jest wykonywany.

```
gwiazdki = "*" * 10
while gwiazdki:      #odwrócona połówka trójkąta
    print(gwiazdki)
    gwiazdki = gwiazdki[1:] # obcinamy pierwszą gwiazdkę
```

Pętla **while** posiada również możliwość dodania bloku **else**: po całkowitym jej wykonaniu, czyli jeśli nie zostanie przerwana za pomocą **break** (patrz poniżej):

```
a = 1
while a < 3:
    print("a = ", a, end = " ")
    a += 1
else:
    print("wykonano!")
# a = 1 a = 2 wykonano!
```

Break, continue, pass

Są to słowa kluczowe języka Python, których znaczenie zapewne dobrze znamy. Kilka przykładów ilustrujących działanie.

```
for n in range(2, 12):
    for x in range(2, n):
        if n % x == 0:
            print(n, "rowna sie ", x, '*', n//x)
            break
    else:
        # to jest wyświetlone gdy petla for sie zakonczy
        print(n, "jest liczba pierwsza")
```

W wyniku działania otrzymamy:

```

2  jest liczba pierwsza
3  jest liczba pierwsza
4  rowna sie  2 * 2
5  jest liczba pierwsza
6  rowna sie  2 * 3
7  jest liczba pierwsza
8  rowna sie  2 * 4
9  rowna sie  3 * 3
10 rowna sie  2 * 5
11 jest liczba pierwsza

```

Zwróćmy uwagę na wcięcia w powyższym programie. Wykorzystuje on składnię **for... else**. W powyższym programie **else** należy do drugiej, zagnieżdżonej pętli, która, jeśli znajdzie dzielnik jakiejś liczby, to przerywa pętlę **for** (za pomocą **break**), a jeśli nie znajdzie i się zakończy, przechodzi do **else** i wypisuje, że dana liczba jest liczbą pierwszą.

```

for num in range(2, 10):
    if num % 2 == 0:
        print("Liczba parzysta", num)
        continue # rowniez jako opuszczenie reszty iteracji za continue
    print("Liczba nieparzysta", num)

```

Powyższy program sprawdza, czy liczba ma resztę z dzielenia przez 2, jeśli nie ma, to jest parzysta i dalsza część kodu nie jest wykonywana, po wydaniu komendy **continue** (przeskoczenie do końca bloku danej iteracji i przejście do kolejnej iteracji).

```

while True:
    pass # pusta instrukcja

```

Powyższa pętla jest nieskończona, aż do zatrzymania programu za pomocą Ctrl-C. Komendę **pass** używa się często w kodzie, który później zostanie napisany, a początkowo wymaga instrukcji pustej.

Operatory

Zestaw operatorów w języku Python jest bardzo podobny do tych spotykanych w innych językach programowania.

```

(arytmetyczne) + - * / % ** //
(przypisanie) = += -= *= /= %= //= **= &= |= ^= >>= <<=
(porównanie) == != > < >= <=
(logiczne) and or not
(bitowe) & | ^ ~ >> <<
(identyczności) is is not
(przynależności) in not in
(przypisanie w wyrażeniu) :=

```

Warto przypomnieć, że operator dzielenia **/** wykonuje dzielenie rzeczywiste (zmiennoprzecinkowe), a dzielenie z odcięciem części ułamkowej wykonuje się operatorem **//**. W Pythonie mamy również operator potęgowania ******, nieobecny w językach C/C++. Nie ma za to operatorów inkrementacji **++** i dekrementacji **--**. Poniższy kod jest poprawny:


```
>>> a = 1
>>> ++a
1
>>> --a
1
>>> +++a
1
>>> ---a
-1
>>> 1
```

lecz nie wynika to z faktu obecności „preinkrementacji” czy „predekrementacji”, tylko wielokrotnego zastosowania operatorów + (nic nie zmienia) i – (zmienia znak na przeciwny), które można w dowolnej ilości umieszczać *przed* obiektem. Taki sam „trik” z operatorem napisanym za obiektem będzie oczywiście zgłoszony jako błąd.

W Pythonie składanie operatorów porównania (takich jak <, >, <=, >=) jest bardziej intuicyjne w porównaniu do wielu innych języków programowania, ponieważ działa ono jak w matematyce. Wyrażenie takie jak `s < d < f` jest oceniane w sposób „łańcuchowy”, co oznacza, że Python rozumie to wyrażenie jako jedno logiczne porównanie, a nie jako dwa oddzielne porównania. Wyrażenie `s < d < f` jest równoważne z `s < d and d < f`. Ważną cechą tego typu zapisu jest to, że Python nie ewaluje wartości `d` dwukrotnie, tylko raz. `d` jest sprawdzane jako wartość pomiędzy `s` a `f` w sposób ciągły. W tradycyjnym zapisie, takim jak `s < d and d < f`, wartość `d` jest oceniana dwukrotnie, co może mieć znaczenie, jeśli obliczenie `d` jest kosztowne lub jeśli wywołuje efekty uboczne. Przykładowo:

```
def kosztowna_operacja():
    print("Obliczam d...")
    return 5

s = 2
f = 10

print(s < kosztowna_operacja() < f) # Obliczam d... (tylko raz)
```

funkcja `kosztowna_operacja()` zostanie wywołana tylko **raz**, mimo że wynik (`d`) bierze udział w dwóch porównaniach (`s < d` i `d < f`).

Warto zwrócić szczególną uwagę na priorytet operatorów, ponieważ mogą one wpływać na kolejność obliczeń i ostateczny wynik wyrażenia. Składnia Pythona jest elastyczna, a operatorzy mają różne priorytety, co może prowadzić do nieoczekiwanych rezultatów, jeśli nie rozumiemy, jak Python wykonuje obliczenia. Rozważmy przykładowe wyrażenie:

```
print(b + c <= e or f + g >= e == f == 5) # False, chyba, że g == 0 lub b+c<=e jest True
```

Najpierw wykonujemy `b + c`, a następnie porównanie `<= e`. Wynik tej części może być `True` lub `False`. Operator `or` jest leniwie oceniany (short-circuiting). Jeśli lewa strona wyrażenia jest `True`, Python nie ocenia dalszej części, a wynik jest `True`. Jeśli lewa strona (`b + c <= e`) jest `False`, Python musi przejść do oceny prawej strony. Ta część jest bardziej złożona, ponieważ zawiera kilka porównań łańcuchowych. Python traktuje to wyrażenie jako: `(f + g >= e)`

`and (e == f) and (f == 5)`. Porównania łańcuchowe (`>=`, `==`) są oceniane od lewej do prawej i są połączone w jedno wyrażenie logiczne. Oznacza to, że wszystkie warunki muszą być spełnione, aby całe wyrażenie zwróciło `True`. Jest to możliwe tylko, gdy `g == 0`.

We wcześniejszych przykładach widzieliśmy, że operator przypisania potrafi działać na całym zestawie nazw obiektów (z lewej strony) i wartości inicjalizujących różnych typów (z prawej strony). Można również wykonać „rozpakowanie” z różnych kontenerów, czy nawet łańcucha znakowego, do typów prostych:

```
>>> x, z, z = "abc"
>>> print(x, y, z)
a b c
```

lub wykonać operację zamiany zawartości dwóch zmiennych (swap) w sposób niezwykle prosty:

```
>>> a = 10; b = "abrakadabra"
>>> print(id(a), id(b))
1840729385552 1840770075504
>>> a, b = b, a
>>> print(a, b)
abrakadabra 10
>>> print(id(a), id(b))
1840770075504 1840729385552
```

Jak widać, po adresach, zamienione obiekty to są te same obiekty. W wykonaniu takiej operacji na pewno pomaga fakt, że w pamięci jest zawartość (odpowiednio, 10 i 'abrakadabra'), do której dowiązana jest referencja obiektu `a` lub `b`. Wystarczy więc zamienić referencje.

Nowością (od Python 3.8) jest „morsowy” operator przypisania wyrażenia `:=` (walrus operator), dzięki któremu możliwa jest operacja przypisania w takich miejscach, w których (tak jak to może się dziać w C/C++) pomylenie operatora `==` z operatorem `=` prowadziło do błędu działania, ale nie błędu składniowego, gdyż operator przypisania był zarazem wyrażeniem o określonej wartości, najczęściej ewaluowanej jako „prawda” logiczna. Dlatego w Pythonie zwykłe przypisanie `=` nie może być procedowane wewnątrz instrukcji warunkowych. Uznano natomiast, że można w takiej sytuacji wprowadzić specjalny, nowy operator, bo jego użycie będzie świadczyło o tym, że zapisano go całkowicie świadomie. I odwrotnie, przypisanie `:=` nie można wykonać w miejscach, gdzie należy użyć zwykłego operatora `=`. Objęcie nawiasami operacji przypisania sprawia, że staje się ona wyrażeniem, z możliwością użycia `:=`. Poniżej kilka przykładów poprawnego zastosowania zarówno operatora `=` jak i operatora `:=`:

```

>>> a = 1
>>> (a = 1)
SyntaxError: invalid syntax
>>> b := a
SyntaxError: invalid syntax
>>> (b := a)
1
>>> if (a == 1): pass

>>> if (b = a): pass
SyntaxError: invalid syntax
>>> if (b := a): pass

```

Operator ten stosuje się w wielu „efekownych” skrótowych zapisach składni, które są charakterystyczne dla Pythona (jeden z przykładów pojawił się już wcześniej w tym dokumencie). Trzeba jednak pamiętać, że jego priorytet jest niższy od pozostałych operatorów, zatem poniższy kod spowoduje najpierw ewaluację całego wyrażenia: `a ** 2 > 6`, które (dla `a` o wartości 3) ma wartość `True`:

```

>>> a = 3
>>> if kwadrat := a ** 2 > 6:
    print(kwadrat)

True

```

Tymczasem, poprawny zapis, zgodny z zamiarem, będzie:

```

>>> a = 3
>>> if (kwadrat := a ** 2) > 6:
    print(kwadrat)

9

```

Przechodząc do operatorów „`is`”, „`is not`” oraz „`in`”, „`not in`”, warto sprecyzować obszar ich zastosowania. Operator `==` porównuje wartość lub równość dwóch obiektów, natomiast operator `is` w Pythonie sprawdza, czy dwie zmienne wskazują na ten sam obiekt w pamięci. Oznacza to, że zazwyczaj używamy operatorów równości `==` i `!=`, chyba że porównujemy z `None`. Zobaczmy ponownie przykład, w którym wiadomo, że wartość `-5` jest przez Pythona tworzona w ramach optymalizacji (liczby całkowite `-5` do `256`) i obiekty o takiej wartości mają ten sam adres, zaś dla innej, na przykład `-6` już tak być nie musi (ale może!), stąd wyniki logiczne jak poniżej. Zatem, operatory w przykładzie sprawdzają, odpowiednio, równość lub nierówność (`==` lub `!=`) oraz identyczność bądź nie (`is` lub `is not`).

```

>>> a = -5
>>> b = -5
>>> a == b
True
>>> a is b
True
>>> a = -6
>>> b = -6
>>> a == b
True
>>> a is b
False

```

W module `sys` jest funkcja `intern(string)`, która umieszcza argument – obiekt typu `str` w tablicy łańcuchów znakowych „utrwalonych” w pamięci i zwraca taki właśnie łańcuch (może to być kopia lub ten sam obiekt co argument). Taki zabieg, podobny to „utrwalonych” liczb całkowitych, poprawia nieco wydajność niektórych operacji, zwłaszcza w przypadku struktury typu słownik: gdy klucze w słowniku są „utrwalone”, to ich porównywanie (po haszowaniu) może być wykonane na poziomie porównań wskaźników zamiast porównań całych łańcuchów znakowych. W kontekście operatora `is` możemy zatem mieć całe łańcuchy znakowe umieszczone w tym samym miejscu w pamięci.

Jeśli więc kolejny łańcuch znakowy (w przykładzie o nazwie `b`) będzie utworzony za pomocą `intern`, to język sprawdzi, czy taki łańcuch nie jest już zachowany i wtedy powiąże nową nazwę z istniejącą wielkością w pamięci.

```

>>> import sys
>>> a = "wlazl kotek na plotek"
>>> id(a)
1840771892848
>>> a = sys.intern(a)
>>> id(a)
1840771892848
>>> b = sys.intern("wlazl kotek na plotek")
>>> id(b)
1840771892848
>>> a is b
True

```

Jeśli jednak utworzymy kolejny łańcuch znakowy (taki sam) nie za pomocą `intern`, to jego adres będzie inny, ale po zastosowaniu na nim funkcji `intern`, zostanie podstawiony adres istniejącego i zachowanego („interned”) wcześniej obiektu:

```
>>> c = "wlazl kotek na plotek"
>>> id(c)
1840771819120
>>> a == c
True
>>> a is c
False
>>> c = sys.intern(c)
>>> a is c
True
```

W większości przypadków, odrębnie utworzone obiekty nawet o tej samej wartości, będą w różnych miejscach pamięci, zatem operator `is` zwróci `False`. Jeśli obiekty utworzymy poprzez operator przypisania, np. `a = b = "abc"`, albo kolejno, `a = "abc"`, `b = a`, to `a` i `b` będą w tym samym miejscu w pamięci. W takiej sytuacji, w przypadku obiektów modyfikowalnych, może nas spotkać niespodzianka. Do tej pory mieliśmy do czynienia z obiektami niemodyfikowalnymi, ale na chwilę, zanim typ lista omówimy szczegółowo, popatrzymy na przykład. Obiekt typu lista to kolekcja innych obiektów (dowolnego typu) i jest to wielkość modyfikowalna. Jeśli teraz utworzymy pod dwiema nazwami obiekt listy w tym samym miejscu w pamięci, modyfikacja jednego z nich pociągnie modyfikację drugiego:

```
>>> a = [1, 2, 3] # lista obiektów int
>>> b = a # b to też lista taka jak a
>>> a is b # ten sam obszar pamięci
True
>>> print(a,b) # ta sama zawartość
[1, 2, 3] [1, 2, 3]
>>> a.append(4) # dodajemy element na końcu listy a
>>> print(a,b) # nadal ta sama zawartość
[1, 2, 3, 4] [1, 2, 3, 4]
```

Na podsumowanie, przykłady z `is` (`is not`) oraz `in` (`not in`) dla prostych obiektów listy.

```
x = ["jablko", "banan"]
y = ["jablko", "banan"]
z = x
print(x is z) # True, gdyż x i z są w tym samym miejscu pamięci
print(x is y) # False, x i y nie są w tym samym miejscu pamięci
print(x == y) # True, bo zawartość x i y jest taka sama
print(x is not z) # False
print(x is not y) # True
print(x != y) # False
print("banan" in x) # True gdyż element "banan" znajduje się w x
print("arbuz" not in x) # True gdyż element "arbuz" nie znajduje się w x
```

Demonstrację działania `in` (`not in`) można oczywiście sprawdzić również na łańcuchach znakowych (proszę przetestować!).

Pola wyboru `match`

W Python 3.10 została dodana struktura językowa dopasowania do pól wielokrotnego wyboru, która często funkcjonuje w innych językach pod nazwą `switch .. case`. Jej ogólna składnia to:

```
match obiekt:
    case <wartosc1>:
        <akcja1>
    case <wartosc2>:
        <akcja2>
    case _:
        <akcja domyslna>
```

Przykład poniżej spowoduje wywołanie `print(3)`, gdyby nie było przypadku pasującego, weszlibyśmy do pola `_`. W miejscach `<akcja>` może być dowolny kod, może być instrukcja `return`, jeśli cały fragment kodu jest częścią funkcji.

```
t = "raz"
match t:
    case "jeden":
        print(1)
    case "dwa":
        print(2)
    case "raz":
        print(3)
    case _:
        print("a jednak")
```

Wartości poszczególnych pól mogą być różnych typów:

```
t = 2 # również t = 3 zostanie zaliczone do przypadku 3.0
match t:
    case "jeden":
        print(1)
    case 2:
        print("dwa int")
    case 3.0:
        print("trzy float")
```

Wartości pól wyborów mogą być wielokrotne połączone `|` (or):

```
case "jeden" | 2:
    print(1, "lub dwa int")
```

Mogą tam wystąpić również typy złożone, nawet w połączeniu z symbolem `_` (traktowanym jako „wildcard” wieloznacznik). Omówienie innych ciekawych przypadków można znaleźć w dokumentacji <https://docs.python.org/3/whatsnew/3.10.html#pep-634-structural-pattern-matching>

4. TYPY ZŁOŻONE

LIST

Lista jest przykładem kontenera o modyfikowalnej zawartości. Listę w zapisie rozpoznajemy po prostokątnych nawiasach, elementy oddzielone przecinkami, co naturalnie upodabnia ten kontener do tablicy. Lista może mieć elementy różnych typów, łącznie z typami złożonymi, czyli np. zagnieżdżoną kolejną listą.

Tworzenie listy

```
>>> lista1 = []
>>> lista2 = list()
>>> type(lista1); type(lista2)
<class 'list'>
<class 'list'>
```

Podstawowe operacje na obiektach listy są podobne do tych dla typu str. Listy można powielać przez mnożenie, dodawać:

```
>>> lista1 = [1,2,3]*4
>>> lista1
[1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3]
>>> lista2 = lista1 + ["a","b","c"]
>>> lista2
[1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 'a', 'b', 'c']
```

Na liście można wykonywać selekcję („slicing”), ale nową rzeczą jest fakt, że wybrany przez taką selekcję fragment listy może być zmodyfikowany lub usunięty.

```
letters = ['a', 'b', 'c', 'd', 'e', 'f', 'g']
letters[2:5] = ['C', 'D', 'E'] # podmiana
letters[2:5] = [] # usunięcie ich
letters[:] = [] # cała lista
len(letters) # długość, ale uwaga: na poziomie głównym
```

Zazwyczaj za pomocą wymienienia elementów, które są w prostokątnym nawiasie, ale równie często korzysta się z konstruktora – ten wymaga dokładnie jednego argumentu, można taką operację postrzegać jako operator konwersji na typ list. W środku bowiem mogą być obiekty w „różnych nawiasach”, ale również generatory czy iteratory. A jeśli argumentem będzie łańcuch znakowy, zostanie stworzona lista z poszczególnych znaków. Ilustracja:

```
lista = ["aa", "cc", "bb"]
# lista = list("aa", "cc", "bb") # błąd
lista = list(["aa", "cc", "bb"]) # [] list
lista = list(("aa", "cc", "bb")) # () tuple
lista = list({"aa", "cc", "bb"}) # {} set
```

```
>>> l = list(range(5)); print(l)
[0, 1, 2, 3, 4]
>>> l = list(enumerate("abcd")); print(l)
[(0, 'a'), (1, 'b'), (2, 'c'), (3, 'd')]
>>> l = list("abcd"); print(l)
['a', 'b', 'c', 'd']
```

Lista może zawierać elementy różnych typów, nawet obiekty generatora czy iteratora. Jak pamiętamy, odczytanie iteratora oznacza skasowanie jego zawartości.

```
>>> l = [1, 1.23, "a", True, range(3), enumerate("abc")]; print(l)
[1, 1.23, 'a', True, range(0, 3), <enumerate object at 0x0000020467D7B380>]
```

Można oczywiście zagnieżdżać obiekty. Ciekawym przypadkiem jest zagnieżdżenie listy w samej sobie... co prowadzi do „niekończącej” się rekurencji w głąb. Widać to podczas „slicingu” tak powstałego obiektu. Przykład:

```
>>> l = [1, 2, 3]
>>> l[2] = l
>>> l
[1, 2, [...]]
>>> l[1:4]
[2, [1, 2, [...]]]
```

Można by napisać funkcję, w której wykonujemy pętlę po kolejnych elementach listy, sprawdzając ich typ, oraz gdy się okaże, że typ elementu to lista (np. warunek `if isinstance(i, list):`), to rekurencyjnie wywołać funkcję z tym elementem. W ten sposób szybko przekonalibyśmy się, że wykonanie zakończy się błędem, typu:

RecursionError: maximum recursion depth exceeded while calling a Python object po wykonaniu *blisko* 1000 rekurencyjnie zagnieżdżonych wywołań. Temat obsługi rekurencyjnych wywołań w Pythonie i ograniczeń może uda się omówić później (vide: Tail Recursion Elimination [↗](http://neopythonic.blogspot.com/2009/04/tail-recursion-elimination.html) <http://neopythonic.blogspot.com/2009/04/tail-recursion-elimination.html>). W module `sys` można sprawdzić na ile zagnieżdżeń ustawiony jest Python.

```
>>> import sys
>>> sys.getrecursionlimit()
1000
>>> sys.setrecursionlimit(500)
```

Za pomocą listy można stworzyć również tablicę wielowymiarową, trzeba tu jednak pamiętać, że nie polega to na „definicji” tablicy o określonej wielkości, tak jak w innych językach programowania, ale na zainicjalizowaniu każdej komórki. Można to zrobić na kilka sprytnych sposobów. Efekt ma doprowadzić do stworzenia listy o podstawowym wymiarze, nazwijmy X, oraz od każdej komórki tego wymiaru kolejnej zagnieżdżonej liście, o wymiarze Y. Bardzo się tu przyda skrócony zapis, który widzieliśmy już przy okazji sum i pętli `for`. A zatem:

```
X = 4
```



```
Y = 2
tab2d = [[0 for y in range(Y)] for x in range(X)]
# [[0, 0], [0, 0], [0, 0], [0, 0]]
```

Logika powyższego jest następująca: dla każdej iteracji podstawowego poziomu (nazwanego x), budującej podstawowy poziom listy tab2d, wykonane jest utworzenie zagnieżdżonej listy listy `[0 for y in range(Y)]` wypełnione zerami. Powyższy zapis można skrócić, tworząc każdą zagnieżdżoną listę za pomocą jednoelementowej listy `[0]` przemnożonej Y razy:

```
tab2d = [[0]*Y for _ in range(X)] # zwróćmy uwagę, że zmiennej _ nie potrzebujemy
```

Czy możliwy jest jeszcze większy skrót? Na przykład, `X*[Y*[0]]`? Niestety, w takim przypadku wewnętrzna lista `Y*[0]` zostanie sklonowana (a nie skopiowana). Choć na pierwszy rzut oka, powstała taka sama struktura, to jednak efekt jest taki, że zmiana zawartości jednej komórki, pociąga za sobą zmianę we wszystkich pozostałych podwymiarach:

```
>>> tab2d = X*[Y*[0]]
>>> tab2d[0][1] = 7
>>> tab2d
[[0, 7], [0, 7], [0, 7], [0, 7]]
```

Nawet w przypadku „poprawnie” skonstruowanej dwuwymiarowej tablicy warto sobie uświadomić, że nawet poszczególne komórki takiej struktury są referencjami do określonych miejsc w pamięci. Jeśli zbudujemy tablicę wypełnioną samymi zerami:

```
>>> tab = [2*[0] for _ in range(4)]
```

a następnie sprawdzimy, jakie są adresy pamięci każdej z komórek:

```
>>> for i,j in tab:
    print(id(i),id(j))

1886783432976 1886783432976
1886783432976 1886783432976
1886783432976 1886783432976
1886783432976 1886783432976
```

to okaże się, że wszystkie pokazują na ten sam adres (nie powinno to dziwić, jeśli pamiętamy, że Python wykonuje optymalizację dla pewnego zakresu wartości całkowitych). Jakakolwiek zmiana zawartości komórki pociąga jednak za sobą zmianę dowiązania adresowego, pokazującego na inną wartość. Podstawiając wartość 7, w pierwszej komórce dowiązujemy adres wskazujący na wartość 7 w pamięci. Poniżej widać sprawdzenie, że adres wartości 7 to ten sam adres, co komórki zawierającej po przypisaniu wartość 7.

```
>>> tab[0][0] = 7
>>> tab
[[7, 0], [0, 0], [0, 0], [0, 0]]
>>> id(7)
1886783433200
>>> id(tab[0][0])
1886783433200
```

Skoro wiadomo, że dwuwymiarowa lista to po prostu listy zagnieżdżone w listach, to można zawsze zacząć od zupełnie pustej listy w liście, `[[[]]`, by potem za pomocą funkcji `extend()` czy `append()` rozszerzać i dokładać...

```
>>> tab = [[]]
>>> tab[0].extend([1,2])
>>> tab
[[1, 2]]
>>> tab.append([3,4])
>>> tab
[[1, 2], [3, 4]]
```

Można też zacząć od jednego wymiaru i pozostałe dokładać później:

```
>>> tab = []
>>> tab.append([1,2])
>>> tab.append([3,4])
>>> tab
[[1, 2], [3, 4]]
```

Poważniejsze używanie macierzy na pewno skieruje nas ku modułowi NumPy, który jest dedykowany do pracy z macierzami, a konkretnie z typem `ndarray` (poznamy w dalszej części kursu).

Kopiowanie listy

Obiekty modyfikowalne, takie jak lista, mają stały początkowy adres w pamięci (**różny** od adresu pierwszego elementu), zatem modyfikacja ich zawartości, bądź powiększenie, nie zmienia tego adresu:

```
>>> tab = [1,2,3]
>>> id(tab)
1886824952000
>>> tab[0] = 7
>>> tab.append([4,5])
>>> tab
[7, 2, 3, [4, 5]]
>>> id(tab)
1886824952000
```

Kopiowanie listy może być operacją wykonywania kopii płytkiej lub głębokiej. Przypisanie innej nazwy do istniejącej nazwy listy nie jest żadnym kopiowaniem, tylko dowiązaniem kolejnej nazwy do tego samego adresu. Zatem wszelkie operacje za pomocą jednej nazwy, wpływają naturalnie na tę samą zawartość w pamięci, odczytywaną za pomocą innej nazwy.

```
>>> a = [1,2,3]
>>> b = a
>>> b[0] = 8
>>> a
[8, 2, 3]
>>> b
[8, 2, 3]
```

Typ **list** posiada funkcję **copy()**, która na podstawowym poziomie elementów wykonuje rzeczywiście kopię.

```
>>> a = [1,2,3]
>>> b = a.copy()
>>> b
[1, 2, 3]
>>> b[0] = 8
>>> b
[8, 2, 3]
>>> a
[1, 2, 3]
```

Jednak jeśli elementem jest zagnieżdżona lista, to kopia nie jest wykonana, skopiowany zostaje adres – a zatem wtedy jest to płytka kopia.

```
>>> a = [1,2,[3,4]]
>>> b = a.copy()
>>> b[0] = 8
>>> b[2][0] = 6
>>> b
[8, 2, [6, 4]]
>>> a
[1, 2, [6, 4]]
```

Jeśli chcemy wykonać głęboką kopię, łącznie z zagnieżdżonymi obiektami, można wykorzystać funkcję z modułu **copy**. Posiada on funkcję **copy()** o takim samym działaniu jak **copy()** dostępne dla typu **list**:

```
>>> import copy
>>> a = [1,2,[3,4]]
>>> b = copy.copy(a)
>>> b[0] = 8
>>> b[2][0] = 6
>>> b
[8, 2, [6, 4]]
>>> a
[1, 2, [6, 4]]
```

ale posiada również funkcję `deepcopy()` (fragment przykładu, początek jak wyżej):

```
>>> b = copy.deepcopy(a)
>>> b[0] = 8
>>> b[2][0] = 6
>>> b
[8, 2, [6, 4]]
>>> a
[1, 2, [3, 4]]
```

Emulację kopiowania (ale tylko na podstawowym poziomie, bo inaczej trzeba by wchodzić również w zagnieżdżone kolekcje i iterować) można wykonać również poprzez przejście po elementach listy (`L`) i tworzenie, wartość po wartości, nowej listy (`nowaL`):

```
>>> nowaL = [L[i] for i in range(len(L))]
```

Elementy listy

Typowa iteracja po elementach listy to pętla `for`. Można również sprawdzić, czy interesujący element znajduje się na liście za pomocą operatora `in`:

```
lista = ["jeden", "dwa", "trzy"]
if "dwa" in lista:
    print("Tak, 'dwa' jest elementem listy.")
```

Innym sposobem do otrzymania elementów listy (i nie tylko) jest rozpakowanie. Poniższy przykład wszystko ilustruje:

W badaniu elementów listy (i nie tylko listy) pomocne mogą być dwie funkcje Pythona: `all()`, `any()`. Pierwsza zwraca `True`, gdy wszystkie elementy listy mają ewaluowaną wartość logiczną `True` (lub `False` w przeciwnym razie – w poniższym przykładzie jeden z łańcuchów znakowych jest pusty). Druga funkcja `any()` zwraca `True` gdy choć jeden element jest prawdziwy.

```
>>> lista = ["a", "b", "c", "d", "e", "", "g", "h"]
>>> all(lista)
False
>>> any(lista)
True
```

Funkcje składowe listy

Poniżej zwięzły przegląd funkcji należących do typu list, warto się z nimi zapoznać.

`append(element)` wstawia element na końcu listy. Jeśli lista jest pusta, to jest to jej pierwszy element:

```
>>> L = []
>>> L.append(123)
>>> L
[123]
```

Modyfikowalność obiektu listy może doprowadzić do zaskakujących efektów. Na przykład, w pętli `for` wykonywanej na obiektach listy, jeśli wewnątrz pętli będziemy dodawać kolejne elementy, to pętla się nie zakończy (przerwanie za pomocą Ctrl-C):

```
>>> L = [1]
>>> for i in L:
    print(i, end=' ')
    L.append(i+1)
```

```
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30
31 32 33 34 35 36 37 38 39 40 41 42 43 44 45Traceback (most recent call last):
  File "<pysHELL#129>", line 2, in <module>
```

`extend(<obiekt>)` dodaje na koniec listy elementy innej wielkości iteracyjnej `<obiekt>`: listy, krotki, łańcucha znakowego czy generatora `range` – który jest ewaluowany, produkując zdefiniowaną sekwencję:

```
>>> l = [1, 2, 3]
>>> l.extend(range(3, 0, -1)) # [1, 2, 3, 3, 2, 1]
>>> l.extend('XYZ') # [1, 2, 3, 3, 2, 1, 'X', 'Y', 'Z']
```

`insert(pozycja, element)` wstawia na wskazanej pozycji element. Przykład demonstruje wstawianie na początku i na końcu:

```
>>> L = [1,2,3,4]
>>> L.insert(0,"a")
>>> L
['a', 1, 2, 3, 4]
>>> L.insert(len(L),"z")
>>> L
['a', 1, 2, 3, 4, 'z']
```

`remove(element)` przeszukuje listę pod kątem wystąpienia elementu i usuwa pierwszy napotkany. Jeśli element nie znajduje się na liście, zgłoszony zostaje wyjątek `ValueError`:

```
>>> L = [1,2,3,4,1,2,3]
>>> L.remove(2)
>>> L
[1, 3, 4, 1, 2, 3]
>>> L.remove(5)
Traceback (most recent call last):
  File "<pyshell#135>", line 1, in <module>
    L.remove(5)
ValueError: list.remove(x): x not in list
```

Aby zapobiec takiej sytuacji, należy przed usunięciem sprawdzić, czy dany element jest na liście:

```
[1, 3, 4, 1, 2, 3]
>>> if 5 in L:
    L.remove(5)
```

`pop()`, opcjonalnie z argumentem indeks, usuwa wskazany element z listy, jeśli jest wywołane bez argumentu, to ostatni. Dodatkowo funkcja zwraca usuwany element. Indeks może być również ujemny, zgodnie z zasadami indeksowania elementów w kontenerze sekwencyjnym. Jeśli indeks jest poza zakresem, zgłoszony zostaje wyjątek `IndexError`:

```
>>> L = ["a", "b", "c", "d", "e", "f"]
>>> ostatni = L.pop(); print(ostatni) # to samo L.pop(-1)
f
>>> drugi = L.pop(1); print(drugi)
b
>>> L.pop(20) # poza zakresem
Traceback (most recent call last):
  File "<pyshell#180>", line 1, in <module>
    L.pop(20) # poza zakresem
IndexError: pop index out of range
```

`del()` usuwa wskazany indeksem element lub zakres elementów, albo i całą zawartość. Składnię można użyć jako wywołanie funkcji lub jako operator `del`, przed wybranym zakresem elementów. Selekcję elementów do usunięcia można określić podając zakres, krok (co który element), również używając indeksów ujemnych. W przypadku błędnego indeksowania, zgłoszony jest wyjątek `IndexError`.

```
>>> L = ["a", "b", "c", "d", "e", "f", "g", "h", "i", "j"]
>>> del L[2::2] # od elementu drugiego do końca, co drugi
>>> L
['a', 'b', 'd', 'f', 'h', 'j']
>>> del(L[:]) # usuwa wszystkie elementy, zostaje pusta lista
>>> del(L) # usuwa wszystko, łącznie z obiektem L
```

clear() usuwa wszystkie elementy z listy, funkcja jest bezargumentowa i niczego nie zwraca. Nawet w przypadku funkcji nic nie zwracającej, w Pythonie można wykonać przypisanie (jak w przykładzie poniżej), wtedy obiekt zwracany to **None**.

```
>>> L = [1, 2, 3, 4]
>>> a = L.clear()
>>> L
[]
>>> type(a)
<class 'NoneType'>
```

count(element) zwraca liczbę obiektów o wartości element znajdujących się na liście. Oczywiście długość listy (jej podstawowy poziom) otrzymujemy za pomocą funkcji **len(L)**, gdzie L to lista.

```
>>> L = [1, 2, 3, 4, 2, 3, 2, 4]
>>> L.count(2)
3
>>> len(L) # można badać długość selekcji np. L[3:] itp.
8
```

index(element) zwraca pozycję indeksową pierwszego napotkanego wystąpienia elementu w liście, a jeśli elementu nie ma, to zostaje zgłoszony wyjątek **ValueError**.

```
>>> L = ["a", "b", "c", "d", "a", "b", "a"]
>>> L.index("a") # pierwsze wystąpienie "a"
0
>>> L[3:].index("a") # wystąpienie "a" liczone względem wyniku selekcji!
1
```

W praktyce aby odczytać wszystkie wystąpienia elementu, trzeba napisać pętlę, wraz ze sprawdzeniem, czy obiekt w ogóle jest na liście (operatorem **in** lub za pomocą funkcji **count**). W takich sytuacjach zamiast użyć **index()** często pisze się składnię przebiegającą po liście wraz z dodatkowym warunkiem sprawdzającym element:

```
>>> L = ["a", "b", "c", "d", "a", "b", "a"]
>>> W = [i for i, val in enumerate(L) if val=="a"]
>>> W
[0, 4, 6]
```

reverse() odwraca kolejność elementów na liście, poprzez zmianę kolejności adresowania.


```
>>> L = [1, 2, 3, "a", "b", "c"]
>>> L.reverse()
>>> L
['c', 'b', 'a', 3, 2, 1]
```

`sort(reverse=False|True, key=myFunc)` umożliwia posortowanie elementów listy, w porządku rosnącym (domyślnie) lub malejącym (jeśli argument `reverse=True`), według kryterium porównania elementów `<` (obiekty np. różnych typów, które nie mogą być porównane, nie mogą być też posortowane:

```
>>> L = [2, 1, "abc"]
>>> L.sort()
Traceback (most recent call last):
  File "<pysHELL#351>", line 1, in <module>
    L.sort()
TypeError: '<' not supported between instances of 'str' and 'int'
```

Sposób określania porównywanych wartości elementów można zdefiniować z pomocą drugiego argumentu `key`, który przyjmuje funkcję (lub wyrażenie lambda, jak później się dowiemy) zdolną wyliczyć jakąś wartość na każdym elemencie, która potem będzie porównywana w procesie sortowania. Może to być na przykład sortowanie według długości elementów:

```
>>> L = ["abcdef", "abcd", "a", "ab"]
>>> L.sort(key=len)
>>> L
['a', 'ab', 'abcd', 'abcdef']
```

Łatwo również posortować bez uwzględniania wielkich i małych liter (przykład dodatkowo z odwróconą kolejnością sortowania:

```
>>> L = ["B", "b", "a", "AA", "CCC", "bb"]
>>> L.sort(reverse=True, key=str.upper)
>>> # tak samo: key=str.casefold str.isupper str.lower
>>> L
['CCC', 'bb', 'B', 'b', 'AA', 'a']
```

Sortowanie odbywa się na tym samym obiekcie listy (in-place) oraz jest to sortowanie stabilne, to znaczy jeśli dwa elementy listy mają taką samą wartość, to ich wzajemne położenie względem siebie zostanie zachowane. Dość łatwo można to zaobserwować na przykładzie, w którym na liście znajduje się mieszanka elementów typu `int` i `str`, czyli bezpośrednio nieporównywalnych. Stosując `key=str` czyli operację rzutowania każdego elementu na typ `str`, takie porównanie jest możliwe, elementy zostają posortowane, przykładowo "1" oraz 1 pozostają względem siebie w takim samym porządku:

```
>>> L = ["1", 1, "2", 0, 4, "3"] # mieszanka int i str
>>> L.sort(key=str)
>>> L
[0, '1', 1, '2', '3', 4]
```

Innym, bardziej uniwersalnym sposobem sortowania, bo działającym nie tylko na listach, ale na wszelkich kolekcjach iterowalnych, jest wbudowana funkcja Pythona, `sorted(iterable, *, key=None, reverse=False|True)`, która zwraca posortowaną listę, wykonując kopię (oryginalny kontener jest niezmieniony). Dwa opcjonalne argumenty `key`, `reverse`, mają takie samo znaczenie. Gwiazdka `*` to znak sterujący, za którym argumenty funkcji to mogą być tylko argumenty nazwane klucz=wartość (patrz omówienie argumentów funkcji).

```
>>> L = ["1", 1, "2", 0, 4, "3"]
>>> id(L) # oryginal
1891326918464
>>> nowaL = sorted(L, key=str)
>>> id(nowaL) # kopia
1891358903040
>>> nowaL
[0, '1', 1, '2', '3', 4]
```

Składanie listy

Skrócony zapis tzw. składanie listy (list comprehension) pozwala na bardzo efektywny zapis, dzięki któremu można stworzyć listę, wykonać iterację, sprawdzić warunek. Przykładowo, poniższy kod

```
pow2 = []
for x in range(10):
    pow2.append(2 ** x)
```

można zastąpić jedną linijką kodu:

```
pow2 = [2 ** x for x in range(10)]
print(pow2) # [1, 2, 4, 8, 16, 32, 64, 128, 256, 512]
```

Przykład z pętlą i dodatkowym warunkiem:

```
pow2 = [2 ** x for x in range(10) if x > 5] # [64, 128, 256, 512]
nieparzyste = [x for x in range(20) if x % 2 == 1] # [1, 3, 5, 7, 9, 11, 13, 15, 17, 19]
```

Podwójna pętla (kombinacja każdego elementu z każdym):

```
[x+y for x in ['Jestem ', 'Mam na imie '] for y in ['Adam', 'Ewa']]
# ['Jestem Adam', 'Jestem Ewa', 'Mam na imie Adam', 'Mam na imie Ewa']
```

Jeszcze inny przykład: przeliczanie jednostek temperatury ze stopni Celsjusza do stopni Fahrenheita:

```
Celsius = [-17.7778, -10, 0, 12.5, 36.6, 38.0, 42.0]
Fahrenheit = [9/5*x+32 for x in Celsius]
print(Fahrenheit)
# [-3.999999999848569e-05, 14.0, 32.0, 54.5, 97.88000000000001, 100.4, 107.60000000000001]
```

Naturalnie, zapewne wolelibyśmy wynik zapisany z precyzją np. do dwóch miejsc po przecinku. Jeśli chcemy od razu wynik z określoną precyzją, można skorzystać z funkcji zaokrąglenia `round(wartość, precyzja)`:

```
Fahrenheit = [round(9/5*x+32, 2) for x in Celsius]
# wynik: [-0.0, 14.0, 32.0, 54.5, 97.88, 100.4, 107.6]
```

Jeśli jednak chodzi o formatowanie wyświetlania, a nie ingerowanie w sam rezultat, można skorzystać z kilku wariantów formatowania typu str w Pythonie. Jedną z możliwości:

```
newF = [ '%.1f' % x for x in Fahrenheit ]
# wynik: ['-0.0', '14.0', '32.0', '54.5', '97.9', '100.4', '107.6']
```

Jak widać, efektem jest lista obiektów typu str, a nie obiektów float. Można jednak wykonać wtórną konwersję:

```
newF = [ float('%s' % x) for x in Fahrenheit ]
# wynik: [-0.0, 14.0, 32.0, 54.5, 97.9, 100.4, 107.6]
```

Kolejka i stos

W informatyce są wyróżnione pewne proste struktury danych, takie jak kolejka i stos. Ich implementację można bezproblemowo wykonać w oparciu o listę.

```
# emulacja kolejki (queue)
tab = [1,2,3]
tab.append(x) # queue.push(x)
tab.pop(0) #queue.pop()
tab[0] #queue.peek()
```

```
# emulacja stosu (stack)
tab = [1,2,3]
tab.append(x) #stack.push(x)
tab.pop() # stack.pop()
tab[-1] # stack.peek()
```

TUPLE

Krotka (tuple) jest kontenerem sekwencyjnym, który jest niemodyfikowalny. Zatem obiekty tuple posiadają wyliczoną wartość hash i mogą być stosowane wszędzie tam, gdzie jest wymagany obiekt niemodyfikowalny – na przykład jako klucz w typie słownikowym (dict).

Sposób utworzenia (pustej) krotki, a także demonstracja, że każdy z takich pustych obiektów ma wyliczony ten sam hash:

```
>>> tuple1 = ()
>>> tuple2 = tuple()
>>> print(type(tuple1), type(tuple2))
<class 'tuple'> <class 'tuple'>
>>> print(hash(tuple1), hash(tuple2))
5740354900026072187 5740354900026072187
>>> hash(())
5740354900026072187
```

Można by się zastanawiać jaki jest sens obiektu pustego, którego nie można zmodyfikować. Jednak wszędzie tam, gdzie spodziewana jest krotka (np. w funkcji można zapisać sekwencję

pozycyjnych argumentów za pomocą krotki), to również powinna istnieć możliwość przekazania pustego obiektu, stąd istnienie obiektu typu pusta krotka ma jak najbardziej sens.

Krotka może zawierać, podobnie jak lista, obiekty różnych typów:

```
krotka1 = (1, 1.23, 'stxt', 4+1j, True, None)
```

Można w tym zapisie opuścić nawiasy:

```
krotka1 = 1, 2, 3, 4 # powstaje (1, 2, 3, 4)
```

Osobliwy jest natomiast zapis krotki jednoelementowej, gdyż zapis: `k1 = (1)` prowadzi do utworzenia obiektu `k1` typu `int`. Ażeby była to krotka jednoelementowa, czyli singleton, trzeba zastosować niezbyt estetyczny przecinek na końcu: `k1 = 1, # albo k1 = (1,)`

Na *tuplach* można stosować dostęp za pomocą indeksów, również selekcję, tak samo jak na liście, np.: dla `krotka1 = 1, 2, 3, 4` zapis: `krotka1[0:1]` da wynik `(1,)` – jak widać, odwołując się przez indeks(y), również otrzymujemy krotkę (może być pusta). Wykonując *slicing* na krotce, otrzymujemy nową krotkę (pod innym adresem), zawierającą wybraną zawartość. Odwołanie się indeksem poza zakres powoduje zgłoszenie wyjątku `IndexError: tuple index out of range`. Za pomocą funkcji `k1.count(<obiekt>)` można odpytać ile razy `<obiekt>` występuje w danej krotce.

Krotki mogą posiadać zagnieżdżone kolejne elementy, również krotki. Taki zapis `k1 = (1,,)`, spowoduje utworzenie `((1,))`. Jeśli krotka składa się z elementów niemodyfikowalnych, np.

```
k1 = 'abc', (1, 1.23), (False, True) # powstaje ('abc', (1, 1.23), (False, True))
```

to nadal pozostaje obiektem, dla którego wyliczony jest hash. Jeśli jednak uczynimy elementem krotki obiekt typu modyfikowalnego, np. listę, to dla takiej krotki nie można już wyliczyć hash (i tym samym, nie można stosować tam, gdzie hash jest wymagany).

```
>>> k1 = 1, [2, 3], 4, 'abc', ['x', 'y'] # (1, [2, 3], 4, 'abc', ['x', 'y'])
...
>>> hash(k1)
...
Traceback (most recent call last):
  File "<pyshell#300>", line 1, in <module>
    hash(k1)
TypeError: unhashable type: 'list'
```

W przypadku jak wyżej, nie możemy modyfikować elementów krotki (*tuple* nie ma operatora przypisania), ale można dostać się do zawartości obiektu modyfikowalnego (np. listy) i jego zawartość dowolnie zmienić:

```
>>> k1[0] = 11 # nie ma operatora =
...
Traceback (most recent call last):
  File "<pyshell#326>", line 1, in <module>
    k1[0] = 11 # nie ma operatora =
TypeError: 'tuple' object does not support item assignment
>>>
>>> k1[1][0:2] = [1, 2, 3, 4, 5] # tu podmieniam zawartosc listy
...
>>> k1
...
(1, [1, 2, 3, 4, 5], 4, 'abc', ['x', 'y'])
```

Niech nas nie zwiedzie, że możemy napisać np.

```
tuple1 = () # pierwsza krotka
tuple1 = (1,2,3) # druga zupełnie nowa krotka
```

bo to będzie zupełnie inny, nowy obiekt, a nie modyfikacja poprzedniej pustej krotki.

Krotkę (podobnie jak dla listy) utworzymy też przez rzutowanie. Przykłady:

```
k1 = tuple("abcd") # utworzy ('a', 'b', 'c', 'd')
k1 = tuple(range(0,10)) # utworzy (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)
```

Zatem jedną z możliwości „modyfikowania krotki” jest wykonanie rzutowania na listę, zrobienie modyfikacji i ponowne rzutowanie na tuple. Oczywiście finalnie dostaniemy całkowicie inną krotkę niż na początku:

```
>>> t1 = ('alfa', 'beta', 'gamma'); print(t1, id(t1))
('alfa', 'beta', 'gamma') 2553137560960
>>> l1 = list(t1); print(l1, id(l1))
['alfa', 'beta', 'gamma'] 2553137277504
>>> l1[0] = 'omega' # teraz modyfikacja możliwa, bo lista
>>> t1 = tuple(l1); print(t1, id(t1)) # nowa krotka
('omega', 'beta', 'gamma') 2553137559872
```

Oprócz rzutowania, krotkę można też otrzymać poprzez rozpakowanie listy:

```
l = list('abrakadabra') # tworzymy ['a', 'b', 'r', 'a', 'k', 'a', 'd', 'a', 'b', 'r', 'a']
t = (*l,) # ('a', 'b', 'r', 'a', 'k', 'a', 'd', 'a', 'b', 'r', 'a')
```

Natomiast aby z powrotem uzyskać string z krotki, można zapisać: `str = ''.join(t)`

Iterowanie po elementach krotki jest podobne jak dla listy. Przykład listy z krotkami:

```
>>> for a,b,c,d in [(1,2,3,4), (5,6,7,8)]:
...     print(f"a={a}, b={b}, c={c}, d={d}")
...
...
a=1, b=2, c=3, d=4
a=5, b=6, c=7, d=8
```

Przykład z rozpakowywaniem:

```
>>> d1, *dn, d2 = (1, 2, 3, 4, 5, 6)
>>> print(d1,dn,d2)
1 [2, 3, 4, 5] 6
```

DICT

Typ słownikowy reprezentuje rodzaj tablicy skojarzeniowej. Każdy element to para wielkości: klucz-wartość. Klucz musi być typem niemodyfikowalnym (precyzyjniej pisząc – takim, dla którego możliwe jest wyliczenie wartości hash). Wartość (może być to złożony obiekt) jest modyfikowalna – stąd typ dict należy do typów modyfikowalnych.

Obiekt dict tworzymy za pomocą pary nawiasów `{ }` lub za pomocą `dict()`:

```
d1 = { } # pusty
d1 = dict()
```

Obiekt wypełniamy parą wielkości klucz-wartość następująco:

```
d1 = { 'a': 'alpha', 123: 1.435, True: 'prawda' } # jak widać można mieszać typy
```

Można też utworzyć z listy krotek:

```
d1 = dict([('a', 'AAA'), ('b', 'BBB')]) # powstaje {'a': 'AAA', 'b': 'BBB'}
```

Natomiast gdy klucze są prostymi stringami, można:

```
d1 = dict(a= 'alpha', a123= 1.435, klucz3='prawda')
# powstaje {'a': 'alpha', 'a123': 1.435, 'klucz3': 'prawda'}
```

Słownik można utworzyć też za pomocą składni „dictionary comprehension” (złożenie słownikowe), które sposobem na elegancki i zwięzły zapis:

```
>>> def p2(x):
...     return 2**x
...
>>> s1 = {x: p2(x) for x in range(1,11)}
>>> s1
{1: 2, 2: 4, 3: 8, 4: 16, 5: 32, 6: 64, 7: 128, 8: 256, 9: 512, 10: 1024}
```

Minimalna składnia wymagana do składania słownikowego to:

```
dictionary = {key: value for vars in iterable}
```

Za pomocą takiej składni można przeliczyć zawartość jednego słownika do drugiego:

```
>>> cena_pln = {'kawa': 15.50, 'mleko': 1.5, 'ciasto': 22.65}
>>> pln_do_usd = 1 / 4.77
>>> cena_usd = {item: value*pln_do_usd for (item, value) in cena_pln.items()}
>>> cena_usd
{'kawa': 3.2494758909853254, 'mleko': 0.3144654088050315, 'ciasto': 4.748427672955975}
```

W składni można dodać warunki selekcji:

```
>>> d = {'Adam': 22, 'Piotr': 33, 'Jan': 12, 'Pawel': 71}
>>> nowy_d = {k: v for (k, v) in d.items() if v % 2 != 0 if v < 40}
>>> nowy_d
{'Piotr': 33}
```

Również po stronie wielkości tworzących słownik (konstrukcja if... else):

```
>>> nowy_d2 = {k: ('stary' if v > 40 else 'mlody' for (k, v) in d.items())}
>>> nowy_d2
{'Adam': 'mlody', 'Piotr': 'mlody', 'Jan': 'mlody', 'Pawel': 'stary'}
```

Elementy (pary) obiektu dict wypełnione są w kolejności dodawania ich (co jest gwarantowane od Python 3.7), zatem nie są posortowane, ich identyfikacja odbywa się poprzez wyliczoną dla każdego klucza wartość hash. Klucz nie może się powtórzyć (czyli jest unikatowy, tak jak element w typie set). Nie da się odwołać do danej pary poprzez indeks – z prostego powodu – indeks czyli typ int również można zastosować jako klucz, bo jest typem niemodyfikowalnym (czyli z wyliczoną wartością hash). Najbardziej oczywistym sposobem odczytania wartości odpowiadającej danemu kluczowi jest składnia: `d1[klucz1]`. W przypadku braku klucza `klucz1` w `d1`, zgłoszony zostaje wyjątek `KeyError: klucz1`. Aby tego uniknąć, można odpytać za pomocą składni, nawet połączonej z odczytaniem poprzez operator and:

```
klucz1 in d1 and d1[klucz1] # jeśli klucz1 nie ma w d1, to nie będzie wywołane d1[klucz1]
```

Dodanie kolejnej pary do obiektu słownika: `d1[klucz1] = wartosc1`. Dla istniejącego wcześniej `klucz1` taki zapis prowadzi do aktualizacji wartości powiązanej z danym kluczem. Kolejności elementów w słowniku nie można zmienić „in place”. Gdybyśmy chcieli to zrobić (np. posortować po kluczu), musielibyśmy usuwać dany klucz-wartość (za pomocą `del`) i dodawać go ponownie (dodany ląduje na końcu). Można też wykonać sortowanie poprzez uzyskanie dynamicznego widoku słownika, o czym powiemy poniżej. Krótki przegląd dostępnych funkcji.

`d1.clear()` usuwa zawartość (pusty słownik)

`d1.get(<klucz>[, <domyślny>])` jest bezpiecznym sposobem odpytania, bo jeśli nie ma w słowniku klucza, to zwrócone zostaje `None`. Natomiast jeśli chcemy, żeby w takiej sytuacji było zwracane coś innego niż `None`, to definiujemy to w polu `<domyślny>`, może to być nawet wywołanie funkcji. Powiedzmy, że w `d1` nie ma klucza 'aa':

```
print(d1.get('aa')) # None
print(d1.get('aa', "masakra".upper())) # MASAKRA
print(d1.get('aa', print("masakra".upper()))) # MASAKRA None bo print nic nie zwraca
```

Z kolei jeśli chcemy umieścić coś w słowniku lub odpytać o wartość, możemy użyć `d1.setdefault(<klucz>[, <domyślny>])`, która to funkcja zwraca wartość odpowiadającą kluczowi, a jeśli takiego w słowniku `d1` nie ma, zwraca `d1`, chyba że podamy argument `<domyślny>`. A zatem, jeśli mamy obiekt słownika `d1` i nie ma w nim klucza 123, a chcemy z tym kluczem związać pustą listę, to możemy napisać:

```
d1.setdefault(123, [])
```

i co więcej, ponieważ takie wywołanie zwraca obiekt `<domyślny>` czyli tutaj pustą listę, to od razu możemy na niej wywołać jakąś funkcję (np. `append`, `extend`):

```
d1.setdefault(123, []).extend(range(10,15)) # {123: [10, 11, 12, 13, 14]}
```

Trzy kolejne funkcje są ważne, z punktu widzenia pozyskiwania widoków oraz selekcji:

`d1.items()` zwraca obiekt `dict_items` zawierający listę z krotkami (klucz, wartość) ze słownika, który łatwo można od razu rzutować na listę. Podobnie `d1.keys()`, jeśli zapiszemy `list(d1.keys())` to od razu mamy listę wszystkich kluczy. Dla `list(d1.values())` mamy listę wszystkich wartości.

Prosta pętla po kluczach i wartościach:

```
>>> for k,v in s1.items():
...     print(k, '-', v, end='|')
...
...
1 - 2|2 - 4|3 - 8|4 - 16|5 - 32|6 - 64|7 - 128|8 - 256|9 - 512|10 - 1024|
```

Gdyby wykonać pętlę bez funkcji `items()`, to dostawalibyśmy klucze, czyli pętla po słowniku oraz pozyskanie wartości wyglądałoby tak:

```
>>> for k in s1:
...     print(k, '-', s1[k], end='|')
...
...
1 - 2|2 - 4|3 - 8|4 - 16|5 - 32|6 - 64|7 - 128|8 - 256|9 - 512|10 - 1024|
```

Usunięcie i zwrócenie elementu wykonać można za pomocą `d1.pop(<klucz>[, <domyślny>])` i jeśli klucza nie ma oraz nie zdefiniowano pola `<domyślny>`, zgłoszony zostaje wyjątek `KeyError`, jeśli zdefiniowano `<domyślny>` to jest to wielkość zwrócona.

Funkcja `s1.popitem()` usuwa i zwraca krotkę z ostatnią parą w słowniku, a jeśli słownik jest pusty, zgłasza wyjątek `KeyError`. Przed Python 3.6, kiedy kolejność obiektów w słowniku nie była gwarantowana, funkcja zwracała losową parę.

Na koniec: `d1.update(<obiekt>)` służy połączeniu (dodaniu) do słownika `d1` elementów ze słownika `<obiekt>`. Jeśli są wśród nich elementy z kluczem już występującym, to wartość jest aktualizowana, a jeśli nie, to para klucz-wartość jest dodawana.

```
>>> d1 = { 1 : 'a', 2 : 'b' }
>>> d2 = { 1 : 'x', 3 : 'y', 0 : 'z' }
>>> d1.update(d2); print(d1)
{1: 'x', 2: 'b', 3: 'y', 0: 'z'}
```

Podobnie jak przy tworzeniu słownika, argumentem `update()` może być lista z krotkami lub argumenty w zapisie `klucza = wartość, kluczb = wartośćb`.

SET

Set to nieuporządkowana, modyfikowalna kolekcja niepowtarzających się obiektów, mogą to być obiekty różnych typów, także złożone. Utworzyć ją możemy za pomocą `set(<iter>)` lub `{ }` gdzie `<iter>` to wielkość iterowalna.

```
>>> s = set([1,3,2,3,1,3,1,2,2,2]) # {1, 2, 3}
>>> s = {1,3,2,3,1,3,1,2,2,2} # {1, 2, 3}
>>> s = set() # pusty set
>>> print(s)
set()
```

Przykład z różnymi typami: `{1.23,1,"a",7+3j,True,(4,5)}`

Elementy obiektu set są niepowtarzalne, ale kolejność ich ułożenia jest niegwarantowana:

```
set("AbrakadAbra") # {'k', 'd', 'a', 'A', 'r', 'b'}
```

Oczywiście można posortować, np. używając funkcji `sorted()`, działającej analogicznie do funkcji `sort()` składowej listy <https://docs.python.org/3/howto/sorting.html>

```
sorted(set("AbrakadAbra")) # ['A', 'a', 'b', 'd', 'k', 'r']
```

ale – otrzymujemy w efekcie listę. Z definicji bowiem, set jest zbiorem nieuporządkowanym. Jeśli więc uprzemy się i powyższe znowu rzutujemy na set, dostaniemy:

```
set(sorted(set("AbrakadAbra"))) # {'d', 'k', 'a', 'A', 'r', 'b'}
```

Jak widać, otrzymany set ma elementy w innej kolejności, co jest bez znaczenia dla typu set.

Inny ciekawy przykład:

```
set(range(6)) # {0, 1, 2, 3, 4, 5}
list(enumerate(range(6))) # [(0, 0), (1, 1), (2, 2), (3, 3), (4, 4), (5, 5)]
```

czyli po kolei ale dla set kolejność krotek będzie przypadkowa:

```
set(enumerate(range(6))) # {(4, 4), (5, 5), (0, 0), (1, 1), (3, 3), (2, 2)}
```

Kilka prostych funkcji modyfikujących `add(<element>)` – dodanie elementu, `clear()` – usunięcie wszystkich elementów, `remove(<element>)` – usunięcie wskazanego elementu i ciekawostka: `pop()` – usunięcie dowolnego, losowego elementu, a jeśli obiekt set jest pusty, zgłoszony zostaje wyjątek `KeyError: 'pop from an empty set'`.

Na obiektach set można zastosować znane z matematyki operacje: unię (sumę), część wspólną (przecięcie), różnicę i różnicę symetryczną. Operacje te można wywołać za pomocą funkcji lub operatorów:


```
s1 = {'a', 'A', 'z', 'Z', 'G', 'g'}
s2 = {'b', 'B', 'z', 'Z', 'G', 'k'}
s1.union(s2) # {'g', 'k', 'G', 'a', 'A', 'B', 'Z', 'z', 'b'}
s1 | s2 # j.w.
s1.intersection(s2) # {'z', 'G', 'Z'}
s1 & s2 # j.w.
s1.difference(s2) # {'g', 'A', 'a'}
s1 - s2 # j.w.
s1.symmetric_difference(s2) # {'g', 'k', 'A', 'b', 'B', 'a'}
s1^s2 # j.w.
```

Pomiędzy użyciem funkcji a operatorów jest subtelna różnica: w przypadku funkcji możemy jako argument podać wielkości iterowalne niebędące typu set, wtedy nastąpi automatyczna konwersja do typu set. W przypadku operatorów, oba argumenty muszą być typu set. Operacje dla operatorów można połączyć z przypisaniem: `|=` (funkcja `update`), `&=` (funkcja `intersection_update`), `-=` (funkcja `difference_update`) oraz `^=` (funkcja `symmetric_difference_update`), wtedy dla przykładów jak wyżej wynik (który jest nowym obiektem set) będzie przypisany do nazwy s1. Operacja z przypisaniem jest wydajniejsza, niż wykonana osobo i potem przypisanie wyniku.

Relacje pomiędzy elementami dwóch kontenerów set można sprawdzić za pomocą:

`s1.isdisjoint(s2)` – jeśli brak elementów wspólnych, zwraca True

`s1.issubset(s2)` – prawda, jeśli wszystkie elementy s1 są podzbiorem s2: można też zapisać za pomocą relacji `<=`

```
s1 = {2,1}
s2 = {1,2,3,4}
s1 <= s2 # True
```

`s1.issuperset(s2)` – prawda, jeśli wszystkie elementy s2 są podzbiorem s1: można zapisać za pomocą relacji `>=`

```
s1 >= s2 # False, przykład jak wyżej, ale:
s2 >= s1 # True
```

FROZENSET

Typ frozen set odpowiada typowi set, ale jest niemodyfikowalny.

```
>>> fs = frozenset(['a', 'b', 'c'])
>>> print(fs, hash(fs))
frozenset({'a', 'b', 'c'}) -1720386717858035631
>>> fse = frozenset(); print(fse)
frozenset()
>>> fs & {'a', 'B', 1.23, True}
frozenset({'a'})
>>> frozenset(enumerate(range(10)))
frozenset({(4, 4), (8, 8), (5, 5), (7, 7), (0, 0), (9, 9), (1, 1), (3, 3), (2, 2), (6, 6)})
```

Natomiast funkcje modyfikujące (`add`, `pop`, `clear`...) są w typie frozen set nieobecne. Nie dajmy się zwieść zapisowi typu `f &= s`, który nie modyfikuje obiektu f, tylko tworzy nowy:

```
>>> f = frozenset(['foo', 'bar', 'baz'])
>>> s = {'baz', 'qux', 'quux'}
>>> f &= s # ten zapis to: f = f & s gdzie f po lewej to nowy obiekt
>>> f
frozenset({'baz'})
```

5. FUNKCJE

Funkcja w Pythonie kreowana jest dynamicznie, posiada nazwę, może mieć argumenty i może coś zwracać. Nigdzie nie określamy typów argumentów czy wartości zwracanej, więc możliwość konkretnego działania danej funkcji sprawdzana jest podczas jej wywołania. Funkcja (jej nazwa) ma swój adres w pamięci, sprawdzalny poprzez `id()`. Przykład:

```
def fib(n):    # ciąg Fibonacciego do n
    """Wypisuje liczby Fibonacciego do n."""
    a, b = 0, 1
    while a < n:
        print(a, end=' ')
        a, b = b, a+b
    print()
```

Tak zwany docstring to łańcuch znakowy (w potrójnym cudzysłowie), stanowiący opis (dokumentację) funkcji, klasy, modułu, skryptu... Dostępny przez atrybut `__doc__`, np.

```
print(fib.__doc__)
```

Zapiszmy skrypt .py z przykładową zawartością, na przykład komentarzem na początku. Tekst ten można wypisać, podając określony argument, z którym wywołamy skrypt. Aby odczytać zewnętrzne parametry wywołania, używa się moduł `argparse` lub `sys`, na przykład:

```
"""Komentarz na temat
skryptu python"""
import sys
print(sys.argv[0]) # nazwa skryptu
if len(sys.argv)>1 and sys.argv[1]=='--help':
    print(__doc__)
```

Wywołując powyższy skrypt z parametrem `--help`, zobaczymy komentarz.

W Pythonie nazwa nie jest związana z typem. Jak już wiemy, możliwe jest stosowanie wskazówek (type hinting). Dla zmiennej:

```
a: int = 1 # wskazówka, że a powinno być typu int
```

Dla funkcji:

```
def func(arg: arg_type, optarg: arg_type = default) -> return_type:
    pass # pusta definicja
```

można wypisać te informacje poprzez atrybut `__annotations__`, np.

```
def fun(tekst: str, n: int = 10) -> int:
    pass
print(fun.__annotations__)
# zobaczymy {'tekst': <class 'str'>, 'n': <class 'int'>, 'return': <class 'int'>}
```

W sesji interaktywnej też jest to dostępne:

```
>>> pi: float = 3.14
>>> n: int = 1
>>> __annotations__
{'pi': <class 'float'>, 'n': <class 'int'>}
```

Oczywiście funkcja, zamiast coś drukować, może zwracać wyliczoną wielkość, np. ciąg liczb Fibonacciego w postaci listy:

```
def fib(n):
    result = []
    a, b = 0, 1
    while a < n:
        result.append(a)
        a, b = b, a+b
    return result

print(fib(10)) # [0, 1, 1, 2, 3, 5, 8]
```

Funkcja może rekurencyjnie wywoływać samą siebie, wtedy:

```
def fib(n):
    return -1 if n<1 else (1 if n==1 or n==2 else fib(n-1)+fib(n-2))
print(fib(-5)) # zły argument
print(fib(20)) # wyliczenie 20. elementu ciągu
for i in range(1,21): print(fib(i),end=' ') # seria od 1 do 20
```

Funkcja może mieć dowolne argumenty (parametry) formalne, określone nazwą. Jej wywołanie to wstawienie, w kolejności i liczbie odpowiadającej definicji, parametrów aktualnych (podczas wywołania). Argumenty mogą mieć wartości domyślne, jeśli nie wszystkie, to musi być zachowana ciągłość idąc od prawej do lewej po liście argumentów. Czyli taki przypadek to oczywiście błąd:

```
def fun(a, b=1, c):
    print(a,b,c)
    def fun(a, b=1, c):
        ^
SyntaxError: non-default argument follows default argument
```

Wartości domyślne nie determinują typu argumentu, bo funkcję można wywołać z czymkolwiek, byle liczba parametrów się zgadzała. Dość wygodną techniką jest przekazywanie wartości domyślnej None i wtedy kod może reagować opcjonalnie na obecność lub brak danego argumentu:

```
def fun(a, b, scale=None):
    if scale is not None:
        return (a + b)*scale
    else:
        return a + b
```

Bardzo istotne jest, że Python dokonuje ewaluacji wartości domyślnej (w sensie obiektu) tylko raz, w momencie definicji funkcji. Skutkiem tego są następujące sytuacje:

```
i = 5
def f(arg=i):
    print(arg)
i = 10
f() # wypisane zostanie 5
```

Jeśli obiekt inicjalizujący jest zmienną (np. lista, słownik), to:

```
def f(a, L=[]):
    L.append(a)
    return L
print(f(1)) # [1]
print(f(2)) # [1, 2]
print(f(3)) # [1, 2, 3]
```

Zatem L zostało utworzone raz i jest współdzielone z kolejnymi wywołaniami funkcji. Rozwiązanie:

```
def f(a, L=None):
    if L is None:
        L = []
    L.append(a)
    return L
```

Definicje funkcji można zagnieżdżać, wtedy funkcja zdefiniowana wewnątrz (inner function) nie jest widzialna na zewnątrz. Funkcje wewnętrzne nie są zdefiniowane, dopóki nie zostanie wywołana funkcja nadrzędna. Są lokalnie ograniczone i istnieją tylko wewnątrz funkcji zewnętrznej jako zmienne lokalne.

```
def factorial(n):
    # tu można sprawdzić poprawność argumentu n
    def inner_factorial(n):
        if n <= 1:
            return 1
        return n*inner_factorial(n-1)
    return inner_factorial(n)
#inner_factorial(10) # jest niewidzialne
print(factorial(10))
```

Realne zastosowania zagnieżdżonych funkcji to tzw. domknięcia (closures). Wewnętrzna funkcja przechowuje obiekty wykorzystane przez funkcję (matkę). Funkcja obejmująca jest fabryką funkcji zagnieżdżonej (zwracanej w instrukcji return):

```
def add_brackets(b1='[', b2=']'):
    def obj_in_bracket(s):
        return b1+s+b2
    return obj_in_bracket # tu zwracana jest referencja do funkcji wewnętrznej

sb = add_brackets('(',')') # zwraca (wytworza) funkcję obj_in_bracket do dalszego wywołania
print(sb("hello")) # (hello)
```

Ciekawszy przykład:

```
def has_permission(page):
    def inner(username):
        if username == 'root':
            return "{0} ma uprawnienia {1}.".format(username, page)
        else:
            return "{0} nie ma uprawnień {1}.".format(username, page)
    return inner

usr = has_permission('administratora')
print(usr('root')) # 'root' ma uprawnienia administratora.
print(usr('user')) # 'user' nie ma uprawnień administratora.
```

W większości przypadków, gdy widzisz dekorowaną funkcję (czyli tę wewnętrzną), dekorator (czyli funkcja zewnętrzna) jest funkcją fabryczną, która przyjmuje funkcję jako argument i zwraca nową funkcję, która zawiera starą funkcję wewnątrz domknięcia.

Argumenty funkcji

Zanim omówimy technikę dekoratorów, trzeba uzupełnić wiedzę na temat argumentów funkcji. Do przekazania argumentów w wywołaniu funkcji można użyć nazwę argumentu (jako klucz), wtedy kolejność argumentów, z jawnym podaniem klucza, jest dowolna. Argumenty podane z kluczem muszą następować po ewentualnych argumentach pozycyjnych.

```
def fun(a,b,c,d=3.14): # d ma wartosc domyślną
    print(a,b,c,d)
fun(b="abc",a=1,c=False) # 1 abc False 3.14
```

Czasem użycie powyższych jest konieczne. Jeśli mamy funkcję `fun` z argumentem pozycyjnym o nazwie `nazwa` oraz drugim – słownikowym:

```
def fun(nazwa, **kws):
    return 'nazwa' in kws
```

to jeśli w słownikowej części pojawi się klucz `nazwa`, dojdzie do kolizji. Wywołanie:

```
fun(5, nazwa=10) # lub: fun(5, **{'nazwa': 10})
```

zakończy się błędem:

```
TypeError: fun() got multiple values for argument 'nazwa'
```

Wtedy rozwiązaniem jest:

```
def fun(nazwa, /, **kws):
    return 'nazwa' in kws
```

gdyż pierwszy argument jest wtedy traktowany tylko jako pozycyjny i nie ma niejednoznaczności nazw kluczy.

DEKORATOR

Sytuacja z zagnieżdżoną definicją funkcji jest interesująca, gdy argumentem zewnętrznej funkcji (**dekoratora**) będzie funkcja, która zostanie w jakiś sposób „udekorowana” poprzez działanie wewnętrznej funkcji. Na przykład:

```
def dekorator(func):
    def wrapper(*args, **kwargs):
        print(" naglowek ".center(30, '-'))
        func(*args, **kwargs)
        print(" stopka ".center(30, '='))
    return wrapper

def dokument(s):
    print(s.center(30))

txt = "lorem ipsum dolor sit amet"

# dokument(txt)
dokument = dekorator(dokument)
dokument(txt)
```

Python pozwala użyć dekoratorów w prostszy sposób, z pomocą symbolu `@` co zastępuje linię z kodem `dokument = dekorator(dokument)`, jak wyżej.

```
@dekorator
def dokument(s):
    print(s.center(30))
```

Teraz, można zorganizować kod w ten sposób, żeby łatwo wielokrotnie użyć takiego dekoratora. Zapiszmy jego definicję w osobnym pliku o nazwie `moje_dekoratory.py`, wtedy w kolejnym pliku będzie:

```
from moje_dekoratory import dekorator

@dekorator
def dok(s):
    print(s.center(30))

dok("hello from the other side") # wywołanie udekorowanej funkcji
```

Za pomocą atrybutu `__name__` można podejrzeć nazwy obiektów. Na przykład, `print(dekorator.__name__)` da nazwę dekorator. Problem pojawia się w przypadku funkcji opakowanej wewnętrznym `wrapper`, gdyż `print(dok.__name__)` da nazwę... `wrapper`. W celu zachowania właściwej nazwy, używa się jeszcze innego dekoratora z modułu `functools`, o nazwie `wraps` (funkcja wywołująca `update_wrapper()` – która aktualizuje `wrapper` aby wyglądał jak funkcja przez niego obkładana). Interesująca nas zmiana w kodzie:

```
import functools

def dekorator(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        ...
```

I wtedy `print(dok.__name__)` da poprawną nazwę `dok`. Uwaga: można też skorzystać z formatowania (i wydrukować w cudzysłowie dzięki `!r` które jest skrótem funkcji `repr()`): `print(f"{dok.__name__!r}")`

Funkcja może stać się dekoratorem również z użyciem modułu `decorator`. Wtedy nie trzeba zwracać wewnętrznej funkcji (closure), tylko kod wygląda wprost tak:

```
import decorator
#from decorator import decorator

@decorator.decorator
def mydec(func, arg, *args, **kwargs):
    print(' naglowek '.center(30, '-'))
    func(arg, *args, **kwargs)
    print(' stopka '.center(30, '='))

txt = "lorem ipsum dolor sit amet"

@mydec
def bar(n):
    print(n)

bar(txt)
```

Na zakończenie ciekawy przykład dekoratora, który mierzy czas wykonania funkcji i ostrzega, gdy jest on przekroczony:

```
from decorator import decorator

@decorator
def warn_slow(func, timelimit=60, *args, **kw):
    t0 = time.time()
    result = func(*args, **kw)
    dt = time.time() - t0
    if dt > timelimit:
        logging.warn('%s took %d seconds', func.__name__, dt)
    else:
```

```

        logging.info('%s took %d seconds', func.__name__, dt)
    return result

@warn_slow # warn if it takes more than 1 minute
def preprocess_input_files(inputdir, tempdir):
    ...

@warn_slow(timelimit=600) # warn if it takes more than 10 minutes
def run_calculation(tempdir, outdir):
    ...

```

WYRAŻENIA LAMBDA

Lambda to małe „anonimowe” wyrażenia, mogą wyglądać jak funkcje, (wraz z map, filter, reduce – patrz poniżej) będące składowymi paradygmatu programowania funkcyjnego. Wyrażenie lambda zapisujemy w jednej linii (chyba, że w nawiasie), nie można w nim robić notatek (annotations) na temat typów, nie mogą wystąpić instrukcje takie jak return, pass, assert, raise. Prosty przykład (funkcja identyczności):

```

def identycznosc(x):
    return x

```

Odpowiadające mu wyrażenie lambda:

```

lambda x: x

```

Wyrażenie lambda można użyć albo bezpośrednio wywołując:

```

(lambda x: x + 1)(3) # wynik 4, IIFE - Immediately Invoked Function Expression

```

albo najpierw przypisując do nazwy, a potem wywołując za pomocą tej nazwy:

```

fsum = lambda x: x + 1
fsum(3) # wynik 4

```

Może być też więcej zmiennych, np.

```

ftriple = lambda a,b,c: a*b*c
ftriple(2,3,4) # wynik 24

```

Zapiszmy ciąg Fibbonacciego jednolinijkowo:

```

hh = lambda x: hh(x-1) + hh(x-2) if x>1 else 1

```

Argumentem może być też coś bardziej złożonego np. inna funkcja (wyrażenie):

```

h2 = lambda x, y: x + y(x)
h2(3, lambda x: x**2) # tutaj w miejsce y podstawione kolejne wyrażenie lambda

```

Na niskim poziomie: nie ma różnic pomiędzy lambda i funkcją (za pomocą modułu **dis** wykonajmy prosty podgląd na kod maszynowy):


```
>>> prod = lambda a,b: a//b
>>> dis.dis(prod)
```

Efekt:

```
1      0 LOAD_FAST      0 (a)
      2 LOAD_FAST      1 (b)
      4 BINARY_FLOOR_DIVIDE
      6 RETURN_VALUE
```

```
def prod(a,b): return a//b
dis.dis(prod)
```

Efekt:

```
1      0 LOAD_FAST      0 (a)
      2 LOAD_FAST      1 (b)
      4 BINARY_FLOOR_DIVIDE
      6 RETURN_VALUE
```

W najnowszych wersjach Pythona poprawiły się komunikaty w przypadku błędu (np. dzielenie przez 0):

```
>>> prod(2,0) # dla lambda
-----
Traceback (most recent call last):
  File "<pyshell#13>", line 1, in <module>
    prod(2,0) # dla lambda
  File "<pyshell#12>", line 1, in prod
    def prod(a,b): return a//b
ZeroDivisionError: integer division or modulo by zero
-----
>>> prod(2,0) # dla funkcji
-----
Traceback (most recent call last):
  File "<pyshell#17>", line 1, in <module>
    prod(2,0) # dla funkcji
  File "<pyshell#16>", line 1, in prod
    def prod(a,b): return a//b
ZeroDivisionError: integer division or modulo by zero
```

Wyrażenia lambda mogą przyjąć takie same zestawy argumentów, jak funkcje:

```
>>> (lambda x, y, z: x + y + z)(1, 2, 3)
>>> (lambda x, y, z=3: x + y + z)(1, 2)
>>> (lambda x, y, z=3: x + y + z)(1, y=2)
>>> (lambda *args: sum(args))(1,2,3)
>>> (lambda **kwargs: sum(kwargs.values()))(one=1, two=2, three=3)
>>> (lambda x, *, y=0, z=0: x + y + z)(1, y=2, z=3)
```

Wracając do przykładów kodu z „dekoratorami”, wyrażen lambda nie da się „udekorować” za pomocą @, ale można wprost zdefiniować (i wysłać jako argument) w wywołaniu funkcji-dekoratora. W naszym przykładzie:

```
dok = dekorator((lambda x: print(x.center(30))))
dok(txt)
```

Dyskutowaliśmy też funkcje – domknięcia (closures), których zaletą było to, że przechwytyują wewnętrzne (lokalne) wartości funkcji, w której są zagnieżdżone i z nimi mogą być wywołane w dowolnym innym miejscu. Przykład dla przypomnienia:

```
def zewnetrzna(x):
    y = 4
    def wrap(z):
        print(f"x = {x}, y = {y}, z = {z}")
        return x + y + z
    return wrap

for i in range(3):
    closure = zewnetrzna(i)
    print(f"closure({i+5}) = {closure(i+5)}")
```

Wykonanie:

```
x = 0, y = 4, z = 5
closure(5) = 9
x = 1, y = 4, z = 6
closure(6) = 11
x = 2, y = 4, z = 7
closure(7) = 13
```

- x jest przekazany jako argument do **zewnetrzna()**
- y jest zmienną lokalną w **zewnetrzna()**
- z jest argumentem przekazanym do **wrap()**

Podobnie, wyrażenie lambda można użyć jako domknięcie (closure):

```
def zewnetrzna(x):
    y = 4
    return lambda z: x + y + z
for i in range(3):
    closure = zewnetrzna(i)
    print(f"closure({i+5}) = {closure(i+5)}")
```

Wykonanie:

```
closure(5) = 9
closure(6) = 11
closure(7) = 13
```

Z wykonaniem funkcji-dopełnienia i lambda-dopełnienia zdarzają się niespodzianki. Przykład:

```
def wrap(n):
    def f():
        print(n)
    return f

liczby = 'jeden', 'dwa', 'trzy'
funkcje = []
for n in liczby:
    funkcje.append(wrap(n)) # tu ewaluacja n

for f in funkcje:
    f()
```

Ten sam kod z wyrażeniem lambda:

```

liczby = 'jeden', 'dwa', 'trzy'
funkcje = []
for n in liczby:
    funkcje.append(lambda: print(n))
# n = 'cztery' # można przykładowo odkomentować
for f in funkcje:
    f() # wiązanie z bieżącą wartością n dopiero tutaj, podczas wykonania (niespodzianka)

```

Obejście problemu:

```

for n in liczby:
    funkcje.append(lambda n=n: print(n)) # teraz tutaj wiązanie z n

```

MAP

W wielu językach programowania `map` to nazwa funkcji wyższego rzędu, która stosuje daną funkcję do każdego elementu funktora, np. listy, zwracając listę wyników w tej samej kolejności. Koncepcja mapy nie ogranicza się do list: działa ona w przypadku kontenerów sekwencyjnych, drzew, a nawet kontenerów abstrakcyjnych (futures, promises). W Pythonie: `map(function, iterable, ...)` zwraca iterator, który stosuje funkcję do każdego elementu iterowalnego, dając wyniki. Jeśli przekazane są dodatkowe iterowalne argumenty, funkcja musi przyjąć te argumenty i jest stosowana równolegle do ich elementów. W przypadku wielu iteracji iterator zatrzymuje się, gdy najkrótsza iteracja zostanie wyczerpana.

Składnia:

```

map(func, *iterables) # zwraca obiekt mapy - generatora, jeśli chcemy listę, trzeba list( ... )

```

Normalnie w kodzie użylibyśmy jakiejś pętli, np.

```

my_pets = ['czarek', 'pikus', 'figa', 'arnold']
uppered_pets = []

for pet in my_pets:
    pet_ = pet.upper()
    uppered_pets.append(pet_)

print(uppered_pets)

```

Za pomocą mapy:

```

my_pets = ['czarek', 'pikus', 'figa', 'arnold']
uppered_pets = list(map(str.upper, my_pets))

print(uppered_pets)

```

Funkcją tu jest `str.upper`, która jest wywoływana przez `map` dla każdego argumentu (obiektu iterowanego z `my_pets`).

Z `map` można bezproblemowo przekazać więcej argumentów. Np. funkcja `round()` przyjmuje dwa, wartość zaokrąglaną i ile miejsc po przecinku:

```
liczby = [1.23456789, 1.23456789, 1.23456789, 1.23456789, 1.23456789, 1.23456789, 1.23456789, 1.23456789]
result = list(map(round, liczby, range(1,8))) # kończy się z krótszym z obiektów iterowanych
print(result)
```

Z pomocą map i wyrażenia lambda można zbudować własną funkcję zip. Zip() łączy elementy iterowane w pary krotki.

```
my_strings = ['a', 'b', 'c', 'd', 'e']
my_numbers = [1,2,3,4,5]
results = list(zip(my_strings, my_numbers))
print(results)
```

A teraz wersja z wyrażeniem lambda i map:

```
my_strings = ['a', 'b', 'c', 'd', 'e']
my_numbers = [1,2,3,4,5]
results = list(map(lambda x, y: (x, y), my_strings, my_numbers))
print(results)
```

FILTER

Podobne w działaniu, ale wykonuje dyskryminację elementów iterowanych przez **funkcję, która zwraca wartość logiczną** – iterowana może być tylko jedna kolekcja. Składnia:

```
filter(func, iterable)
```

Przykład:

```
liczby = [1,2,3,4,5,6,7,8,9]

def wiecej_niz_5(n):
    return n > 5
wiecej_niz = list(filter(wiecej_niz_5, liczby))
print(wiecej_niz)
```

Inny przykład z wyrażeniem lambda (znajdywanie palindromów):

```
pal = ("ada", "abrakadabra", "madam", "potop", "kajak", "kiosk")
palindromes = list(filter(lambda word: word == word[::-1], pal))
print(palindromes)
```

REDUCE

Aby skorzystać z **reduce** należy (Python 3) zaimportować:

```
from functools import reduce
```

Składnia:

```
reduce(func, iterable[, initial])
```

Działanie – przebiega po wielkościach iterowalnych, przekazując z nich dwa argumenty do func (gdy nie ma initial). Jeśli wartość initial jest obecna, to jest ona pierwszym argumentem, drugi (i kolejne) jest z iterable. Jeśli nie ma initial, to pierwszy i drugi (i kolejne) argumenty są pobierane z iterable, redukując kolekcję wartości do jednej wartości. Np. silnię można policzyć tak:

```
from functools import reduce
numbers = [1,2,3,4,5,6]

def silnia(n1, n2):
    return n1 * n2

print(reduce(silnia, numbers))
```

Przykład z wartością początkową oraz wyrażeniem lambda w miejscu funkcji:

```
from functools import reduce
slova = ['alfa', 'beta', 'gamma', 'delta']
print(reduce(lambda x,y: x+' '+y, slova, 'Oto slova:')) # Oto slova: alfa beta gamma delta
```

6. PRZESTRZENIE NAZW

Python jest językiem dynamicznego typowania, nazwy pojawiają się wraz z przypisaniem im jakiejś wartości lub znaczenia, na pięć sposobów: 1) przyisanie (x = wartość), 2) import modułu (import module lub from module import nazwa), 3) definicja funkcji def fun(): ..., 4) argumenty funkcji def fun(arg1, arg2, ..., argN): ..., 5) definicja klasy class MojaKlasa: ...

Podstawowe kategorie zakresów: lokalny (np. wewnątrz funkcji) i globalny (na zewnątrz wszelkich funkcji, w pliku). W Pythonie pojęcie zakresu związane jest ściśle z pojęciem „przestrzeni nazw” (namespace) – miejsca, gdzie składowana jest zmienna. Namespace realizowane są jako słowniki (lokalne, globalne, wbudowane built-in, zagnieżdżone w obiektach / metodach). Nazwy są dostępne poprzez atrybut `__dict__`. Na przykład w modułach, jest to `modul.__dict__`

Można sprawdzić, np.

```
>>> import sys
>>> sys.__dict__.keys() # zwraca listę
>>> dict_keys(['__name__', '__doc__', '__package__', '__loader__', '__spec__', 'addaudithook',
'audit', 'breakpointhook', 'callstats', '_clear_type_cache', '_current_frames', 'displayhook',
'exc_info', 'excepthook', 'exit', 'getdefaultencoding', 'getallocatedblocks',
'getfilesystemencoding', 'getfilesystemcodeerrors', 'getrefcount', 'getrecursionlimit',
'getsizeof', '_getframe', 'getwindowsversion', '_enablelegacywindowsfsencoding', 'intern',
'is_finalizing', 'setcheckinterval', 'getcheckinterval', 'setswitchinterval',
'getswitchinterval', 'setprofile', 'getprofile', 'setrecursionlimit', 'settrace', 'gettrace',
'call_tracing', '_debugmallocstats', 'set_coroutine_origin_tracking_depth',
'get_coroutine_origin_tracking_depth', 'set_asyncgen_hooks', 'get_asyncgen_hooks',
'unraisablehook', 'modules', 'stderr', '_stderr_', '_displayhook_', '_excepthook_',
'_breakpointhook_', '_unraisablehook_', 'version', 'hexversion', '_git', '_framework',
'api_version', 'copyright', 'platform', 'maxsize', 'float_info', 'int_info', 'hash_info',
'maxunicode', 'builtin_module_names', 'byteorder', 'dllhandle', 'winver', 'version_info',
'implementation', 'flags', 'float_repr_style', 'thread_info', 'meta_path',
'path_importer_cache', 'path_hooks', 'path', 'executable', '_base_executable', 'prefix',
```

```
'base_prefix', 'exec_prefix', 'base_exec_prefix', 'pycache_prefix', 'argv', 'warnoptions',
'_xoptions', 'dont_write_bytecode', '__stdin__', 'stdin', '__stdout__', 'stdout', '_home',
'__interactivehook__', 'ps1', 'ps2', 'ps3', 'last_type', 'last_value', 'last_traceback']])
```

Wbudowana przestrzeń nazw zawiera nazwy wszystkich wbudowanych obiektów Pythona. Są one dostępne przez cały czas działania Pythona. Lista obiektów we wbudowanej przestrzeni nazw:

```
>>> dir(__builtins__)
```

Interpreter Pythona tworzy tę przestrzeń nazw podczas uruchamiania i istnieje ona do momentu zakończenia działania interpretera. Python implementuje ją jako moduł:

```
>>> type(__builtins__)
<class 'module'>
```

Co ciekawe, z pomocą `__builtins__` można odwołać się do nazw typów nadpisanych przez program. Na przykład, jeśli „niechcący” zdefiniujemy listę o nazwie takiej jak typ:

```
>>> list = []
>>> print(list)
[]
```

to zasłaniamy naszym obiektem wbudowane `list` i próba użycia skończy się następująco:

```
>>> list('abcd')
Traceback (most recent call last):
  File "<pysHELL#212>", line 1, in <module>
    list('abcd')
TypeError: 'list' object is not callable
```

Wtedy można:

```
>>> __builtins__.list('abcd')
['a', 'b', 'c', 'd']
```

jak również skasować nasz obiekt (`del list`) i wszystko wróci do normy.

Globalna przestrzeń nazw zawiera dowolne nazwy zdefiniowane na poziomie programu głównego. Python tworzy globalną przestrzeń nazw, gdy zaczyna się główna treść programu i pozostaje ona do momentu zakończenia działania interpretera. Może to nie być jedyna istniejąca globalna przestrzeń nazw. Interpreter tworzy również globalną przestrzeń nazw dla każdego modułu, który program ładuje za pomocą instrukcji `import`.

Interpreter tworzy nową przestrzeń nazw za każdym razem, gdy funkcja jest wykonywana. Ta przestrzeń nazw jest lokalna dla funkcji i istnieje do momentu jej zakończenia.

Wreszcie możemy mieć też do czynienia z zagnieżdżoną definicją funkcji wewnątrz innej funkcji, przykładowo, niech w definicji funkcji `f()` jest zagnieżdżona definicja funkcji `g()`, po której, wewnątrz funkcji `f()` następuje wywołanie `g()`. Gdy główny program wywołuje `f()`, Python tworzy nową przestrzeń nazw dla `f()`. Podobnie, gdy `f()` wywołuje `g()`, `g()` otrzymuje własną, oddzielną przestrzeń nazw. Przestrzeń nazw utworzona dla `g()` to lokalna przestrzeń nazw, a przestrzeń nazw utworzona dla `f()` to tzw. otaczająca (enclosing)

przestrzeni nazw. Każda z tych przestrzeni nazw istnieje, dopóki jej odpowiednia funkcja nie zostanie zakończona. Python może nie odzyskać od razu pamięci przydzielonej dla tych przestrzeni nazw, gdy ich funkcje się zakończą, ale wszystkie odwołania do zawartych w nich obiektów przestają być ważne.

Istnienie wielu odrębnych przestrzeni nazw oznacza, że kilka różnych wystąpień określonej nazwy może istnieć jednocześnie podczas działania programu w Pythonie. Dopóki każda instancja znajduje się w innej przestrzeni nazw, wszystkie są utrzymywane osobno i nie będą ze sobą kolidować. Jeżeli jakaś nazwa się powtarza, skąd Python wie, której użyć w danym miejscu? Zależy to od zakresu (scope) ważności danej nazwy. Zakres nazwy to ta część programu, w którym ta nazwa ma znaczenie. Interpreter określa to w czasie wykonywania na podstawie tego, gdzie występuje definicja nazwy i gdzie w kodzie występuje odwołanie do nazwy. Wyszukiwanie nazw odbywa się według reguły LEGB (Local, Enclosing, Global, and Built-in). Jeśli interpreter nie znajdzie nazwy w żadnej z tych lokalizacji, Python zgłosi wyjątek `NameError`.

```
>>> x = "global" # gdy nie ma local lub enclosing
>>> def f():
    x = "enclosing" # gdy nie ma local
    def g():
        x = "local" # najpierw wybrane
        print(x)
    g()

>>> f()
local
```

Python implementuje globalne i lokalne przestrzenie nazw jako słowniki. Wbudowane funkcje zwane `globals()` i `locals()`, umożliwiają dostęp do globalnych i lokalnych słowników przestrzeni nazw. Wbudowana (built-in) przestrzeń nazw nie zachowuje się jak słownik.

Wbudowana funkcja `globals()` zwraca odwołanie do bieżącego słownika globalnej przestrzeni nazw, którą można użyć, aby uzyskać dostęp do obiektów w globalnej przestrzeni nazw.

```
>>> type(globals())
<class 'dict'>
>>> globals()
{'__name__': '__main__', '__doc__': None, '__package__': None, '__loader__': <class 'frozen_importlib.BuiltinImporter'>, '__spec__': None, '__annotations__': {}, '__builtins__': <module 'builtins' (built-in)>}
```

Jeśli zdefiniujemy globalną zmienną `x`, to jest ona widoczna również jako wpis w słowniku `globals()`

```
>>> x = "abc"
>>> globals()
{'__name__': '__main__', '__doc__': None, '__package__': None, '__loader__': <class 'frozen_importlib.BuiltinImporter'>, '__spec__': None, '__annotations__': {}, '__builtins__': <module 'builtins' (built-in)>, 'x': 'abc'}
```

Oprócz zwykłego sposobu użycia obiektu za pomocą jego nazwy, można też dostać się do niego za pomocą `globals()`, a za pomocą operatora `is` sprawdzimy, że jest to rzeczywiście ten sam obiekt:

```
>>> x
'abc'
>>> globals()["x"]
'abc'
>>> x is globals()["x"]
True
```

Podobnie, można stworzyć nową zmienną:

```
>>> globals()["y"] = "nowy obiekt"
>>> globals()["y"]
'nowy obiekt'
>>> globals()
{'__name__': '__main__', '__doc__': None, '__package__': None, '__loader__': <class 'frozen_importlib.BuiltinImporter'>, '__spec__': None, '__annotations__': {}, '__builtins__': <module 'builtins' (built-in)>, 'x': 'abc', 'y': 'nowy obiekt'}
```

Python zapewnia również funkcję o nazwie `locals()`, za pomocą której uzyskujemy dostęp do obiektów w lokalnej przestrzeni nazw:

```
>>> def f(x,y,z):
    tekst = "abc"
    print(locals())

>>> f(1,2.2,"xyz")
{'x': 1, 'y': 2.2, 'z': 'xyz', 'tekst': 'abc'}
```

`locals()` wywołane w przestrzeni globalnej daje to samo co `globals()`. Subtelna różnica pomiędzy obiema funkcjami jest też taka, że `globals()` zwraca referencję do aktualnego słownika obiektów globalnych, tak że modyfikacja tej zawartości jest odzwierciedlona np. w zmiennej, do której przypisaliśmy `globals()`:

```
>>> x = globals()
>>> x
{'__name__': '__main__', '__doc__': None, '__package__': None, '__loader__': <class 'frozen_importlib.BuiltinImporter'>, '__spec__': None, '__annotations__': {}, '__builtins__': <module 'builtins' (built-in)>, 'x': {...}}
>>> y = "abc" # dodajemy obiekt globalny
>>> x
{'__name__': '__main__', '__doc__': None, '__package__': None, '__loader__': <class 'frozen_importlib.BuiltinImporter'>, '__spec__': None, '__annotations__': {}, '__builtins__': <module 'builtins' (built-in)>, 'x': {...}, 'y': 'abc'}
>>> # jak widać jest on również pod nazwą x, czyli aktualna zawartość
```

Natomiast `locals()` zwraca kopię słownika, więc modyfikacja lokalnych obiektów nie jest w niej odzwierciedlona:


```
>>> def f():
    tekst = "abc"
    x = locals()
    print(x)
    y = 1.23
    print(x) # bez y
    x["tekst"] = "zmiana" # zmiana w kopii tylko
    print(tekst) # lokanie tekst bez zmian

>>> f()
{'tekst': 'abc'}
{'tekst': 'abc'}
abc
```

x zawiera kopię lokalnych obiektów, więc dopisanie y nie ujawnia się w wypisanym ponownie x, i odwrotnie, modyfikacja obiektu tekst pozyskanego pod nazwą x, nie wpływa na obiekt lokalny tekst, ponieważ pod x jest jego kopia.

Modyfikacja obiektów spoza bieżącego zasięgu

W przypadku obiektów niemodyfikowalnych (typy proste), funkcja nie może zmienić obiektu spoza swojego lokalnego zasięgu, bo powstaje zawsze kopia. W przypadku typów modyfikowalnych, w funkcji może powstać też kopia, ale możliwa jest też modyfikacja oryginalnego obiektu (in-place).

```
>>> x = 1 # globalny
>>> def f():
    x = 2 # lokalny
    print(x)

>>> f()
2
>>> x
1
```

Typ `int` reprezentuje obiekty niemodyfikowalne i jak widać, lokalny x jest niezależny od globalnego x. Inaczej jest w przypadku modyfikowalnej np. listy:

```
>>> x = [0, 0, 0] # globalna
>>> def f():
    x[0] = 1 # modyfikacja elementu listy
    print(x)

>>> f(); x
[1, 0, 0]
[1, 0, 0]
```

Jeśli jednak wewnątrz funkcji przypiszemy do nazwy `x` listę, to będzie to utworzenie lokalnego, innego niż globalny, obiektu.

```
>>> x = [0, 0, 0] # globalna
>>> def f():
        x = [1, 1, 1] # lokalna
        print(x)

>>> f(); x
[1, 1, 1]
[0, 0, 0]
```

Podobną obserwację można wykonać dla zmiennych w obszarze otaczającym (enclosing) wewnątrz funkcji `f()` i lokalnym (local) dla zagnieżdżonej funkcji `g()`.

```
>>> def f():
        x = 0 # enclosing
        Lx = [0, 0, 0] # enclosing
        print(x, Lx)
        def g():
            x = 1 # local
            Lx = [1, 1, 1] # local
            print(x, Lx)
        g() # wywołanie zagnieżdżonej funkcji
        print(x, Lx) # obiekty z obszaru f()

>>> f()
0 [0, 0, 0]
1 [1, 1, 1]
0 [0, 0, 0]
```

Python pozwala na sięgnięcie do obiektów globalnych z lokalnego zakresu, do tego celu służy deklaracja `global`.

```
>>> x = 1 # globalny
>>> def f():
        global x # będziemy pracować na globalnym x
        x = 2
        print(x)

>>> f(); print(x)
2
2
```

Co ciekawe, gdyby globalny `x` nie istniał, to deklaracja `global x` wewnątrz definicji funkcji stworzy taki obiekt w przestrzeni globalnej.

```

>>> x # nie istnieje
Traceback (most recent call last):
  File "<pyshell#26>", line 1, in <module>
    x # nie istnieje
NameError: name 'x' is not defined
>>> def f():
        global x
        x = 1
        print(x)

>>> f(); print(x)
1
1

```

Podobnie, możliwe jest sięgnięcie do obiektów otaczających (enclosing) z wnętrza zagnieżdżonej funkcji, za pomocą deklaracji **nonlocal** (przy czym za pomocą **nonlocal** nie da się utworzyć nieistniejącego wcześniej obiektu, natomiast z poziomu zagnieżdżenia można utworzyć nieistniejący obiekt globalny).

```

>>> def f():
        x = 1 # enclosing
        def g():
            nonlocal x # sięgamy po x z f()
            x = 2
            global y # tworzymy nieistniejący globalny y
            y = 3

        g()
        print(x, y)

>>> f()
2 3
>>> y
3

```

Obydwie możliwości sięgania do obiektów spoza zakresu, za pomocą **global** lub **nonlocal**, nie są rekomendowanym sposobem zmieniania wartości obiektów. Lepiej jest wykonywać to za pomocą wartości zwracanej przez funkcję.

7. MODUŁY I PAKIETY

Podział kodu pomiędzy różne pliki jest naturalnym procesem wytwarzania czytelnego i dobrego strukturalnie kodu. Służy prostocie, łatwości utrzymania, możliwości wielokrotnego wykorzystania oraz tworzenia odrębnych, izolowanych przestrzeni nazw, zapobiegających kolizjom. Moduł (najczęściej kojarzony jako odrębny plik z kodem), może być napisany

całkowicie w Pythonie, może być napisany w języku C i ładowany dynamicznie podczas wykonania, jak np. moduł wyrażeń regularnych **re**. Moduł wbudowany jest wewnętrznie zawarty w interpreterze języka, jak np. moduł **itertools**. Podstawowy sposób uzyskania dostępu do każdego z tych rodzajów modułów jest poprzez instrukcję **import**. Najprostszy moduł to dowolny kod Pythona, zapisany w pliku z rozszerzeniem **.py**.

Instrukcja **import** szuka danego modułu na liście katalogów zebranych z następujących źródeł: 1) katalog, z którego uruchomiono skrypt wejściowy lub bieżący katalog, jeśli interpreter jest uruchamiany interaktywnie, 2) lista katalogów zawartych w zmiennej środowiskowej **PYTHONPATH**, jeśli jest ustawiona (format **PYTHONPATH** jest taki jak zmienna środowiskowa **PATH**), 3) zależna od instalacji lista katalogów skonfigurowanych w czasie instalacji Pythona. Wynikowa ścieżka wyszukiwania jest dostępna w zmiennej Pythona **sys.path**, którą uzyskuje się z modułu o nazwie **sys**:

```
>>> import sys
>>> sys.path
['', 'C:\\Python39\\Lib\\idlelib', 'C:\\Python39\\python39.zip', 'C:\\Python39\\DLLs', 'C:\\Python39\\lib', 'C:\\Python39', 'C:\\Python39\\lib\\site-packages', 'C:\\Python39\\lib\\site-packages\\win32', 'C:\\Python39\\lib\\site-packages\\win32\\lib', 'C:\\Python39\\lib\\site-packages\\Pythonwin']
```

Powyższą listę ścieżek można uzupełnić dynamicznie podczas wykonania:

```
>>> sys.path.append(r"C:\tmp\PYTHON")
```

Kiedy już załadujemy moduł, można sprawdzić jego lokalizację za pomocą atrybutu **__file__**:

```
>>> import numpy
>>> numpy.__file__
'C:\\Python39\\lib\\site-packages\\numpy\\__init__.py'
```

Wyrażenie **import** może przyjmować kilka postaci. Najprostsza: **import <module_name>**. Zawartość modułu jest bezpośrednio dostępna dla wywołującego. Każdy moduł posiada własną prywatną tablicę symboli, która służy jako globalna tablica symboli dla wszystkich obiektów zdefiniowanych w module. W ten sposób moduł tworzy oddzielną przestrzeń nazw.

Instrukcja **import <nazwa_modułu>** umieszcza moduł **<nazwa_modułu>** w tabeli symboli wywołującego. Obiekty zdefiniowane w module pozostają w prywatnej tablicy symboli modułu. Obiekty w module są dostępne tylko wtedy, gdy mają przedrostek **<nazwa_modułu>** w notacji kropkowej (tak jak widzieliśmy już na przykładzie **import sys** a potem **sys.path**). Moduł **sys** jest przykładem modułu wbudowanego.

```
>>> import sys, numpy
>>> sys
<module 'sys' (built-in)>
>>> numpy
<module 'numpy' from 'C:\\Python39\\lib\\site-packages\\numpy\\__init__.py'>
```

W jednej instrukcji importu można określić kilka modułów oddzielonych przecinkami:

```
import <module_name>[, <module_name> ...]
```

Alternatywna forma instrukcji import umożliwia importowanie poszczególnych obiektów z modułu bezpośrednio do tablicy symboli wywołującego:

```
from <module_name> import <name(s)>
```

Po wykonaniu powyższej instrukcji, do nazw <name(s)> można odwoływać się w środowisku wywołującego bez przedrostka <module_name>. Ponieważ ta forma importu umieszcza nazwy obiektów bezpośrednio w tablicy symboli wywołującego, to wszelkie istniejące już wcześniej obiekty o tej samej nazwie zostaną nadpisane. Możliwe jest nawet zaimportowanie wszystkiego z modułu:

```
from <module_name> import *
```

Spowoduje to umieszczenie nazw wszystkich obiektów z <nazwa_modułu> w lokalnej tablicy symboli, z wyjątkiem tych, które zaczynają się od znaku podkreślenia (_). Składnia taka jest wysoce ryzykowna, ponieważ prowadzi do zaimportowania dużej liczby nazw, natomiast jest wygodna podczas testowania, gdyż mamy do dyspozycji nazwy z modułu, bez konieczności pisania poprzedzającej nazwy modułu i kropki. W importowanym module można zdefiniować listę obiektów, które mają zostać zaimportowane w przypadku użycia powyższej składni z gwiazdką. Nazwy podaje się w liście przypisanej do atrybutu `__all__`. Przykładowo, jeśli w pliku `test.py` mamy:

```
__all__ = ["g"]

def f():
    print("f")

def g():
    print("g")
```

zaś w głównym pliku:

```
from test import *
print(dir())
```

to zobaczymy:

```
['__annotations__', '__builtins__', '__cached__', '__doc__', '__file__',
 '__loader__', '__name__', '__package__', '__spec__', 'g']
```

Jak widać, nie ma w głównej przestrzeni nazw nazwy funkcji `f`.

Możliwe jest również importowanie pojedynczych obiektów, ale z wprowadzeniem ich do lokalnej tablicy symboli z alternatywnymi nazwami, dzięki czemu unikanie konfliktów z wcześniej istniejącymi nazwami, według następującej składni:

```
from <module_name> import <name> as <alt_name>[, <name> as <alt_name> ...]
```

Można również zaimportować cały moduł pod alternatywną nazwą:

```
import <module_name> as <alt_name>
```

Moduły lub wybrane ich elementy mogą być importowane wewnątrz definicji funkcji:

```
def f():
    from timeit import timeit
    start = timeit()
    print("hello")
    end = timeit()
    print(end - start)
```

Aczkolwiek nie można zaimportować w ten sposób wszystkich nazw (za pomocą gwiazdki):

```
>>> def f():
    from timeit import *

SyntaxError: import * only allowed at module level
```

Do określenia jakie nazwy zostały załadowane w bieżącej przestrzeni nazw, pomocna może być wbudowana funkcja `dir()`. Przykładowo, `dir()` w sesji interaktywnej oraz po zaimportowaniu wszystkich nazw z modułu `sys`:

```
>>> dir()
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__', '__pack
age__', '__spec__']
>>> from sys import *
>>> dir()
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__', '__pack
age__', '__spec__', 'addaudithook', 'api_version', 'argv', 'audit', 'base_exec_p
refix', 'base_prefix', 'breakpointhook', 'builtin_module_names', 'byteorder', 'c
all_tracing', 'copyright', 'displayhook', 'dllhandle', 'dont_write_bytecode', 'e
xc_info', 'excepthook', 'exec_prefix', 'executable', 'exit', 'flags', 'float_inf
o', 'float_repr_style', 'get_asyncgen_hooks', 'get_coroutine_origin_tracking_dep
th', 'getallocatedblocks', 'getdefaultencoding', 'getfilesystemencodingerrors', 'g
etfilesystemencoding', 'getprofile', 'getrecursionlimit', 'getrefcount', 'getsiz
eof', 'getswitchinterval', 'gettrace', 'getwindowsversion', 'hash_info', 'hexver
sion', 'implementation', 'int_info', 'intern', 'is_finalizing', 'maxsize', 'maxu
nicode', 'meta_path', 'modules', 'path', 'path_hooks', 'path_importer_cache', 'p
latform', 'platlibdir', 'prefix', 'pycache_prefix', 'set_asyncgen_hooks', 'set_c
oroutine_origin_tracking_depth', 'setprofile', 'setrecursionlimit', 'setswitchin
terval', 'settrace', 'stderr', 'stdin', 'stdout', 'thread_info', 'unraisablehook
', 'version', 'version_info', 'warnoptions', 'winver']
```

Można wykonać import modułu i podejrzeć nazwy, podając nazwę modułu jako argument `dir`, czyli np. `import sys`, potem `dir(sys)`. Generalnie, moduł napisany w Pythonie może być zarówno zaimportowany jak i uruchomiony. Jeżeli w treści modułu chcemy wykonywać jakieś operacje wejścia/wyjścia, to raczej w przypadku wykonywania pliku jako programu, a nie podczas importu. Python udostępnia specjalne atrybuty (*dunder variables*) <https://docs.python.org/3/reference/datamodel.html> wśród których szczególnie przydatne jest `__name__`, któremu przypisana jest nazwa `__main__` podczas wykonania skryptu jako programu lub nazwa pliku podczas importu. Można to sprawdzić na jednym pliku – zapiszmy plik o nazwie `test.py` o następującej zawartości:

```
print("nazwa: ", __name__)
import test # importujemy plik do siebie samego
if __name__ == "__main__":
    import sys # importujemy moduł sys
    print(sys.__name__) # drukujemy jego nazwę
```

W wyniku wywołania zobaczymy:

```
nazwa: __main__
nazwa: test
sys
```

Ta własność jest powszechnie stosowana do zapisania odpowiedniego wykonania skryptu, często połączona z wywołaniem funkcji o nazwie **main()** zdefiniowanej oczywiście przez użytkownika, co upodabnia program Pythona do innych języków programowania.

Ze względu na wydajność, importowany moduł jest ładowany tylko raz w sesji interpretera, co jest wystarczające dla definicji funkcji i klas, które zazwyczaj stanowią większość zawartości modułu. Moduł może również zawierać instrukcje wykonywalne, zwykle jakieś inicjalizacje. Należy pamiętać, że te instrukcje zostaną wykonane tylko przy pierwszym zaimportowaniu modułu, np. jeśli w pliku `test.py` zapiszemy linię `print("drukuje")`, to kilkukrotne wywołanie:

```
import test # drukuje
import test
import test
```

wywoła `print` tylko za pierwszym razem. Możliwe jest ponowne załadowanie modułu, za pomocą funkcji **reload** z modułu **importlib** (tak jest od Pythona w wersji 3.4+), czyli np.

```
import importlib
importlib.reload(test)
```

Należy zachować czujność, ponieważ takie przeładowanie wykonywane jest tylko dla danego modułu, a nie modułów w nim zagnieżdżonych, nie odświeża też ewentualnych referencji ustawionych na obiekty przeładowywanego modułu – trzeba o to zadbać osobno. Jedną z możliwości wykonania pełnego przeładowania (deep reload) jest użycie rozszerzeń IPython lub Jupyter (wspomnianych na początku tych notatek). Składniowo, żeby nie było kolizji, wyglądałoby to przykładowo tak:

```
from IPython.lib.deepreload import reload as dreload
dreload(test)
```

PAKIETY W PYTHONIE

Pakiety (packages) umożliwiają utworzenie hierarchicznej struktury modułów i ich przestrzeni nazw, z użyciem notacji z odniesieniem przez kropkę. Ta struktura wykorzystuje własności systemów operacyjnych, pozwalających na tworzenie katalogów i podkatalogów, wraz

zawierającymi je plikami. W ten sam sposób, w jaki moduły pomagają uniknąć kolizji między nazwami zmiennych globalnych, pakiety pomagają uniknąć kolizji między nazwami modułów.

Stworzenie prostego pakietu to utworzenie katalogu oraz umieszczenie w nim plików z rozszerzeniem .py. Ilustrując przykładem, katalog **pkg** będzie zawierał jakieś pliki, niech plik **md1.py**:

```
def fun1():
    print("fun1")

class K1:
    pass
```

i analogicznie, plik **md2.py** niech ma funkcję **fun2** i klasę **K2**, a plik **md3.py** funkcję **fun3** i klasę **K3**. Jeśli sam katalog **pkg** jest w lokacji widzialnej dla Pythona, dla poszczególnych modułów można dostać się poprzez **import pkg.md1**, **pkg.md2**, **pkg.md3**. Po zaimportowaniu do składowych nazw również odnosimy się podając odpowiednią strukturę, np. **pkg.md1.fun1()**. Działają również alternatywne metody importu, czyli **from pkg.md1 import fun1** i wtedy **fun1()** jest bezpośrednio dostępne, **from pkg.md2 import K2 as Klasa2**, wtedy klasa **K2** jest dostępna pod nazwą **Klasa2**. Natomiast **import pkg** nie prowadzi do niczego, poza pojawieniem się nazwy **pkg** w głównej przestrzeni nazw, nie następuje automatyczny import. Aby jakaś akcja miała miejsce, konieczne jest umieszczenie pliku o nazwie **__init__.py** w podkatalogu z modułami, w naszym przykładzie obok plików **md1.py** itd.



Plik ten może być pusty, ale może też być w nim zapisana jakaś pożyteczna praca do wykonania podczas importowania pakietu. Można utworzyć jakieś obiekty, które będą dostępne dla modułów pakietu, można wreszcie zaimportować same moduły.

```
print("Importowanie pkg")
L = [ 1, 2, 3 ]
import pkg.md1
from pkg.md2 import fun2 as fun2
from pkg.md3 import K3 as K3
```

Również z punktu widzenia poszczególnych modułów, mogą one uzyskać dostęp do obiektów z przestrzeni **__init__.py**, albo z sąsiednich modułów, przykładowo:

```
def fun1():
    print("fun1")
    from pkg import L
    print(L)
    from pkg.md2 import fun2
    fun2()
```

Jeśli zawartość pliku **__init__.py** jest jak wyżej, oraz w jakimś pliku wykonamy:

```
import pkg
pkg.md1.fun1()
```

spowodujemy wykonanie tej zawartości, czyli wypisanie na ekranie:

Importowanie pkg

fun1

[1, 2, 3]

fun2

Założmy na chwilę, że w podkatalogu **pkg** nie ma pliku `__init__.py` (nie jest on obowiązkowy). Wtedy również składnia **from pkg import *** nie da żadnego efektu. Po wykonaniu:

```
print("Przed: ")
print(dir())
from pkg import *
print("Po: ")
print(dir())
```

zobaczymy:

Przed:

```
['__annotations__', '__builtins__', '__cached__', '__doc__', '__file__', '__loader__',
 '__name__', '__package__', '__spec__']
```

Po:

```
['__annotations__', '__builtins__', '__cached__', '__doc__', '__file__', '__loader__',
 '__name__', '__package__', '__spec__']
```

Aby nastąpił import modułów, w pliku `__init__.py` powinna się znaleźć lista `__all__`, z nazwami modułów do zaimportowania. Na przykład,

```
__all__ = [ "md2", "md3" ]
```

będzie skutkowało taki sam kod jak poprzednio, pojawieniem się nazw md2 i md3.

Po:

```
['__annotations__', '__builtins__', '__cached__', '__doc__', '__file__', '__loader__',
 '__name__', '__package__', '__spec__', 'md2', 'md3']
```

W tej przestrzeni nazw można wtedy odwołać się do składowych modułów md2 i md3, np. `md2.fun2()`.

Możliwa jest również organizacja pakietów i pakietów zagnieżdżonych. Obowiązuje wtedy taka sama notacja z wypisaniem struktury nazw kolejnych katalogów i podkatalogów, oddzielonych kropką. Przykładowo, niech teraz główny katalog **pkg** zawiera dwa podkatalogi (subpakiety) **pkg1** i **pkg2**, oraz plik `__init__.py`

```
print(__name__)
from pkg.pkg1 import *
from pkg.pkg2 import *
```

W podkatalogu **pkg1** będą pliki moduły `md1.py` oraz `md2.py` i `__init__.py`

```
print(__name__)
__all__ = [ "md1", "md2" ]
```

zaś w podkatalogu pkg2 będzie plik moduł md3.py oraz `__init__.py`

```
print(__name__)
__all__ = [ "md3" ]
```

Dla takiej struktury, wykonując główny program:

```
from pkg import *
print(dir())
md1.fun1()
```

zobaczymy wydruki:

```
pkg
pkg.pkg1
pkg.pkg2
['__annotations__', '__builtins__', '__cached__', '__doc__', '__file__', '__loader__',
 '__name__', '__package__', '__spec__', 'md1', 'md2', 'md3', 'pkg1', 'pkg2']
fun1
```

Jak widać, w globalnej przestrzeni nazw ostatecznie są zaimportowane moduły md1, md2 i md3, które dają dostęp do obiektów w nich zawartych.

Na koniec warto zauważyć, że poszczególne zagnieżdżone moduły też mogą sięgać po zawartość innych modułów, nawigacja w składni import przypomina nawigację po strukturze katalogów. Wtedy `..` (dwie kropki) oznaczają przejście na poziom katalogu nadrzędnego, zaś zapis `..<nazwa_katalogu>` oznacza przejście o katalog wyżej i odniesienie się do katalogu (pakietu) o nazwie `<nazwa_katalogu>`. Przykładowo, jeśli chcemy w md3.py zrobić import subpakietu pkg1 oraz zaimportować konkretnie z md2 funkcję fun2, w pliku md3.py napiszemy:

```
from .. import pkg1
print(pkg1)

from ..pkg1.md2 import fun2
fun3()
```

zaś w głównym programie:

```
from pkg.pkg2 import md3
```

W efekcie powinniśmy zobaczyć coś podobnego do (zależnie od lokacji plików):

```
pkg
pkg.pkg1
pkg.pkg2
<module 'pkg.pkg1' from 'c:\\tmp\\PYTHON\\TEST\\pkg\\pkg1\\__init__.py'>
fun2
```

Podsumowując sytuacje związane z importowaniem, począwszy od wersji Pythona 3.3 wprowadzono niejawne pakiety przestrzeni nazw (implicit namespace packages), zobacz dokument PEP 420 <https://www.python.org/dev/peps/pep-0420/>. Oznacza to, że istnieją trzy typy obiektów, które można utworzyć poprzez import (np. niech będzie import test):

- 1) moduł reprezentowany przez plik test.py
- 2) zwykły pakiet, reprezentowany przez katalog test zawierający plik `__init__.py`
- 3) pakiet przestrzeni nazw, reprezentowany przez jeden lub więcej katalogów bez żadnych plików `__init__.py`

8. KLASY

DRY- don't repeat yourself. Tworzenie klas, budowanie z nich złożonych typów, a także dziedziczenie pomagają w optymalnym używaniu napisanego kodu. Definicja klasy, która na razie jest pusta:

```
class Czlowiek: # konwencja: piszmy nazwy klas z wielkiej litery
    pass
```

Python jest językiem typowania dynamicznego, dlatego również składowe klasy, które chcemy mieć, tworzone są poprzez inicjalizację, podczas budowania obiektu, za pomocą funkcji `__init__()`, będącej funkcją inicjalizującą (używanie nazwy „konstruktor” jest tu nadużyciem, ponieważ model tworzenia obiektu w Pythonie jest po prostu inny, opiszemy to później). Funkcja `__init__` może mieć różne argumenty, ale pierwszy z nich jest „specjalny” i zwyczajowo nazywa się **self** – reprezentuje bieżącą instancję obiektu. Na przykład, rozwińmy zawartość klasy:

```
class Czlowiek:
    def __init__(self, imie, nazwisko, wiek=20): # np. wartość domyślna dla wiek
        self.imie = imie
        self.nazwisko = nazwisko
        self.wiek = wiek
```

Metody składowe klasy, które mają pracować na konkretnym obiekcie (instancji), mają „zarezerwowany” pierwszy argument jako ten, który przekazuje metodzie referencję do bieżącego obiektu. Podkreślmy, że nazwa pierwszego argumentu jest *dowolna* i wręcz można sobie stosować w każdej funkcji inną – jednak z powodu zwyczajowej praktyki wszyscy używają i piszą nazwę **self**. W innych językach, np. w C++ ta informacja o obiekcie przekazywana jest każdej niestatycznej metodzie składowej klasy niejawnie, za pomocą wskaźnika **this**. Można więc powiedzieć, że w C++ ten niejawny „argument” jest wyróżniony, podczas gdy w Pythonie jest on taki sam jak inne argumenty i przekazywany jawnie, na liście argumentów, jako pierwszy. Oprócz funkcji instancji, w klasie mogą pojawić się także funkcje klasy (class methods) oraz funkcje statyczne (static methods), opiszemy je w dalszej części.

Można łatwo zademonstrować, że adres przekazywany za pomocą nazwy pierwszego argumentu (zwyczajowo **self**) to adres obiektu danej klasy:

```
class Klasa:
    def __init__(self):
        print("init", id(self)) # na przykład 2618069448160

k1 = Klasa()
print("obiekt Klasa:", id(k1)) # ten sam adres 2618069448160
```

Do istniejącego obiektu można dodawać składowe dynamicznie, jednak taka modyfikacja dotyczy tylko tego konkretnego obiektu, a nie klasy (czyli przepisu na obiekty):

```
k1.x = 1.2
k1.tekst = 'składowa'
print(k1.x, k1.tekst) # 1.2 składowa - tylko w tym obiekcie k1
```

Atrybuty obiektu (ale również klasy, jak opiszemy poniżej) można dodawać także za pomocą wbudowanej funkcji: `setattr(object, attribute, value)`, pierwszy argument to obiekt (dla atrybutu obiektu) lub nazwa klasy (dla atrybutu klasy), drugi to nazwa (w postaci str, czyli w cudzysłowie), a trzeci to nadawana wartość. Przykładowo, dla powyższej klasy Człowiek:

```
c1 = Czlowiek('Jan', 'Kowalski')
setattr(c1, 'wiek', 34)
print(c1.imie, c1.nazwisko, c1.wiek) # Jan Kowalski 34
setattr(c1, 'pesel', 123456789)
print(c1.pesel) # 123456789 - tylko w obiekcie c1
c2 = Czlowiek('Adam', 'Nowak')
print(c2.pesel) # błąd jak niżej
```

```
Traceback (most recent call last):
  File "test.py", line 71, in <module>
    print(c2.pesel)
AttributeError: 'Czlowiek' object has no attribute 'pesel'
```

Czy można dynamicznie dodać metodę do obiektu? Można, ale jest to równie albo jeszcze bardziej niezalecane, jak dynamiczne dodawanie atrybutów. Ciekawy przegląd możliwości: <https://stackoverflow.com/questions/972/adding-a-method-to-an-existing-object-instance> Python w żaden sposób nie ogranicza przypisania czegoś, co np. całkowicie zmieni typ atrybutu wcześniej ustawionego w `__init__()`:

```
setattr(c1, 'imie', [1,2,3,4]) # bez sensu, ale można
c1.wiek = True # jak wyżej
print(c1.imie, c1.nazwisko, c1.wiek) # [1, 2, 3, 4] Kowalski True
```

Uzupełniając, za pomocą `hasattr(object, attribute)` można sprawdzić czy dany obiekt (klasa) posiada atrybut lub metodę, kontynuując powyższy przykład mamy:

```
print(hasattr(c1, 'imie')) # True
print(hasattr(c1, 'pesel')) # True
print(hasattr(c2, 'pesel')) # False - bo ten obiekt tego atrybutu nie ma, patrz wyżej
print(hasattr(Czlowiek, 'nazwisko')) # False - bo 'nazwisko' to atrybut obiektu, a nie klasy
print(hasattr(Czlowiek, '__init__')) # True
```

Za pomocą `delattr(object, attribute)` można usunąć atrybut z danego obiektu albo klasy, jak również metodę. Dodajmy np. do powyższej klasy Człowiek taką metodę:

```
def funkcja(self):
    print(self.imie)
```

Wtedy:

```
delattr(c1, 'imie')
print(hasattr(c1, 'imie')) # False
print(hasattr(Czlowiek, 'funkcja')) # True
delattr(Czlowiek, 'funkcja')
print(hasattr(Czlowiek, 'funkcja')) # False
```

Ostatnia z funkcji `getattr(object, attribute, default)` ma dość oczywiste przeznaczenie, zwraca wartość atrybutu dla danego obiektu, albo jeśli jest to metoda, to można ją „uchwycić” pod jakąś nazwą i potem wywołać (z wymagany argumentem, co najmniej jednym – pierwszym, odpowiadającym „self” – w postaci obiektu lub nazwy klasy i resztą formalnych parametrów danej metody). Jeśli obiekt nie ma danego atrybutu, to zgłoszony zostaje wyjątek (np. `AttributeError: type object 'Czlowiek' has no attribute 'funkcja'`), o ile nie zdefiniowano trzeciego argumentu (*default*), przykładowo wpisując `None`.

Klasa może mieć składowe przynależne do każdego obiektu (atrybut obiektu, *instance attribute*) lub składowe wspólne dla wszystkich instancji (obektów) klasy (atrybut klasy, *class attribute*) – te pola są pisane w ciele klasy wraz z wartościami je inicjalizującymi. Np. atrybut narodowość

```
class Czlowiek:
    '''Klasa Czlowiek w celu edukacyjnym'''
    narodowosc = "polska" # to jest przykład atrybutu klasy (class attribute)
    def __init__(self, imie, nazwisko, wiek=20):
        self.imie = imie # to i kolejne jest przykład atrybutu obiektu (instance attribute)
        self.nazwisko = nazwisko
        self.wiek = wiek
```

Do atrybutu klasy można odnieść się zarówno przez nazwę klasy (`Czlowiek.narodowosc`) jak i przez utworzony obiekt tej klasy (`c1.narodowosc`). W omawianej klasie `Czlowiek`, dodaliśmy też komentarz, więc możemy:

```
c1 = Czlowiek("Jan", "Kowalski")
print(c1.__doc__)
print(c1.imie, c1.nazwisko, c1.wiek, c1.narodowosc)
```

Podobnie, klasa może posiadać tzw. metody klasy (class methods), które można wywołać za pomocą nazwy klasy, bądź obiektu. Taki rodzaj metod przyjmuje jako pierwszy argument referencję do klasy (a nie konkretnego obiektu) i zwyczajowo ten pierwszy argument nazywany jest `cls`. Dodatkowo metodę klasy poprzedzić należy dekoratorem `@classmethod`, metoda ta nie ma dostępu do atrybutów instancji (próba ich użycia w definicji kończy się zgłoszeniem wyjątku `AttributeError`), tylko do atrybutów klasy. Dodajmy taką funkcję do klasy `Czlowiek`:

```
@classmethod
def kraj(cls):
    print(cls.narodowosc)
```

Metodę taką można wywołać:

```
c1.kraj()
Czlowiek.kraj()
```

Zwróćmy uwagę, że gdyby nie dekorator `@classmethod` nasza funkcja byłaby zwykłą metodą składową dla instancji klasy (instance method), bo użyta nazwa (tutaj `cls`) nie ma znaczenia. Zatem wywołanie `Czlowiek.kraj()` byłoby błędem, technicznie należałoby przekazać jako argument jakiś konkretny obiekt, np. `Czlowiek.kraj(c1)`. Więcej zastosowań, również na temat metod statycznych, poznamy w rozdziale o metaklasach.

Krótki przegląd rodzajów funkcji w klasie ↗ https://docs.quantifiedcode.com/python-anti-patterns/correctness/method_has_no_argument.html

Składowe klasy, zarówno składowe instancji jak i klasy, są całkowicie dostępne, można zmieniać ich zawartość. Podejście w Pythonie jest takie, że nie klasyfikujemy zmiennych jako „prywatne” czy „publiczne”. Są to raczej mechanizmy wskazujące np. na zmienne lub metody, które nie są częścią API (czytaj: nie masz ich używać, albo robisz to na własne ryzyko) – i są to nazwy poprzedzone pojedynczym podkreśleniem, np. `_a`. Nazwy widzialne w wewnętrznym zakresie klasy (składowe i metody) są poprzedzone podwójnym podkreśleniem, np. `__secret` (do ukrywania jest jeszcze dekorator `@property`). Klasy Pythona oraz ich instancje mają swoje własne, odrębne przestrzenie nazw reprezentowane przez predefiniowane atrybuty `Klasa.__dict__` oraz `instance_of_Klasa.__dict__`. Jeśli odnosimy się do atrybutu Pythona z instancji klasy, najpierw sprawdza ona swoją przestrzeń nazw instancji. Jeśli znajdzie atrybut, zwraca powiązaną wartość. Jeśli nie, następnie szuka w przestrzeni nazw klasy i zwraca atrybut (jeśli jest obecny, w przeciwnym razie zgłasza błąd). Na przykład:

```
>>> class Test:
...     war = 1
...     def __init__(self, s):
...         self.war = s
...
>>> t = Test("abc")
>>> t.war
'abc'
>>> Test.war
1
```

Warto przypomnieć, że stosując nazwy zaczynające się od podwójnego podkreślenia sprawiamy, że dostęp do składowych, czy to klasy czy instancji, staje się nieco utrudniony:

```
>>> class Test:
...     __war = 1
...     def __init__(self, s):
...         self.__war = s
...
>>> t = Test("abc")
>>> t.__war
Traceback (most recent call last):
  File "<pyshell#51>", line 1, in <module>
    t.__war
AttributeError: 'Test' object has no attribute '__war'
>>> Test.__war
Traceback (most recent call last):
  File "<pyshell#52>", line 1, in <module>
    Test.__war
AttributeError: type object 'Test' has no attribute '__war'
```

Wynika to z faktu zamiany nazwy według konwencji zapisanej jako „prywatna”, na nazwę z dodanym przedrostkiem (nazwa w obu przestrzeniach nazw, instancji i klasy, to `_Test__war`):

```
>>> t.__dict__
{'_Test_war': 'abc'}
>>> Test.__dict__
mappingproxy({'__module__': '__main__', '_Test_war': 1, '__init__': <function Test.
__init__ at 0x000002457CAA8040>, '__dict__': <attribute '__dict__' of 'Test' objects
>, '__weakref__': <attribute '__weakref__' of 'Test' objects>, '__doc__': None})
```

Do słownika klasy (`__dict__`) można się też dostać za pomocą obiektu klasy, ale trzeba wykorzystać po drodze atrybut `__class__`:

```
>>> t.__class__.__dict__
mappingproxy({'__module__': '__main__', '_Test_war': 1, '__init__': <function Test.
__init__ at 0x000002457CAA9120>, '__dict__': <attribute '__dict__' of 'Test' objects
>, '__weakref__': <attribute '__weakref__' of 'Test' objects>, '__doc__': None})
```

Wewnątrz metody instancji, odwołanie do atrybutów klasy uzyskuje się poprzez `self.__class__`.

Dopiszmy kolejną metodę składową klasy:

```
def print(self):
    print(self.imie, self.nazwisko, self.wiek, self.narodowosc)
```

Zamiast metody `print` w klasach Pythona raczej zdefiniowalibyśmy kolejną specjalną metodę, zwracającą opis klasy, o nazwie `__str__()`, na przykład:

```
def __str__(self):
    return '{self.imie}, {self.nazwisko}, {self.wiek}, {self.narodowosc}'.format(self=self)
```

Wtedy zamiast np. `c1.print()` użyjemy: `print(c1)`. Być może zastanawia nas, co jest czym w zapisie „self=self”. Otóż `self` po prawej stronie jest argumentem `__str__` czyli reprezentuje instancję konkretnego obiektu. Natomiast `self` po lewej stronie jest argumentem `format`, za pomocą którego zapisana jest treść w formatowanym łańcuchu znakowym, czyli `self.imie` itd. Nie ma żadnego obowiązku używania nazwy `self`, ale taka jest praktyka. Można by więc zapisać:

```
def __str__(self):
    return '{param.imie}, {param.nazwisko}, {param.wiek}, {param.narodowosc}'.format(param=self)
```

Definiowanie funkcji `__str__()` ma na celu czytelne zaprezentowanie informacji o obiekcie klasy. Gdy chodzi o precyzyjne informacje, bardziej nadające się dla programistów czyli w celach do debugowania, używana jest funkcja `__repr__()`. Przykładowo:

```
print(c1.__repr__())
```

wywoła domyślną postać informacji: `<__main__.Czlowiek object at 0x000001957AEADCF0>`
Jeśli nie została zdefiniowana metoda `__str__()`, to jej wywołanie będzie tożsame z wywołaniem `__repr__()`. Warto w tym miejscu zrobić dygresję, pokazującą różnice na podstawie wbudowanych funkcji `str()` i `repr()`, obie zwracają typ `str`, ale różnią się sposobem prezentacji informacji:

```
>>> import datetime
>>> dzis = datetime.datetime.now()
>>> print(str(dzis)) # czytelna informacja
2022-11-12 18:40:20.982564
>>> print(repr(dzis)) # oficjalny format obiektu datetime
datetime.datetime(2022, 11, 12, 18, 40, 20, 982564)
```

W jaki sposób zdefiniować `__repr__()` dla naszej klasy `Czlowiek`? Zazwyczaj definicja jest taka:

```
def __repr__(self):
    return f'Czlowiek(imie={self.imie}, nazwisko={self.nazwisko}, wiek={self.wiek})'
    # jeśli chcemy atrybut klasy, to można: Czlowiek.narodowosc
```

co daje: `Czlowiek(imie=Jan, nazwisko=Kowalski, wiek=20)`.

W Pythonie obiekty usuwane są przez odśmiecacz (garbage collector), gdy do danej instancji nie ma już żadnego dowiązania. Destruktory nie są zatem tak potrzebne jak w innych językach, ale można je zdefiniować jako `__del__()` wewnątrz klasy, np.

```
def __del__(self):
    print('Koniec')
```

Zasadniczo rekomenduje się, żeby nie pisać kodu tak, żeby zawartość `__del__()` była istotna.

Wracając do mechanizmu tworzenia obiektu, metoda kontrolująca `__init__` jest metodą związaną z inicjalizacją. Metoda kontrolująca tworzenie nowego obiektu to `__new__()`, jest to metoda statyczna. Jest wołana najpierw (gdy „wołana jest” klasa) i zwraca tworzony obiekt, po czym wołane jest `__init__`. Jeśli `__new__` nie zwróciłoby obiektu, `__init__` by nie było wywołane. Nawet jeśli nie napisaliśmy `__new__` to jest ono dziedziczone ze specjalnej klasy o nazwie `object` – jest to klasa bazowa w hierarchii klas. Argumenty `__new__` i sam zapis wyglądają tak:

```
class Foo(object):
    def __new__(cls, *args, **kwargs):
        print(cls)
        print(args)
        print(kwargs)

obj1 = Foo(1,2,3, key=1, key2=2)
```

Wykonanie daje:

```
<class '__main__.Foo'>
(1, 2, 3)
{'key': 1, 'key2': 2}
```

Zobaczmy kompletny kod, dzięki któremu zbudowana jest instancja `Foo` i uruchomiony również `__init__`. Przy okazji, za pomocą składowego `__new__` można zbudować też coś innego:

```
import datetime

class Bar:
    pass

class Foo(object):
```



```
def __new__(cls, *args, **kwargs):
    print("I am in __new__", cls)
    nowy_obiekt = object.__new__(cls)
    nowy_obiekt.data = datetime.datetime.now()
    return nowy_obiekt

def __init__(self, a, b, c):
    print("I am in __init__", self.data)
    self.a, self.b, self.c = a, b, c
    print(self.__dict__)

obj1 = Foo(1, 2, 3)
obj2 = Foo.__new__(Bar)
```

Przykładowy wynik:

```
I am in __new__ <class '__main__.Foo'>
I am in __init__ 2022-11-12 23:35:45.592783
{'data': datetime.datetime(2022, 11, 12, 23, 35, 45, 592783), 'a': 1, 'b': 2, 'c': 3}
I am in __new__ <class '__main__.Bar'>
```

Python w naturalny sposób jest językiem obiektowym, to znaczy większość elementów, z którymi mamy do czynienia, można traktować jako obiekty – to znaczy, że mają one nie tylko jakąś wartość czy stan (poprzez stan rozumiemy również zespół wszystkich wartości składników np. obiektu klasy), ale również pewien zespół metod, którymi można działać na danym obiekcie. Najbardziej widoczne jest to w przypadku prostych typów wbudowanych, np. typu `int`. W wielu językach obiekt typu `int` reprezentuje wyłącznie wartość, nie można na takim obiekcie wywołać „metody składowej”. W Pythonie przeciwnie, na prostym obiekcie, albo nawet stałej, można wywołać szereg metod. Większość z nich to metody „magiczne”, albo „specjalne” (ang. dunder methods, od skróconego **d**ouble **u**nderscore **m**ethods), z ich listą można się zapoznać poprzez funkcję `dir`:

```
>>> dir(int) # to samo dla dir(2) itp.
['_abs_', '_add_', '_and_', '_bool_', '_ceil_', '_class_', '_delattr_',
'_dir_', '_divmod_', '_doc_', '_eq_', '_float_', '_floor_', '_floordi',
'_v_', '_format_', '_ge_', '_getattribute_', '_getnewargs_', '_gt_', '_has',
'_h_', '_index_', '_init_', '_init_subclass_', '_int_', '_invert_', '_le_',
'_lshift_', '_lt_', '_mod_', '_mul_', '_ne_', '_neg_', '_new_', '_or_',
'_pos_', '_pow_', '_radd_', '_rand_', '_rdivmod_', '_reduce_', '_re',
'_reduce_ex_', '_repr_', '_rfloordiv_', '_rlshift_', '_rmod_', '_rmul_', '_r',
'_ror_', '_round_', '_rpow_', '_rrshift_', '_rshift_', '_rsub_', '_rtrue',
'_div_', '_rxor_', '_setattr_', '_sizeof_', '_str_', '_sub_', '_subclassho',
'_ok_', '_truediv_', '_trunc_', '_xor_', '_as_integer_ratio_', '_bit_length_', '_con',
'_jugate_', '_denominator_', '_from_bytes_', '_imag_', '_numerator_', '_real_', '_to_bytes_']
```

Jeśli zachowanie wbudowanej funkcji lub operatora nie jest zdefiniowane w klasie za pomocą specjalnej metody, otrzymamy `TypeError` albo `AttributeError`. Na przykład, dla typu `str` mamy:

```
>>> dir(str)
['_add_', '_class_', '_contains_', '_delattr_', '_dir_', '_doc_', '_eq_',
'_format_', '_ge_', '_getattribute_', '_getitem_', '_getnewargs_', '_gt_',
'_hash_', '_init_', '_init_subclass_', '_iter_', '_le_', '_len_', '_lt_',
'_mod_', '_mul_', '_ne_', '_new_', '_reduce_', '_reduce_ex_', '_repr_', '_r',
'_rmod_', '_rmul_', '_setattr_', '_sizeof_', '_str_', '_subclasshook_', '_capita',
'_lize_', '_casefold_', '_center_', '_count_', '_encode_', '_endswith_', '_expandtabs_', '_find_', '_forma',
'_t_', '_format_map_', '_index_', '_isalnum_', '_isalpha_', '_isascii_', '_isdecimal_', '_isdigit_', '_isi',
'_dentifier_', '_islower_', '_isnumeric_', '_isprintable_', '_isspace_', '_istitle_', '_isupper_', '_joi',
'_n_', '_ljust_', '_lower_', '_lstrip_', '_maketrans_', '_partition_', '_removeprefix_', '_removesuffix_',
'_replace_', '_rfind_', '_rindex_', '_rjust_', '_rpartition_', '_rsplit_', '_rstrip_', '_split_', '_spl',
'_itlines_', '_startswith_', '_strip_', '_swapcase_', '_title_', '_translate_', '_upper_', '_zfill_']
```

Dla typu `str` nie ma operatora potęgowania (atrybutu `__pow__`), zatem:

```
>>> 'abc'.__pow__(2)
Traceback (most recent call last):
  File "<pyshell#74>", line 1, in <module>
    'abc'.__pow__(2)
AttributeError: 'str' object has no attribute '__pow__'. Did you mean: '__doc__'?
```

Z kolei operacja mnożenia (`__mul__`) może być wykonana na typie `str`, ale drugim argumentem musi być `int`. W przeciwnym razie:

```
>>> 'abc'.__mul__('abc')
Traceback (most recent call last):
  File "<pyshell#72>", line 1, in <module>
    'abc'.__mul__('abc')
TypeError: 'str' object cannot be interpreted as an integer
```

W przypadku klasy `object` oraz przykładowej klasy `Klasa` napisanej przez użytkownika, zobaczymy (proszę przyjrzeć się różnicom):

```
>>> dir(object)
['__class__', '__delattr__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__',
 '__getattr__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__le__',
 '__lt__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__setattr__
__', '__sizeof__', '__str__', '__subclasshook__']
>>> class Klasa(object):
    pass

>>> dir(Klasa)
['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__format__
', '__ge__', '__getattr__', '__gt__', '__hash__', '__init__', '__init_subclass__
', '__le__', '__lt__', '__module__', '__ne__', '__new__', '__reduce__', '__reduce_e
x__', '__repr__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', '__wea
kref__']
```

Jak widać (na przykładzie typu wbudowanego), bardzo wiele metod reprezentuje działania operatorów, np. `+` realizowane jest przez `__add__`. Zasadniczo funkcji tych nie wywołuje się wprost, choć jest to możliwe, np.:

```
>>> liczba = 2 # tworzymy obiekt liczba
>>> liczba + 3 # prosta operacja
5
>>> 3 + liczba # prosta operacja
5
>>> liczba.__add__(3) # tu jawnie wywołane __add__
5
>>> liczba.__add__(liczba) # tu jawnie wywołane __add__
4
```

Jednak operacje jak niżej nie działają:

```
>>> 3.__add__(2)
SyntaxError: invalid decimal literal
>>> 3.__add__(liczba)
SyntaxError: invalid decimal literal
```

Wynika to z faktu, że parser języka Python działa w prosty sposób, interpretując napotkane obiekty od lewej do prawej. W tym przypadku spotkawszy kropkę po liczbie 3 całość zostaje zinterpretowana jako liczba zmiennoprzecinkowa 3. (float), stąd cała reszta wyrażenia jest niepoprawna. Co ciekawe, we wcześniejszych wersjach Pythona zgłaszany wyjątek miał komentarz `SyntaxError: invalid syntax`. Oto kilka możliwych rozwiązań:

```
>>> (3).__add__(2) # liczba 3 w nawiasie
5
>>> 3 .__add__(2) # spacja po liczbie 3
5
>>> 3.__add__(2) # . po 3. ale wtedy dodajemy 2 do 3. typu float
5.0
```

Bezpośrednie definiowanie „magicznych metod”, celem osiągnięcia funkcjonalności typów wbudowanych, jest stosowane dla typów użytkownika. W innych językach technika znana jest jako przeładowywanie (przeciążanie) operatorów tak, żeby możliwe było ich stosowanie z nowo zdefiniowanym typem. W Pythonie można zdefiniować funkcję (operator) dla typu użytkownika, ale nie istnieje coś takiego jak przeciążenie metod realizowane tak, że dla metody o tej samej nazwie możemy napisać dwie wersje, z taką samą liczbą argumentów, ale o różnej zawartości. Powód jest oczywisty, dynamiczne określanie typów, których nie można „zdefiniować” wcześniej (używanie anotacji w niczym tu nie pomoże). Jeśli zdefiniujemy drugą metodę w klasie, ta druga definicja nadpisze pierwszą i pozostanie tylko druga. Trochę inna sytuacja jest w przypadku odmian metody o różnej liczbie argumentów, bo wtedy można użyć techniki z modułu `multipledispatch` (opiszemy to później).

Generalnie, jeśli istnieje wbudowana funkcja `func()` i odpowiadająca jej specjalna metoda to `__func__()`, to Python interpretuje wywołanie tej funkcji jako `obj.__func__()`, gdzie `obj` to instancja (obiekt). Przykładowo, jeśli wołamy `len()` na jakimś obiekcie, Python wykonuje `obj.__len__()`. Jeśli używamy operatora `[]` na wielkości iterowalnej aby dostać jej wartość pod wskazanym indeksem, Python wykonuje `itr.__getitem__(index)`, gdzie `itr` jest obiektem iterowalnym, zaś `index` to indeks, pod którym chcemy odczytać wartość. W przypadku operatorów, jeśli mamy operator `opr` i odpowiadająca mu specjalna metoda to `__opr__()`, Python wykona `obj1 <opr> obj2` jako `obj1.__opr__(obj2)`. Dlatego definiując metody specjalne we własnej klasie, nadpisujemy zachowanie powiązanej z nimi funkcji lub operatora, ponieważ w tle Python wywołuje metodę użytkownika.

Wiele specjalnych metod zdefiniowanych w Pythonie (polecam przejrzeć <https://docs.python.org/3/reference/datamodel.html>) może służyć do zmiany zachowania funkcji, takich jak `len`, `abs`, `hash`, `divmod` itd. Aby to zrobić, wystarczy zdefiniować odpowiednią metodę specjalną w klasie użytkownika. Kilka przykładów:

```
class Zamowienie:
    def __init__(self, koszyk, klient):
        self.koszyk = list(koszyk)
        self.klient = klient

    def __len__(self):
        return len(self.koszyk)

zamowienie = Zamowienie(['rower', 'pilka', 'arbuz'], 'Uczen Pythona')
print(len(zamowienie)) # 3
```

Python oczekuje, że nasz `__len__` zwróci typ `int`, gdybyśmy zdefiniowali inaczej, dostaniemy błąd `TypeError`. Inny przykład, dodajmy rzutowanie (funkcję specjalną) `bool`, taką, że jeśli nasz koszyk jest pusty, to zwróci `False`, a gdy niepusty zwróci `True`.

```
def __bool__(self):
    return len(self.koszyk) > 0
```

Wtedy:

```
print(bool(zamowienie)) # True
```

Dodajmy operator indeksowania `[]`:

```
def __getitem__(self, key):
    return self.koszyk[key]
```

Argument `key` to numer indeksu, ale nie tylko. Wszędzie tam, gdzie działamy na kolekcjach, w których używa się `[]`, my też możemy – użyć string (na określenie `key` w typie `dict`), albo obiekt do selekcji (slicing <https://docs.python.org/3/library/functions.html#slice>), czyli np.

```
print(zamowienie[0], zamowienie[1::-1]) # rower ['pilka', 'rower']
```

Zaimplementujmy możliwość dodawania nowych pozycji do naszego koszyka w klasie `Zamowienie` za pomocą operatora `+`. Zgodnie z zalecaną praktyką operator zwróci **nową** instancję obiektu `Zamowienie`, która zawiera listę z dodanym elementem:

```
def __add__(self, other):
    nowy_koszyk = self.koszyk.copy()
    nowy_koszyk.append(other)
    return Zamowienie(nowy_koszyk, self.klient)
```

Wtedy:

```
print((zamowienie + 'but').koszyk) # ['rower', 'pilka', 'arbuz', 'but']
zamowienie = zamowienie + 'drugi but' # konieczne jest przypisanie wyniku
print(zamowienie.koszyk) # ['rower', 'pilka', 'arbuz', 'drugi but']
```

Kolejny oczekiwany operator `+=` jest skróconą realizacją wyrażenia `obj1 = obj1 + obj2`. Odpowiada mu specjalna metoda `__iadd__()`, która wykonuje zmianę bezpośrednio na obiekcie `self`, zwracając obiekt `self`. Jeśli pominęlibyśmy instrukcję `return`, to efekt działania `+=`, którego pierwszą częścią jest dodanie elementu, a drugą przypisanie, będzie taki, że przypisane (zwrócone) zostanie `None`, a wartość modyfikowanego obiektu utraciona.

```
def __iadd__(self, other):
    self.koszyk.append(other)
    return self # ważne żeby nie zapomnieć
```

Demonstracja działania:

```
zamowienie += 'pierwszy but'
print(zamowienie.koszyk) # ['rower', 'pilka', 'arbuz', 'drugi but', 'pierwszy but']
```

W tym miejscu warto przyjrzeć się przykładowi bliżej reprezentującemu działania numeryczne. Zdefiniujmy prostą klasę `Liczba`, która będzie posiadać jedną składową. Dodatkowo od razu zdefiniujemy `__add__()` oraz `__iadd__()`:

```
class Liczba:

    def __init__(self, num):
        self.num = num

    def __str__(self):
        return str(self.num)

    def __add__(self, other):
        return Liczba(self.num + other.num)

    def __iadd__(self, other):
        self.num += other.num
        return self
```

Przykład działania:

```
k1 = Liczba(3)
k2 = Liczba(4)
k3 = k1 + k2
k3 += k1
print(k1,k2,k3) # 3 4 10
```

Warto zwrócić uwagę na istnienie atrybutu `__class__`, który jest referencją do klasy danego obiektu. Można go użyć w ten sposób, że np. definicję metody `__add__`, która zwraca nowy obiekt klasy, zapiszemy:

```
def __add__(self, other):
    return self.__class__(self.num + other.num)
```

czyli bez użycia nazwy klasy `Liczba`. Może to być korzystne w sytuacji gdy zechcemy zmienić nazwę klasy `Liczba` na jakąś inną, wtedy tej definicji nie trzeba w ogóle aktualizować. Typ klasy można odpytać zarówno poprzez funkcję `type()` – zalecane, jak i atrybut `__class__`, oraz dodatkowo nazwę klasy poprzez atrybut `__name__`:

```
print(type(k1), k1.__class__) # <class '__main__.Liczba'> <class '__main__.Liczba'>
print(type(k1).__name__, k1.__class__.__name__) # Liczba Liczba
```

Dalej, nasuwa się pytanie, czy możliwe jest wykonanie operacji, gdy drugim argumentem będzie na przykład `int` lub `float`, czyli:

```
k2 = k1 + 5
k3 += 6
```

Dla oryginalnej definicji jest to niemożliwe, oczywiście dostaniemy błąd `AttributeError: 'int' object has no attribute 'num'`. Rozwiązaniem jest zbadanie typu argumentu wewnątrz definicji metody i odpowiednie jej zapisanie. Jeśli zakładamy tylko możliwość argumentu typu `Liczba` lub takiego, który będzie mógł wykonać operację z atrybutem instancji `num`, to definicja może wyglądać następująco:

```
def __add__(self, other):
    return Liczba(self.num + other.num) if isinstance(other, Liczba) else Liczba(self.num + other)

def __iadd__(self, other):
    self.num += other.num if isinstance(other, Liczba) else other
    return self
```

Wtedy działa również mieszkanca Liczba oraz typów int, float, a nawet complex:

```
k1 = Liczba(3)
k2 = Liczba(4)
k3 = k1 + k2 + 5
k2 += 6 + 1.23 + 1+0j
print(k1,k2,k3) # 3 (12.23+0j) 12
```

A jednak to nie koniec, ponieważ przypadek gdy lewy argument to na przykład int, a prawy to Liczba, nie działa:

```
k3 = 5 + k1
```

dostajemy błąd `TypeError: unsupported operand type(s) for +: 'int' and 'Liczba'`. Aby obsłużyć sytuację typu `x + obj`, należy dodatkowo zdefiniować specjalną metodę typu „reverse add” `__radd__()`. Co ciekawe, definicja pozostaje praktycznie identyczna jak dla `__add__()`, więc można ją zrealizować tak:

```
def __radd__(self, other):
    return self.__add__(other)
```

Podobne definicje należy napisać dla odejmowania (`__sub__()`, `__rsub__()`, `__isub__()`), i mnożenia (`__mul__()`, `__rmul__()`, `__imul__()`). W przypadku dzielenia sytuacja się nieco „komplikuje”, gdyż mamy dzielenie „prawdziwe” (operator jeden `/`) i dzielenie jak dla typu całkowitego (operator dwa `//`). Dla prawdziwego dzielenia trzeba zdefiniować metody `__truediv__()`, `__rtruediv__()` oraz `__itruediv__()`. Dla dzielenia całkowitego mamy metody `__floordiv__()`, `__rfloordiv__()`, `__ifloordiv__()`. Jeśli chcemy całkowicie wyposażać naszą klasę w możliwe numeryczne operacje, należałoby napisać także (podaję tylko nazwę podstawową, domyślnie zakładając również, że są wersje z „r” oraz „i” z przodu w nazwie):

`__matmul__()` – funkcja, która może być zaskoczeniem, ponieważ reprezentuje operator `@`, który pojawił się w Pythonie 3.5 i służy do zapisu mnożenia macierzy (sam znaczek wymyślono ze słowa **m**ATrices). O ile sami nie napiszemy takiej klasy, operator ten możliwy jest do użycia tylko z niektórymi bibliotekami, np. NumPy. Póki co, w naszym kodzie, pojawia się on w innych okolicznościach, w technice dekoratorów.

`__mod__()` czyli operator modulo `%` reszty z dzielenia

`__divmod__()` czyli funkcja `divmod()`, dla typu int zwraca parę będącą `(a // b, a % b)`, patrz <https://docs.python.org/3/library/functions.html#divmod>

`__pow__()` funkcja `pow()` lub operator `**`, ale należy w implementacji uwzględnić też opcjonalny trzeci argument <https://docs.python.org/3/library/functions.html#pow>

`__lshift__()` oraz `__rshift__()` czyli operatory przesunięcia bitowego `<<` i `>>`

`__and__()`, `__xor__()`, `__or__()` czyli operatory `&`, `^`, `|`

Jednoargumentowe operacje arytmetyczne (realizowane za pomocą operatorów lub funkcji `-`, `+`, `abs()` wartość bezwzględna, `~`) można zrealizować implementując `__neg__(self)`, `__pos__(self)`, `__abs__(self)`, `__invert__(self)`. Operacje rzutowania `complex()`, `int()`, `float()` realizują metody `__complex__()`, `__int__()` oraz `__float__()`.

`__index__()` ta metoda jest wywoływana na obiekcie, aby uzyskać powiązaną z nim wartość całkowitą. Zwrócona liczba całkowita jest używana podczas operacji „slicingu” lub jako podstawa konwersji we wbudowanych funkcjach `bin()`, `hex()` i `oct()`. W naszym przykładzie napisalibyśmy w definicji `return self.num` (uwaga, funkcja musi zwrócić typ int).

Pozostają jeszcze funkcje `__round__()`, `__trunc__()`, `__floor__()` i `__ceil__()`.

Funkcje określające relacje pomiędzy obiektami są ważne nie tylko dla wielkości numerycznych, stąd wyposażenie klasy w następujące metody: `x < y` wywołuje `x.__lt__(y)`, `x <= y` wywołuje `x.__le__(y)`, `x == y` wywołuje `x.__eq__(y)`, `x != y` wywołuje `x.__ne__(y)`, `x > y` wywołuje `x.__gt__(y)`, a `x >= y` wywołuje `x.__ge__(y)`. Jako wynik, zwracane są wartości `False` i `True`, jednak metody te mogą zwracać dowolną wartość, a w takim przypadku Python wywoła funkcję `bool()` na wartości aby określić, czy wynik jest prawdziwy, czy fałszywy. Operatory (metody) mogą zwrócić singleton `NotImplemented` w przypadku braku implementacji operacji dla danej pary argumentów.

PROTOKÓŁ DESKRYPTORÓW

Wydawać się może, iż dostęp do atrybutów obiektu to po prostu składnia typu `obiekt.atrybut`. Okazuje się, że mechanika języka jest tu o wiele ciekawsza, stwarzając dodatkowo możliwości w kreowaniu obiektowego i bardziej zautomatyzowanego kodu. Python pozwala na zdefiniowanie zestawu zachowań, nazywanego protokołem deskryptorów, za pomocą poniższych metod:

```
__get__(self, obj, type=None) -> object
__set__(self, obj, value) -> None
__delete__(self, obj) -> None
__set_name__(self, owner, name) # ta metoda od Pythona 3.6
```

Metody te uruchamiane są dla obiektów będących składowymi (atrybutami) instancji innych klas, podczas operacji odczytu lub modyfikacji danej składowej. A konkretnie, odwołanie się do `obj.attribute` uruchamia `__get__`, przypisanie `obj.attribute = new_value` woła `__set__`, `del obj.attribute` woła metodę `__delete__`, zaś metoda `__set_name__` umożliwia deskryptorowi (obiektowi klasy z zaimplementowanym protokołem jak wyżej) poznać nazwę atrybutu do którego został przypisany. Jest automatycznie wywoływana, gdy obiekt, który implementuje protokół deskryptora, zostaje przypisany do atrybutu. Przykład:

```
class Deskryptor:
    def __init__(self):
        print('__init__')

    def __set_name__(self, owner, name):
        print(f'__set_name__(owner={owner}, name={name})')
        self.name = name

    def __get__(self, instance, owner=None):
        print(f'__get__(instance={instance}, owner={owner})')
        return instance.__dict__.get(self.name)

    def __set__(self, instance, value):
        print(f'__set__(instance={instance}, value={value})')
        instance.__dict__[self.name] = value

    def __delete__(self, instance):
        print(f'__delete__(instance={instance})')
        del instance.__dict__[self.name]

class JakasKlasa:
    a = Deskryptor()
```

Zaraz po uruchomieniu:

```
__init__
__set_name__(owner=<class '__main__.JakasKlasa'>, name=a)
```

Teraz utwórzmy obiekt (instancję) oraz wykonajmy operacje, które spowodują wywołanie metod protokołu deskryptora (efekt podany w komentarzu pod każdą linią):

```
obiekt = JakasKlasa()
obiekt.a = 123
# __set__(instance=<__main__.JakasKlasa object at 0x0000027EE5D977F0>, value=123)
print(objekt.a)
# __get__(instance=<__main__.JakasKlasa object at 0x0000027EE5D977F0>, owner=<class '__main__.JakasKlasa'>)
# 123
del obiekt.a
# __delete__(instance=<__main__.JakasKlasa object at 0x0000027EE5D977F0>)
print(objekt.a)
# __get__(instance=<__main__.JakasKlasa object at 0x0000027EE5D977F0>, owner=<class '__main__.JakasKlasa'>)
# None
```

Pokazaliśmy, że metody protokołu rzeczywiście są wołane przy okazji zwyczajnych operacji czytania, zapisywania lub usuwania składowej klasy. Ten fakt działania metod można praktycznie wykorzystać. Najczęściej demonstruje się, że można w ten sposób „zablokować” próby modyfikowania składowej, czyniąc z niej praktycznie wielkość tylko do odczytu (proszę pamiętać, że dzieje się to w kontekście faktu, że Python nie ma czegoś takiego jak składowe prywatne czy publiczne).

```
class ReadOnly:
    def __init__(self, value):
        self.value = value
    def __set_name__(self, owner, name):
        self.name = name
    def __get__(self, instance, owner=None):
        return self.value
    def __set__(self, instance, value):
        msg = '{} is a read-only attribute'.format(self.name)
        raise AttributeError(msg)

class NKlasa:
    a = ReadOnly('tekst na poczatek')
```

Utwórzmy obiekt NKlasa i spróbujmy wykonać operacje czytania oraz modyfikacji składowej a.

```
obiektR = NKlasa()
print(objektR.a) # tekst na poczatek
obiektR.a = 'proba przypisania' # wywołanie __set__ spowoduje wyjątek
```

```
Traceback (most recent call last):
  File "C:\deskryptor.py", line 56, in <module>
    obiektR.a = 'proba przypisania'
  File "C:\deskryptor.py", line 48, in __set__
    raise AttributeError(msg)
AttributeError: a is a read-only attribute
```

Zgłoszenie wyjątku `AttributeError` w przypadku próby przypisania wartości jest rekomendowanym sposobem na implementację deskryptorów „tylko do odczytu”.

Wiemy już jakie metody składają się na interfejs protokołu deskryptorów, a teraz poznamy ich klasyfikację. Jest ona istotna ze względu na to, że priorytet deskryptorów danych jest wyższy niż deskryptorów nie-danych. Podział jest następujący:

- deskryptor danych (data descriptor) ma zdefiniowaną metodę `__get__` oraz `__set__` lub `__delete__`
- deskryptor nie-danych (non-data descriptor) ma tylko metodę `__get__`

Jeśli odwołamy się do składowej (nazwijmy ją atrybut) obiektu: `obiekt.atrybut`, Python zaczyna poszukiwanie w kolejności: sprawdza czy jest deskryptor danych dla atrybutu, jeśli nie, sprawdza słownik obiektu `__dict__` szukając klucza o nazwie atrybut, jeśli też nie, sprawdza czy jest deskryptor nie-danych atrybutu. Jeśli i to się nie uda, przeszukiwany jest słownik typu danych obiektu pod kątem wystąpienia klucza atrybut. W przypadku klasy potomnej (gdymyśmy rozważali już sytuację dziedziczenia), dalsze poszukiwanie penetrowałoby słownik typu klasy bazowej, również uwzględniając strategię nazywaną MRO (method resolution order), o której później powiemy. Na sam koniec jeśli nic by nie zostało znalezione, zgłoszony zostanie wyjątek `AttributeError`. Ten opis wygląda niezachęcająco, ale zilustrujmy priorytety poszukiwań na prostym przykładzie.

```
class DataDescriptor:
    def __set_name__(self, owner, name):
        self.name = name

    def __get__(self, obj, owner=None):
        default_value = '{} not defined'.format(self.name)
        print('In __get__ of {}'.format(self))
        return obj.__dict__.get(self.name, default_value)

    def __set__(self, obj, value):
        print('In __set__ of {}'.format(self))
        obj.__dict__[self.name] = value

class NonDataDescriptor:
    def __set_name__(self, owner, name):
        self.name = name

    def __get__(self, obj, owner=None):
        default_value = '{} not defined'.format(self.name)
        print('In __get__ of {}'.format(self))
        return obj.__dict__.get(self.name, default_value)

class Przyklad:
    a = DataDescriptor()
    b = NonDataDescriptor()
```

W klasie o nazwie `Przyklad` mamy dwa atrybuty reprezentujące, odpowiednio, klasę `DataDescriptor` (gdyż jest obecna metoda `__set__`) i klasę `NonDataDescriptor` (tylko metoda `__get__`). Tworzymy obiekt klasy `Przyklad`:

```
p1 = Przyklad()

print(p1.__dict__) # {}
print(p1.a)
print(p1.b)
```

W efekcie widzimy, że słownik obiektu `p1` jest pusty {}, a w efekcie odwołania (`print`) `p1.a` i `p1.b` dają, kolejno:

```
In __get__ of <__main__.DataDescriptor object at 0x0000021A19AF7DC0>
a not defined
In __get__ of <__main__.NonDataDescriptor object at 0x0000021A19AF7D90>
b not defined
```

Wpiszmy zatem do słownika wartości

```
p1.__dict__['a'] = 'AAAAA'
p1.__dict__['b'] = 'BBBBB'
```

i powtórzmy:

```
print(p1.__dict__) # {'a': 'AAAAA', 'b': 'BBBBB'}
print(p1.a)
# In __get__ of <__main__.DataDescriptor object at 0x000002CED62E7FD0>
# AAAAA
print(p1.b)
# BBBBB
```

Odniesienie się do składowej a (która jest rodzaju data descriptor) nadal używa metody `__get__`, ponieważ dla „data descriptor” ma ona priorytet przed przeglądaniem słownika obiektu. Dla składowej b (rodzaju non-data descriptor) mamy odniesienie się do słownika, gdyż `__get__` w tym przypadku ma niższy priorytet.

Okazuje się, że protokół deskryptorów jest często używany, ale jego realizacja odbywa się za pomocą klasy (bo jest to klasa, a nie funkcja, choć często się tak potocznie mówi) `property()` <https://docs.python.org/3/library/functions.html#property> najczęściej z użyciem techniki dekoratora `@property`.

Zacznijmy ponownie od prostej klasy:

```
class Punkt:
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

```
punkt = Punkt(4, 11)
print(punkt.x, punkt.y) # 4 11
```

W Pythonie całkowicie naturalne jest, że ekspozycja na składowe instancji odbywa się bez pośrednictwa metod typu „get” czy „set”. A jednak takie podejście ma poważny mankament – co jeśli będziemy chcieli zmienić sposób wyliczenia czy użycia składowych x i y, zakładając, że przecież nie chcemy zmian w interfejsie. Tu z pomocą przychodzi technika zamieniania atrybutów we własności: te drugie reprezentują pośrednią funkcjonalność pomiędzy czystym atrybutem (jak x i y) a metodą, czyli pozwalają na utworzenie metod, które będą zachowywały się tak jak atrybuty. Z ich pomocą będzie można dowolnie zmieniać wyliczenie wartości atrybutów bez jakiejkolwiek zmiany API.

Klasa `property()` jest nietypową klasą w Pythonie, zaprojektowaną tak, żeby działała jak funkcja, z pomocą której można łączyć wskazane metody z atrybutem. Klasa ta zwraca „wyszpazony” atrybut, na rzecz którego wywołano `property`. W ten sposób odwołanie w programie nadal będzie się odbywało bezpośrednio do atrybutu, bez ekspozowania metod, które będą na jego rzecz uruchamiane. Pełna sygnatura `property` wygląda następująco:

```
property(fget=None, fset=None, fdel=None, doc=None)
```

gdzie dwa pierwsze argumenty pełnią rolę metod typu „get” i „set”, można również podać sposób kasowania atrybutu oraz uzupełnić wszystko komentarzem. Gdy odwołamy się do obiekt.atrybut, Python automatycznie wywoła fget(), gdy przypiszemy nową wartość do atrybutu, obiekt.atrybut = wartosc, Python wywoła fset(), w którym wartosc będzie argumentem. Wreszcie, del obiekt.atrybut wywoła fdel(), a podanie doc przyda się jako informacja, otrzymana czy to z pomocą funkcji help() czy wykorzystana i wyświetlana pomocniczo w różnych środowiskach pracy IDE.

Property może być użyte wprost jako klasa („funkcja”) lub jako dekorator. Zobaczmy jeszcze bardziej uproszczony przykład klasy z jednym atrybutem.

```
class Okrag:
    def __init__(self, value):
        self._r = value

    def _get_r(self):
        print("Czytaj promien")
        return self._r

    def _set_r(self, value):
        print("Ustaw promien")
        self._r = value

    def _del_r(self):
        print("Usun promien")
        del self._r

    r = property(
        fget=_get_r,
        fset=_set_r,
        fdel=_del_r,
        doc="Wlasnosci promienia."
    )
```

Zwróćmy uwagę na składową klasy r, która jest typu <class 'property'>. Utwórzmy obiekt i wykonajmy operacje odpowiadające wszystkim podłączonym za pomocą property metodom:

```
ok = Okrag(12.34)

print(ok.r)
# Czytaj promien
# 12.34
ok.r = 10.23
# Ustaw promien
del ok.r
# Usun promien
```

Próba wywołania print(ok.r) po skasowaniu daje:

```
Czytaj promien
Traceback (most recent call last):
  File "C:\deskryptor.py", line 36, in <module>
    print(ok.r)
  File "C:\deskryptor.py", line 7, in _get_r
    return self._r
AttributeError: 'Okrag' object has no attribute '_r'. Did you mean: 'r'?
```

Natomiast help(Okrag) daje:

```

Help on class Okrag in module __main__:

class Okrag(builtins.object)
  Okrag(r)

  Methods defined here:

  __init__(self, r)
    Initialize self.  See help(type(self)) for accurate signature.

  -----
  Data descriptors defined here:

  __dict__
    dictionary for instance variables (if defined)

  __weakref__
    list of weak references to the object (if defined)

  r
    Wlasnosci promienia.

```

Za pomocą `dir(Okrag.r)` możemy podejrzeć:

```

['_class__', '__delattr__', '__delete__', '__dir__', '__doc__', '__eq__', '__format__',
'__ge__', '__get__', '__getattr__', '__gt__', '__hash__', '__init__', '__init_subclass__',
'__isabstractmethod__', '__le__', '__lt__', '__ne__', '__new__', '__reduce__', '__reduce_ex__',
'__repr__', '__set__', '__set_name__', '__setattr__', '__sizeof__', '__str__',
'__subclasshook__', 'deleter', 'fdel', 'fget', 'fset', 'getter', 'setter']

```

czyli, że obiekt klasy property zawiera nie tylko atrybuty bezpośrednio zdefiniowanych przez nas metod `fget`, `fset`, `fdel`, ale również protokół deskryptora: `__get__`, `__set__`, `__delete__` oraz `__set_name__`.

Analiza powyższego mechanizmu „wzbogacania” atrybutu klasy o pewne metody przypomina znany idiom dekoratora. I rzeczywiście, obecnie najczęściej będziemy używać property właśnie jako dekorator. Powyższy kod zamienimy na:

```

class Okrag:
    def __init__(self, value):
        self._r = value

    @property
    def r(self):
        """Wlasnosci promienia."""
        print("Czytaj promien.")
        return self._r

    @r.setter
    def r(self, value):
        print("Ustaw promien")
        self._r = value

    @r.deleter
    def r(self):
        print("Usun promien")
        del self._r

```

Metoda odpowiadająca `fget` lub `__get__` to definicja metody składowej `r`, udekorowanej za pomocą `@property`. Pozostałe dwie, czyli `fset` dostajemy przez dekorator `@r.setter` oraz `fdel`

przez dekorator `@r.deleter`. Wszystko działa identycznie jak poprzednio, a kod jest o wiele bardziej czytelny!

Pomimo atrakcyjnej wizualnie techniki, sam interfejs, czyli sposób zapisu i interakcji ze składową klasy nie ulega zmianie, a tylko ubogaceniu w możliwość wywołania metod, które coś przeliczą. Ogólnie rzecz biorąc, należy unikać przekształcania atrybutów, które nie wymagają dodatkowego przetwarzania we właściwości, bo ich wykonanie stanowi narzut na czas CPU. Oczywiście, jeśli potrzebujemy coś więcej niż sam dostęp do atrybutów, używajmy dekoratora `@property`.

MRO (METHOD RESOLUTION ORDER)

Język Python implementuje strategię rozstrzygania kolejności w jakiej przeszukiwane są klasy bazowe podczas wykonywania metody lub dostępu do atrybutu. Jednym z referencyjnych tekstów na ten temat jest <https://www.python.org/download/releases/2.3/mro/>, sam mechanizm opiera się na algorytmie monotonicznej linearyzacji dla klas nadrzędnych https://en.wikipedia.org/wiki/C3_linearization. Jeśli nie korzystasz intensywnie z wielokrotnego dziedziczenia i nie tworzysz skomplikowanych hierarchii, nie musisz rozumieć algorytmu C3, niemniej zrozumienie reguł linearyzacji jest wskazane.

Oto skrócona wersja strategii wyznaczania kolejności, niech rozważana klasa nazywa się C , lista klas $C_1, \dots, C_N$ to $[C_1, C_2, \dots, C_N]$, czoło listy `head` = C_1 , ogon listy `tail` = $C_2 \dots C_N$. Używa się również notacji $C + (C_1 \ C_2 \ \dots \ C_N) = C \ C_1 \ C_2 \ \dots \ C_N$ na oznaczenie list $[C] + [C_1, C_2, \dots, C_N]$. Linearyzację dla klasy C oznaczmy jako $L[C]$, czasem używa się również oznaczenia $MRO(C)$. Jeśli klasa C dziedziczy tylko z klasy `object`, to wynik jest trywialny, $L[\text{object}] = \text{object}$. Załóżmy, że klasa C dziedziczy wielobazowo z klas $B_1, B_2, \dots, B_N$. Wtedy $L[C(B_1 \dots B_N)]$ jest sumą C oraz połączenia (merge) linearyzacji dla każdej z klas bazowych oraz samej listy klas bazowych: $L[C(B_1 \dots B_N)] = C + \text{merge}(L[B_1] \dots L[B_N], B_1 \dots B_N)$. Następnie stosowana jest procedura: weź czoło pierwszej listy, czyli $L[B_1][0]$. Jeśli to czoło nie występuje w ogonie żadnej z innych list, dodaj je do linearyzacji klasy C i usuń z list w procesie scalania. Jeśli nie, spójrz na czoło następnej listy i weź je, jeśli jest odpowiednie. Następnie powtarzaj operację, aż wszystkie klasy zostaną usunięte lub nie będzie możliwe znalezienie odpowiednich czołów. W takim przypadku nie można skonstruować scalenia i Python zgłosi wyjątek. Przepis gwarantuje, że operacja łączenia zachowa odpowiednie uszeregowanie.

Przestudiujmy kilka przykładów. Przypadek najprostszy, gdy klasa C dziedziczy tylko z jednej klasy B , prowadzi do $L[C(B)] = C + \text{merge}(L[B], B) = C + L[B]$. Rozważmy przypadek dziedziczenia wielobazowego:

```
class O(object): pass
class F(O): pass
class E(O): pass
class D(O): pass
class C(D,F): pass
class B(D,E): pass
class A(B,C): pass
```

Linearyzacja dla klas bazowych jest trywialna. $L[O] = O$, $L[F] = FO$, $L[E] = EO$, $L[D] = DO$. Linearyzacja dla klasy C: $L[C] = C + \text{merge}(DO, FO, DF)$. Klasa D jest dobrym kandydatem z czoła, więc bierzemy ją i redukujemy dalsze rozważanie do $\text{merge}(O, FO, F)$. Teraz O nie jest dobrym elementem czoła, bo znajduje się również na końcu FO. Musimy przejść do kolejnego elementu. Widzimy, że F jest dobrym kandydatem, więc zabieramy F i zostaje $\text{merge}(O, O)$, które kończy się wybraniem O. Stąd otrzymana sekwencja $L[C] = C D F O$. Tak samo postępujemy w przypadku $L[B] = B + \text{merge}(DO, EO, DE) = B + D + \text{merge}(O, EO, E) = B + D + E + \text{merge}(O, O) = B D E O$. Na koniec klasa najbardziej potomna, czyli $L[A] = A + \text{merge}(BDEO, CDFO, BC) = A + B + \text{merge}(DEO, CDFO, C) = A + B + C + \text{merge}(DEO, DFO) = A + B + C + D + \text{merge}(EO, FO) = A + B + C + D + E + \text{merge}(O, FO) = A + B + C + D + E + F + \text{merge}(O, O) = A B C D E F O$. W tym przykładzie, linearyzacja jest uporządkowana zgodnie z poziomem dziedziczenia, czyli niższe poziomy (czyli bardziej wyspecjalizowane klasy) mają wyższy priorytet. Jednakże, nie zawsze tak jest – rozważmy identyczne klasy jak poprzednio, ale w klasie B zamierzmy kolejność dziedziczenia na `class B(E,D): pass`. Okazuje się, że w takiej sytuacji procedura linearyzacji doprowadzi nas do wyniku $L[A] = A B E C D F O$, czyli klasa E, będąca klasą wyżej położoną w hierarchii dziedziczenia (bliżej nadrzędnej klasy O), wyprzedzi klasę C, która jest w tej hierarchii niżej (bliżej klasy A).

Kolejność, w której zostały uporządkowane klasy nadrzędne w procedurze MRO można łatwo podejrzeć za pomocą metody `mro()` lub atrybutu `__mro__`. Przykładowo, dla ostatniego przykładu otrzymamy w interpreterze:

```
>>> A.mro()
[<class '__main__.A'>, <class '__main__.B'>, <class '__main__.E'>, <class '__main__.C'>,
<class '__main__.D'>, <class '__main__.F'>, <class '__main__.O'>, <class 'object'>]
>>> A.__mro__
(<class '__main__.A'>, <class '__main__.B'>, <class '__main__.E'>, <class '__main__.C'>,
<class '__main__.D'>, <class '__main__.F'>, <class '__main__.O'>, <class 'object'>)
```

Można taką informację uformować nieco bardziej przejrzysto (np. żeby drukować zamiast `<class '__main__.A'>` samą nazwę A), jeśli, wybiegając nieco tematem w kierunku programowania metaklas, zdefiniujemy metodę reprezentacji `__repr__()` dla pewnej klasy Type, dziedziczącej z `type`, w następujący sposób:

```
class Type(type):
    def __repr__(cls):
        return cls.__name__
```

a następnie:

```
class O(object, metaclass=Type): pass
```

W efekcie:

```
>>> A.mro()
[A, B, E, C, D, F, O, <class 'object'>]
>>> A.__mro__
(A, B, E, C, D, F, O, <class 'object'>)
```

Znając strategię szeregowania kolejności klas nadrzędnych, możemy przyrzeć się teraz sytuacji, w której mamy określone klasy i metody, śledząc, które metody są wywoływane, a które nie są. Zaczniemy od dwóch klas bazowych:

```
class Baza(object):
    def __new__(cls, *args):
        print("-> Baza __new__", *args)
        nowy_obiekt = object.__new__(cls)
        print("<- Baza __new__")
        return nowy_obiekt
    def __init__(self, x):
        print("-> Baza __init__", x)
        super().__init__()
        print("-- Baza __init__")
        self.x = x
        print("<- Baza __init__")
    def __str__(self):
        return "{self.x}".format(self=self)
    def id(self):
        print("-Baza-")

class A(object):
    def __new__(cls, *args):
        print("-> A __new__", *args)
        nowy_obiekt = object.__new__(cls)
        print("<- A __new__")
        return nowy_obiekt
    def __init__(self, x):
        print("-> A __init__", x)
        super().__init__(x)
        print("-- A __init__")
        self.x = x
        print("<- A __init__")
    def __str__(self):
        return "{self.x}".format(self=self)
    def id(self):
        print("-A-")
```

Wyposażyliśmy je w funkcję tworzącą instancję obiektu `__new__()`, inicjalizującą `__init__()`, konwertującą obiekt danej klasy na łańcuch znakowy czyli `__str__()`, a także metodę składową instancji klasy `id()`. Warto zwrócić uwagę na jedyną różnicę, mianowicie funkcja `__init__` w klasie Baza, wywołując `__init__()` dla swojej supekasy, `super().__init__()`, nie przekazuje żadnego argumentu. Jest to poprawne, bo funkcja `__init__()` klasy `object` nie oczekuje żadnego argumentu. Klasa A przeciwnie, ma wywołanie `super().__init__(x)`, a to oznacza, że jeśli zechcemy utworzyć obiekt klasy A, to dostaniemy błąd:

```
>>> a = A(123)
-> A __new__ 123
<- A __new__
-> A __init__ 123
Traceback (most recent call last):
  File "<pyshell#90>", line 1, in <module>
    a = A(123)
  File "<pyshell#85>", line 9, in __init__
    super().__init__(x)
TypeError: object.__init__() takes exactly one argument (the instance to initialize)
```

W przypadku tworzenia obiektu klasy Baza wszystko jest w porządku:

```
>>> b = Baza(123)
-> Baza __new__ 123
<- Baza __new__
-> Baza __init__ 123
-- Baza __init__
<- Baza __init__
```

Zdefiniujmy kolejne klasy potomne.

```
class B(Baza):
    def __new__(cls, *args):
        print("-> B __new__", *args)
        nowy_obiekt = super().__new__(cls)
        print("<- B __new__")
        return nowy_obiekt
    def __init__(self, x):
        print("-> B __init__", x)
        super().__init__(x)
        print("-- B __init__")
        self.x = x
        print("<- B __init__")
    def __str__(self):
        return "{self.x}".format(self=self)
    def id(self):
        print("-B-")

class C(B):
    def __new__(cls, *args):
        print("-> C __new__", *args)
        nowy_obiekt = super().__new__(cls)
        print("<- C __new__")
        return nowy_obiekt
    def __init__(self, x):
        print("-> C __init__", x)
        super().__init__(x)
        print("-- C __init__")
        self.x = x
        print("<- C __init__")
    def __str__(self):
        return "{self.x}".format(self=self)
    def id(self):
        print("-C-")
```

Tu widzimy, że klasa B dziedziczy z klasy Baza, z kolei klasa C dziedziczy z klasy B. Utwórzmy przykładowo obiekt klasy B:

```
>>> b2 = B(123)
-> B __new__ 123
-> Baza __new__
<- Baza __new__
<- B __new__
-> B __init__ 123
-> Baza __init__ 123
-- Baza __init__
<- Baza __init__
-- B __init__
<- B __init__
```

Następnie obiekt klasy C:


```
>>> c = C(123)
-> C __new__ 123
-> B __new__
-> Baza __new__
<- Baza __new__
<- B __new__
<- C __new__
-> C __init__ 123
-> B __init__ 123
-> Baza __init__ 123
-- Baza __init__
<- Baza __init__
-- B __init__
<- B __init__
-- C __init__
<- C __init__
```

Otrzymane wydruki są zgodne z oczekiwaniem, jednak warto są komentarza. Pomimo tego, że widzimy metodę `__new__()` uruchomioną łącznie trzykrotnie, to tak naprawdę nie widzimy właściwego momentu tworzenia instancji (alokacji pamięci) – obiekt utworzony zostaje poprzez metodę `__new__()` z klasy `object`. Wszystkie kolejne wywołania to część mechanizmu dziedziczenia, pozwalającego na uruchomienie odpowiednich operacji inicjalizacyjnych, zdefiniowanych w klasach bazowych. Innymi słowy, wywołanie `__new__()` z metody `super()`, jak w klasach `Baza`, `B`, `C`, nie tworzy nowej instancji, ale daje dostęp do metody klasy bazowej dla bieżącej instancji, która jest właśnie w procesie tworzenia. Zwróćmy też uwagę, że w powyższym kodzie celowo nie przekazano argumentu (tutaj `int` o wartości 123) do kolejnych wołań `__new__()`, począwszy od klasy `C`, dlatego argument 123 widzimy tylko w wywołaniu z wydrukiem `-> C __new__ 123`. Dodajmy ostatnią klasę, która będzie dziedziczyć wielokrotnie:

```
class D(A, C):
    def __new__(cls, *args):
        print("-> D __new__", *args)
        nowy_obiekt = super().__new__(cls, *args)
        print("<- D __new__")
        return nowy_obiekt
    def __init__(self, x):
        print("-> D __init__", x)
        super().__init__(x)
        print("-- D __init__")
        self.x = x
        print("<- D __init__")
    def id(self):
        print("-D-")
```

Zacznijmy od MRO dla klasy `D` jak powyżej: `D A C B Baza object`. Teraz utworzymy obiekt klasy `D`:

```

>>> d = D(123)
-> D __new__ 123
-> A __new__ 123
<- A __new__
<- D __new__
-> D __init__ 123
-> A __init__ 123
-> C __init__ 123
-> B __init__ 123
-> Baza __init__ 123
-- Baza __init__
<- Baza __init__
-- B __init__
<- B __init__
-- C __init__
<- C __init__
-- A __init__
<- A __init__
-- D __init__
<- D __init__

```

Widzimy interesującą sekwencję wywołań metody `__new__()`, odbiegającą od poprzednich obserwacji, mianowicie, najpierw wołane jest `__new__()` z klasy D, zgodnie z oczekiwaniem, następnie, również zgodnie z MRO, `__new__()` z klasy A. Ponieważ w klasie D mamy wywołanie `nowy_obiekt = super().__new__(cls, *args)`, czyli z przekazaniem argumentów, stąd widzimy również argument (tutaj 123) w wołaniu `__new__` w klasie A. Intrygujące jest, co dzieje się dalej, nie widzimy wywołania metod `__new__()` z klasy C, B, czy Baza. Powodem jest wybrany sposób wołania `__new__()` w klasie A. Mianowicie, zamiast użycia metody `super()`, zapisano: `nowy_obiekt = object.__new__(cls)`, czyli użycie bezpośredniego odwołania do metody `__new__()` z klasy `object`. Dalsze podążanie według kolejności MRO zostaje przez to przerwane, co oczywiście ma swoje konsekwencje. Ponieważ w klasie A użyto `object.__new__(cls)` zamiast `super().__new__(cls)`, proces nie kontynuuje się dalej w górę hierarchii dziedziczenia. To oznacza, że metody `__new__` z klas C, B, i Baza nie są wywoływane w procesie tworzenia instancji D. W rezultacie, alokacja pamięci dla instancji klasy D ma miejsce właśnie w klasie A podczas wywołania `object.__new__(cls)`. A brak wywołania metod `__new__()` z klas C, B i Baza, może oczywiście prowadzić do błędnego działania kodu, o ile tak zapisany kod nie został stworzony całkowicie świadomie i celowo.

Również ciekawą obserwacją jest fakt, że wykonanie kodu nie kończy się błędem, pomimo wywołania `__init__()` w klasie A, które przekazuje argument `x` dalej. Jest to spowodowane właśnie kolejnością klas w MRO. Gdy próbowaliśmy tworzyć obiekt klasy A, kolejną klasą na liście była klasa `object`, która nie przyjmuje żadnego argumentu w swojej funkcji `__init__()`. Tutaj mamy inną sytuację, kolejną klasą po A jest klasa C, i to do niej odwołuje się wywołanie `super().__init__(x)` z wnętrza klasy A, co jest całkowicie poprawne, bo `__init__()` z klasy C oczekuje argumentu. Jak widać, tym razem odniesienie się nastąpiło poprzez funkcję `super()`. Gdybyśmy, tak jak miało miejsce w metodzie `__new__()` klasy A, użyli `object.__init__(x)`, przerwalibyśmy łańcuch kolejnych wywołań `__init__()`, co na pewno nie byłoby poprawnym kodem. Jednak nadal nie byłoby tu błędu związanego z przekazaniem argumentu `x` do `__init__` z klasy `object`. Wymaga to pewnego komentarza: `object.__init__()` jest specjalnie zaprojektowany, aby akceptować i ignorować dodatkowe argumenty. Dlatego wywołanie `object.__init__(x)` bezpośrednio z klasy A nie powoduje błędu – argument `x` jest po prostu ignorowany. Gdy używasz `super().__init__(x)` w klasie A, `super()` działa inaczej. Funkcja `super()` w Pythonie zwraca obiekt pośrednika, który deleguje wywołania metod do klas w hierarchii dziedziczenia klasy, z której jest wywoływana. W przypadku klasy A, która dziedziczy

bezpośrednio po object, `super().__init__(x)` efektywnie próbuje wywołać `object.__init__()` z argumentem `x`, ale robi to w kontekście `super()`, który nie akceptuje dodatkowych argumentów dla `object.__init__()`. Kiedy `super().__init__(x)` jest wywoływane, Python oczekuje, że metoda `__init__()` klasy nadrzędnej (w tym przypadku `object`) obsłuży ten argument. Ponieważ `object.__init__()` nie przyjmuje argumentów poza `self`, przekazanie `x` poprzez `super()` powoduje błąd typu `TypeError`, ponieważ jest to nieoczekiwany argument dla `object.__init__()`. Zatem kluczowa różnica polega na tym, że bezpośrednie wywołanie `object.__init__(x)` pozwala na ignorowanie dodatkowych argumentów, podczas gdy użycie `super().__init__(x)` stara się przekazać ten argument do klasy nadrzędnej, co w przypadku `object` jest nieprawidłowe i prowadzi do błędu. Mechanizm `super()` ma za zadanie umożliwienie płynnej współpracy klas w hierarchii dziedziczenia, ale wymaga również, aby sygnatury metod były kompatybilne z metodami klas nadrzędnych.

Co się stanie jeśli w klasie D zakomentujemy (lub nie zdefiniujemy) funkcję `__new__()`? Zostanie wywołana, jako pierwsza, funkcja `__new__()` z kolejnej na liście MRO klasy, w tym przypadku klasy A. Tak samo stanie się z funkcją `__init__()`. W klasach mamy, jak widać, zdefiniowaną również metodę instancji `id()`. Jeśli mamy taką metodę w klasie D:

```
def id(self):
    print("-D-")
```

to wywołanie `d.id()` spowoduje wydrukowanie `'-D-'`. Jeśli zakomentujemy `id()` w klasie D, to zostanie wykonana pierwsza napotkana metoda `id()` z klas według porządku MRO. W naszym przypadku następna po klasie D jest klasa A, zatem zostanie wydrukowane `'-A-'`. Jeśli zakomentujemy `id()` w klasie A, to wykonane zostanie to z klasy C, i tak dalej. W Pythonie można wywołać metodę z konkretnej klasy w hierarchii dziedziczenia, nawet jeśli nie jest to pierwsza klasa w kolejności MRO. Aby to zrobić, musisz jawnie odwołać się do klasy, z której chcesz wywołać metodę, i przekazać `self` jako argument. Jest to znane jako "wywołanie niezależne od instancji" (unbound method call). Przykładowo, jeśli chcę wywołać `id()` z klasy A, z użyciem obiektu `d` klasy D, można to zrobić poprzez wywołanie `A.id(d)`. W tym wywołaniu jawnie wskazujemy klasę, z której chcemy wywołać metodę, i przekazujemy bieżącą instancję (`d`) jako argument. To pozwala na kontrolę nad tym, która implementacja metody jest używana, pomimo kolejności MRO. Nie da się tak zrobić w przypadku atrybutów instancji klasy, bo jeśli różne klasy w hierarchii mają atrybut o tej samej nazwie, atrybut ten zostanie nadpisany przez klasę najbardziej potomną, która go zdefiniuje.

Teraz przeanalizujemy sytuację, w której zamieniamy kolejność dziedziczenia na `class D(C, A)`. Bardzo szybko zorientujemy się, że teraz problem z argumentem `x` przekazywanym w `__init__()` dotknie nas na etapie klasy Baza – która w oryginale ma `__init__` bez argumentu, ale teraz według MRO, za Baza stoi klasa A, która ma `__init__` wymagające argumentu. Gdy już to poprawimy, znowu dostaniemy błąd w klasie A, dotyczący przekazywania argumentu `x` do `object.__init__()` jeśli w kodzie mamy zapis `super().__init__(x)` – patrz dyskusja powyżej.

Wróćmy do oryginalnej wersji, czyli `class D(A, C)` i zastanówmy się nad operacjami rzutowania obiektu `d` klasy D. Próba `A(d)` spowoduje wywołanie `__new__` z klasy A, następnie z klasy `object` (alokacja pamięci), a następnie `__init__` z klasy A, no i w kolejnym kroku mamy,

w wywołaniu `super().__init__(x)` nieakceptowalną próbę przemyślenia argumentu `x` do funkcji `__init__()` klasy `object`. Rzutowanie `C(d)` lub `B(d)` lub `Baza(d)` nie stwarza żadnych problemów.

Na koniec pokażmy przypadek, w którym procedura MRO zawodzi. Gdyby napisać, że klasa `class D(B, C)`, to otrzymamy błąd `TypeError: Cannot create a consistent method resolution order (MRO) for bases B, C`. Rozpiszmy kolejne kroki, przypominając, że teraz mamy:

```
class Baza(object): pass
class A(object): pass
class B(Baza): pass
class C(B): pass
class D(B, C): pass
```

a zatem: `L[B] = B Baza object`, `L[C] = C B Baza object`, `L[D] = D + merge(L[B], L[C], BC) = D + merge(B Baza object, C B Baza object, BC)`. `B` nie może być wybrane jako pierwsze, bo znajduje się w ogonie `L[C]`. `C` nie może być wybrane jako pierwsze, ponieważ występuje w ogonie listy klas bazowych dla `D` (`BC`). W tym momencie algorytm utknął, ponieważ nie może wybrać ani `B`, ani `C` jako pierwszego elementu MRO dla `D`. W rezultacie Python zgłosi błąd, ponieważ nie jest w stanie zbudować spójnej kolejności MRO dla klasy `D` ze względu na konflikt w dziedziczeniu. W wielu przypadkach taki konflikt w dziedziczeniu wskazuje na problem w strukturze projektu klasy i może wymagać przemyślanej restrukturyzacji hierarchii dziedziczenia.

METAPROGRAMOWANIE

Metaprogramowanie to możliwość dynamicznego generowania lub modyfikowania kodu w czasie wykonywania. W Pythonie metaprogramowanie jest wspierane przez technikę dekoratorów, metaklasy oraz dynamiczne wykonanie kodu za pomocą funkcji `exec()` i `eval()`.

Dekoratory zostały omówione już wcześniej. W klasycznym zastosowaniu dekorator opakowuje funkcję, dodając jej dodatkowe zachowania, np. dekorator do logowania wywołań funkcji, mierzenia czasu wykonania lub obsługi wyjątków. Dekoratory mogą być używane do kontrolowania dostępu do powiązanej opakowanej funkcji lub metody, np. sprawdzenia ograniczeń bezpieczeństwa, czy aktualny użytkownik jest prawidłowy i czy posiada prawa dostępu do funkcji lub metody. Dekoratory mogą być używane do tworzenia formy buforowania, często określanej jako memoizacja. Ta metoda pozwala funkcji na zapisywanie wyników generowanych dla określonych wartości parametrów. Bufor przechowuje informacje o parametrach i wcześniej wygenerowanych wynikach. Oznacza to, że gdy funkcja jest wywoływana, wykonywane jest sprawdzenie, czy dla danej kombinacji parametrów wynik został już zapisany w buforze. Jest to szczególnie użyteczne w przypadku obliczeń, które są kosztowne w wykonaniu, ponieważ obliczenie jest wymagane tylko raz. Popularny przykład – liczenie wyrazów ciągu Fibonacciego metodą rekurencyjną. Dekoratory mogą być również używane do walidacji i oczyszczania wartości wejściowych. Definiując dekoratory wielokrotnego użytku, które mogą dokonywać takiej ewaluacji danych wejściowych, można zapewnić wspólną obsługę integralności w różnych funkcjach i metodach. Wreszcie, dekoratory są szeroko stosowane w wielu frameworkach i bibliotekach API do łączenia funkcji, klas i metod z tymi frameworków.

Metaklasy zapewniają sposób definiowania zachowania samych klas. W języku Python 3 pojęcie klasy i typu zostało zunifikowane, to znaczy:

```
>>> class Foo:
...     pass
...
>>> x = Foo()
>>> x.__class__
<class '__main__.Foo'>
>>> type(x)
<class '__main__.Foo'>
```

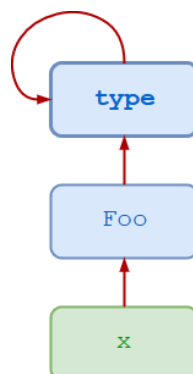
W Pythonie wszystko jest obiektem, klasy też są obiektami. W rezultacie klasa musi mieć typ.

```
>>> class Foo:
...     pass
...
>>> x = Foo()
>>> type(x)
<class '__main__.Foo'>
>>> type(Foo)
<class 'type'>
```

Typ x to klasa Foo, ale typ Foo, samej klasy, to 'type'. Ogólnie typem każdej klasy (Python 3) jest 'type', również typ klas wbudowanych, Sprawdźmy:

```
for t in int, float, dict, list, tuple:
    print(type(t))
```

Tak jak zwykły obiekt jest instancją klasy, tak każda klasa w Pythonie 3, jest instancją metaklasy 'type'. Również type(type) jest <class 'type'>



Funkcję type() można wywołać z trzema argumentami: type(<name>, <bases>, <dict>), gdzie

- <name> określa nazwę klasy, czyli będzie to atrybutem __name__ klasy.
- <bases> określa krotkę klas bazowych, z których klasa dziedziczy, czyli atrybut __bases__ klasy.
- <dict> określa słownik przestrzeni nazw zawierający definicje treści klasy, czyli atrybut __dict__ klasy.

Wywołanie type() w ten sposób tworzy nową instancję metaklasy 'type'. Innymi słowy, dynamicznie tworzy nową klasę.

W pierwszym poniższym przykładzie argumenty <bases> i <dct> przekazane do type() są puste. Nie określono dziedziczenia z żadnej klasy nadrzędnej i nic nie jest początkowo umieszczane w słowniku przestrzeni nazw. Oto najprostsza możliwa definicja klasy:

```
>>> Foo = type('Foo', (), {})
>>> x = Foo()
>>> x
<__main__.Foo object at 0x04CFAD50>
```

Warto tu zwrócić uwagę na atrybuty __name__ oraz __qualname__ (qualified name). __name__ jest to nazwa klasy. Reprezentuje ona nazwę, jaką nadaliśmy klasie podczas jej definiowania. __name__ jest używany w wielu kontekstach, takich jak reprezentacje stringowe klasy, logowanie, debugowanie itp. Natomiast wprowadzony w Pythonie 3.3, __qualname__ jest rozszerzeniem __name__. Dla klas zdefiniowanych na najwyższym poziomie modułu, __qualname__ i __name__ są zwykle takie same:

```
>>> class Foo:
...     pass
...
>>> print(Foo.__name__)
Foo
>>> print(Foo.__qualname__)
Foo
```

Dla klas zagnieżdżonych, __qualname__ zawiera pełną ścieżkę, która prowadzi do tej klasy wewnątrz zewnętrznej struktury kodu. Na przykład, dla klasy zdefiniowanej wewnątrz innej klasy, __qualname__ uwzględnia nazwę klasy zewnętrznej:

```
class Zewnetrzna:
    class Wewnetrzna:
        pass

print(Zewnetrzna.Wewnetrzna.__name__)      # Wypisze 'Wewnetrzna'
print(Zewnetrzna.Wewnetrzna.__qualname__)  # Wypisze 'Zewnetrzna.Wewnetrzna'
```

Podobnie jest dla metody wewnątrz klasy:

```
>>> class Foo:
...     def fun():
...         pass
...
>>> print(Foo.__qualname__)
Foo
>>> print(Foo.fun.__qualname__)
Foo.fun
>>> print(Foo.fun.__name__)
fun
```

__qualname__ jest więc szczególnie przydatny do identyfikacji i rozróżniania klas lub funkcji o tej samej nazwie, ale zdefiniowanych w różnych kontekstach lub hierarchiach w obrębie tego samego kodu. Zapewnia to większą jasność, szczególnie przy korzystaniu z narzędzi introspekcyjnych, analizy kodu, czy debugowania.

Zastanówmy się, czy możliwa jest zmiana atrybutu `__name__` dla utworzonej klasy? Najpierw przypomnijmy, że jeśli tworzymy klasę, w dowolny sposób, a potem jej obiekt (instancję), to pojawiają się wpisy w słowniku danej przestrzeni nazw. Jeśli dzieje się to w głównym skrypcie, to oczywiście jest to przestrzeń nazw o atrybucie `__name__` równym `'__main__'`. Utwórzmy przykładową klasę za pomocą `type()`, dodamy atrybut instancji o nazwie `'a'`, ale również metodę o nazwie `__call__`, która zwracać będzie instancję klasy. Można to porównać do przeciążonego operatora funkcyjnego w C++, dzięki któremu obiekt staje się wywoływalny (callable). Metoda `__call__` pozwala obiektom instancji klasy zachowywać się jak funkcje, czyli można je „wywołać” jak funkcje.

```
Foo = type('Foo', (), {'a': 11, '__call__': lambda self: Foo()})
x = Foo() # tworzenie instancji x klasy Foo
y = x() # to tworzy nową instancję klasy Foo
```

Po wywołaniu `locals()` potrafimy znaleźć słownikowe wpisy dla `'Foo'` oraz `'x'` i `'y'`:

```
>>> locals()
{'__name__': '__main__', '__doc__': None, '__package__': None, '__loader__': <class 'frozen_importlib.BuiltinImporter'>, '__spec__': None, '__annotations__': {}, '__builtins__': <module 'builtins' (built-in)>, 'Foo': <class '__main__.Foo'>, 'x': <__main__.Foo object at 0x000002179ED6B350>, 'y': <__main__.Foo object at 0x00000217A117C860>}
```

Typ oraz atrybut `__name__` dla obiektu `x` i klasy `Foo`:

```
>>> type(x)
<class '__main__.Foo'>
>>> type(x).__name__
'Foo'
>>> Foo.__name__
'Foo'
```

Możemy teraz nadpisać atrybut `__name__`

```
>>> Foo.__name__ = 'Bar' # tak samo dla x.__name__ = 'Bar'
>>> type(x).__name__
'Bar'
>>> Foo.__name__
'Bar'
```

Niemniej, jeśli sprawdzimy `locals()` to nadal widzimy `'Foo': <class '__main__.Foo'>`, `'x': <__main__.Foo object at 0x000002179ED6B350>`, jak również:

```
>>> type(x)
<class '__main__.Foo'>
```

zatem zmiana `__name__` nie zmienia pełnej identyfikacji klasy w kontekście modułu, w którym została zdefiniowana. W skrócie, `__name__` to tylko jedna z właściwości klasy, która odpowiada za jej nazwę, a nie za jej pełną identyfikację w systemie. Pełna identyfikacja klasy obejmuje nazwę modułu i nazwę klasy, a nie jest ona zmieniana poprzez zmianę atrybutu `__name__`. Co ciekawe, na przykład w wydruku `Foo.__dict__` nie ma bezpośrednio atrybutu `__name__` z przestrzeni nazw klasy `Foo`. Wynika to z faktu, że atrybut `__name__` nie jest przechowywany w `__dict__` klasy, a raczej jest to specjalny atrybut klasy, który jest przechowywany osobno

w metadanych klasy. Oprócz `__dict__`, istnieje kilka wbudowanych atrybutów specjalnych, które klasy w Pythonie mają domyślnie, takie jak `__name__`, `__module__`, `__bases__` itp. Te atrybuty można wydrukować razem z zawartością `__dict__`. Na przykład:

```
Foo = type('Foo', (), {'a': 11})
Foo.__name__ = 'Bar'

# Drukowanie atrybutów klasy z __dict__
for key, value in Foo.__dict__.items():
    print(f"{key}: {value}")

# Drukowanie innych ważnych atrybutów specjalnych
print(f"__name__: {Foo.__name__}") # tu otrzymamy naszą nową nazwę, __name__: Bar
print(f"__module__: {Foo.__module__}")
print(f"__bases__: {Foo.__bases__}")
```

Jak widać, żeby odwołać się do tych specjalnych atrybutów klasy, trzeba znać ich nazwy. Jeśli ich nie znamy, można użyć niektórych funkcji z modułu `inspect`, który jest zaprojektowany do wydobywania informacji z obiektów. `inspect` zawiera funkcje takie jak `getmembers()`, które mogą pomóc w uzyskaniu bardziej kompleksowego obrazu atrybutów obiektu lub klasy. Na przykład, można użyć `inspect.getmembers()` do wylistowania atrybutów klasy:

```
import inspect

Foo = type('Foo', (), {'a': 11})
Foo.__name__ = 'Bar'

# Używanie inspect.getmembers() do wylistowania atrybutów klasy Foo
for name, value in inspect.getmembers(Foo):
    print(f"{name}: {value}")
```

Używając funkcji `inspect.getmembers()` na klasie `Foo`, otrzymaliśmy listę tupli, gdzie każda tupla zawiera nazwę atrybutu i jego wartość. Oto niektóre z kluczowych atrybutów wraz z ich wartościami dla klasy `Foo`:

`__class__`: `<class 'type'>` wskazuje, że klasa `Foo` jest typu `type`, co jest standardowym typem metaklas w Pythonie.

`__dict__`: `{'a': 11, '__call__': <function <lambda> at 0x00000217A17276A0>, '__module__': '__main__', '__dict__': <attribute '__dict__' of 'Bar' objects>, '__weakref__': <attribute '__weakref__' of 'Bar' objects>, '__doc__': None}` jest to słownik atrybutów dla klasy `Foo`. Zawiera atrybuty zdefiniowane w klasie, takie jak `'a'`.

`__module__`: `__main__` wskazuje, że klasa `Foo` została zdefiniowana w module `__main__`.

`a: 11` jest to atrybut klasy z wartością 11.

Ponadto, lista zawiera wiele innych atrybutów specjalnych i metod, które są standardowe dla obiektów klas w Pythonie, takie jak metody porównywania (`__eq__`, `__ne__` itd.), metody magiczne (`__repr__`, `__str__` itd.), oraz wsparcie dla garbage collection (`__weakref__`).

Podobny rezultat uzyskamy (gdzie `x` to obiekt klasy `Foo`) za pomocą:

```
for attr_name in dir(x.__class__):
    # getattr() zwraca wartość atrybutu dla podanej nazwy
    attr_value = getattr(x.__class__, attr_name)
    print(f"{attr_name}: {attr_value}")
```

Jakiego sposobu by nie użyć, wyniki **nie** zawierają atrybutu `__name__` z oczekiwaną wartością `'Bar'`. To potwierdza, że niektóre atrybuty specjalne, takie jak `__name__`, `__bases__`, nie są

zwracane przez `dir(Foo)` lub `inspect.getmembers(Foo)` i wymagają bezpośredniego odpytania, bo nie ma ich w słowniku klasy `Foo` czy instancji klasy. Można jednak uzyskać listę nazw takich atrybutów odwołując się do atrybutów typu `type`, ale **nie** będą one posiadać żadnej przypisanej wartości. Przykładowo, za pomocą dekoratora:

```
def deco(cls):
    for attr_name in dir(type(cls)):
        try:
            attr_value = getattr(type(cls), attr_name)
            print(f"{attr_name}: {attr_value}")
        except AttributeError:
            print(f"{attr_name}: nie można pobrać wartości")
    return cls

@deco
class Foo:
    a = 11

# Zmiana nazwy klasy
Foo.__name__ = 'Bar'

# Tworzenie instancji Foo, dekorator zostanie wywołany podczas definicji klasy
foo_instance = Foo()
```

Wtedy wśród nazw zobaczymy `__name__`: `type`, `__bases__`: (`<class 'object'>`,) i inne. Zatem, jeśli automatycznie chcemy pozyskać nazwy specjalnych atrybutów, które dziedziczone są z typu `type`, to najpierw musimy je pobrać i zachować np. w liście, a potem zrobić drugą iterację po atrybutach już konkretnej klasy lub instancji tej klasy.

```
def deco(cls):
    # Pozyskanie nazw atrybutów metaklasy
    special_attrs = dir(type(cls))

    # Wydrukowanie wartości atrybutów klasy
    for attr_name in special_attrs:
        if hasattr(cls, attr_name):
            try:
                attr_value = getattr(cls, attr_name)
                print(f"{attr_name}: {attr_value}")
            except AttributeError:
                print(f"{attr_name}: nie można pobrać wartości")

    return cls

class Foo:
    a = 11

# Zmiana nazwy klasy
Foo.__name__ = 'Bar'

# Tworzenie instancji Foo obciążonej funkcją deco
foo_instance = deco(Foo)()
```

Dopiero po takiej procedurze udaje się zobaczyć, niejako automatycznie `__name__`: `Bar`. Wydawać by się mogło prosty pomysł (jak dostać wartość specjalnych atrybutów klasy, które np. sobie zmieniliśmy) doprowadził nas do całkiem skomplikowanych rozwiązań, wymagających zrozumienia, gdzie dane atrybuty tak naprawdę są obecne i że pozyskanie ich listy jest możliwe na całkiem innym poziomie (tutaj słownika `__dict__` klasy `type`), ale same pozyskanie wartości dla jakiejś konkretnej klasy (np. naszej klasy `Foo`) wymaga albo jawnego

odwołania się do konkretnej nazwy (np. `Foo.__name__`), albo do pozyskania tej nazwy właśnie najpierw z innego słownika (klasy `type`). Powyższy przykład można zapisać za pomocą dekoratora, ale sam dekorator musi zostać zaaplikowany już po naszej zmianie nazwy na 'Bar', czyli:

```
# Definicja klasy Foo
class Foo:
    a = 11

# Zmiana nazwy klasy
Foo.__name__ = 'Bar'

# Zastosowanie dekoratora do klasy Foo po zmianie nazwy
@deco
class Foo(Foo):
    pass

# Tworzenie instancji zmodyfikowanej klasy Foo
foo_instance = Foo()
```

Przejdźmy do kolejnego przykładu utworzenia klasy za pomocą `type()`. W poniższym przykładzie `<bases>` jest krotką z pojedynczym elementem `Foo`, określającym klasę nadrzędną, z której dziedziczy `Bar`. Atrybut `attr` jest umieszczony w słowniku przestrzeni nazw:

```
>>> Bar = type('Bar', (Foo,), dict(attr=100))
>>> x = Bar()
>>> x.attr
100
>>> x.__class__
<class '__main__.Bar'>
>>> x.__class__.__bases__
(<class '__main__.Foo'>,)
```

Jest to ekwiwalent normalnego definiowania:

```
>>> class Bar(Foo):
...     attr = 100
... 
```

W kolejnym przykładzie `<bases>` jest ponownie puste. Dwa obiekty są umieszczane w słowniku przestrzeni nazw za pośrednictwem argumentu `<dict>`. Pierwszy to atrybut o nazwie `attr`, a drugi to funkcja o nazwie `attr_val`, która staje się metodą zdefiniowanej klasy:

```
>>> Foo = type(
...     'Foo',
...     (),
...     {
...         'attr': 100,
...         'attr_val': lambda x : x.attr
...     }
... )

>>> x = Foo()
>>> x.attr
100
>>> x.attr_val()
100
```

Powyższemu odpowiada definicja:

```
>>> class Foo:
```

```
...     attr = 100
...     def attr_val(self):
...         return self.attr
```

W kolejnym przykładzie nieco bardziej złożona funkcja jest zdefiniowana na zewnątrz, a następnie przypisywana do `attr_val` w słowniku przestrzeni nazw za pomocą nazwy `f`:

```
>>> def f(obj):
...     print('attr =', obj.attr)
...
>>> Foo = type(
...     'Foo',
...     (),
...     {
...         'attr': 100,
...         'attr_val': f
...     }
... )

>>> x = Foo()
>>> x.attr
100
>>> x.attr_val()
attr = 100
```

Typowa definicja wyglądałaby tak:

```
>>> def f(obj):
...     print('attr =', obj.attr)
...
>>> class Foo:
...     attr = 100
...     attr_val = f
... 
```

Przenalizujemy teraz po kolei, co się dzieje podczas tworzenia trywialnej klasy `Foo()`:

```
>>> class Foo:
...     pass
...
>>> f = Foo()
```

Wyrażenie `Foo()` prowadzi do utworzenia nowej instancji klasy `Foo`. Kiedy interpreter napotyka `Foo()`, zachodzą następujące zdarzenia:

- wywoływana jest metoda `__call__()` klasy nadrzędnej do `Foo`. Tutaj klasą nadrzędną jest metaklasa `'type'`, więc wywoływana jest metoda `__call__()` dla `'type'`
- ta metoda `__call__()` z kolei wywołuje:
 - `__new__()`
 - `__init__()`

Jeśli `Foo` nie definiuje `__new__()` i `__init__()`, domyślne metody są dziedziczone z przodków `Foo`. Ale jeśli `Foo` definiuje te metody, zastępują one te z klas przodków, co pozwala na zdefiniowanie działania podczas tworzenia `Foo`. Poniżej zdefiniowano niestandardową metodę o nazwie `new()` i przypisano ją jako metodę `__new__()` dla `Foo`:

```
>>> def new(cls):
...     x = object.__new__(cls)
...     x.attr = 100
...     return x
```

```
...
>>> Foo.__new__ = new
>>> f = Foo()
>>> f.attr
100
```

Powyższe przypisanie do `Foo.__new__` modyfikuje zachowanie instancji klasy `Foo`: za każdym razem, gdy tworzona jest instancja `Foo`, domyślnie jest ona inicjowana atrybutem o nazwie `attr`, który ma wartość 100, choć częściej robimy to w metodzie `__init__()`.

Takiego triku (przypisania) nie da się jednak wykonać w stosunku to metaklasy 'type' (w sensie: `type.__new__ = new` zgłosi `TypeError`).

Jakie jest rozwiązanie, jeśli chcemy dostosować proces konkretyzacji klasy?

Jednym z możliwych rozwiązań jest metaklasa, a technicznie ujmując klasa, która pochodzi od 'type', w której możemy wykonywać dowolne zmiany. Zdefiniowanie metaklasz wywodzącej się z 'type':

```
>>> class Meta(type):
...     def __new__(cls, name, bases, dct):
...         x = super().__new__(cls, name, bases, dct)
...         x.attr = 100
...         return x
... 
```

Ponieważ 'type' jest metaklasą, to klasa Meta też jest metaklasą.

Metoda `new` () wykonuje następujące operacje:

- deleguje przez `super()` metodę `__new__()` metaklasz bazowej (type), aby faktycznie utworzyć nową klasę
- tworzy i przypisuje atrybut klasy (ale to zobaczymy dopiero gdy zrobimy klasę z tej metaklasz) `attr` o wartości 100
- zwraca nowo utworzoną (meta)klasę

Teraz, mając już `Meta`, możemy zrobić tak – definiujemy nową klasę `Foo` i określamy, że jej metaklasa jest niestandardową metaklasą `Meta`, a nie standardowym `'type'`. Odbyna się to za pomocą słowa kluczowego `metaclass` w definicji klasy:

```
>>> class Foo(metaclass=Meta):
...     pass
...
>>> Foo.attr
100
```

Atrybut klasy zmieniony poprzez wywołanie `Klasa.atrybut = <nowa wartosc>` zmienia wartość atrybutu `Klasa` wszystkich instancji `Klasa`. Ale jeśli odniesiemy się przez instancję, np. mamy `x = Klasa()`, następnie `x.atrybut = <nowa wartosc>`, to tym samym stworzymy tylko dla obiektu `x` składową instancji o nazwie `atrybut`. W obiekcie `x` nadal da się „dostać” do atrybutu klasy w następujący sposób: **`type(x).atrybut`**

Jeśli na klasę powiemy „fabryka obiektów”, to metaklasy są czasami nazywane „fabrykami klas”. Fabryka obiektów:

```
>>> class Foo:
...     def __init__(self):
...         self.attr = 100
...
>>> x = Foo()
>>> x.attr
```

```
100
>>> y = Foo()
>>> y.attr
100
```

Fabryka klas:

```
>>> class Meta(type):
...     def __init__(
...         cls, name, bases, dct
...     ):
...         cls.attr = 100
...
>>> class X(metaclass=Meta):
...     pass
>>> X.attr
100
>>> class Y(metaclass=Meta):
...     pass
>>> Y.attr
100
```

Dla osób szczególnie zainteresowanych tematem metaklas, można polecić dwa dokumenty: Understanding Python metaclasses (<https://blog.ionelmc.ro/2015/02/09/understanding-python-metaclasses/>) oraz Python metaclasses (<https://jfreeman.dev/blog/2020/12/07/python-metaclasses/>).

PROJEKTOWANIE OBIEKTOWE

W języku C++ programowanie obiektowe umożliwia definiowanie abstrakcyjnych klas bazowych, które są z natury przeznaczone do tego, żeby z nich dziedziczyć, ale nie tworzyć obiektów tych klas. W języku Python możliwość taką daje moduł `abc`. W module tym mamy metaklasę `ABCMeta` oraz pomocniczą klasę `ABC` (Abstract Base Classes). Klasy abstrakcyjne to klasy zawierające jedną lub więcej metod abstrakcyjnych. Metoda abstrakcyjna to metoda, która została zadeklarowana, ale nie zawiera implementacji. Nie można utworzyć instancji klas abstrakcyjnych i wymagają one, aby podklasy zapewniały implementacje metod abstrakcyjnych. W przypadku zwykłego dziedziczenia, nie ma żadnego mechanizmu, który by zabronił utworzyć obiekt „klasy abstrakcyjnej”, np.:

```
class AbstractClass:
    def do_something(self):
        pass

class B(AbstractClass):
    pass

a = AbstractClass()
b = B()
```

W naszym przykładzie zaimplementowano przypadek prostego dziedziczenia, który nie ma nic wspólnego z klasą abstrakcyjną. W rzeczywistości Python sam w sobie nie zapewnia klas abstrakcyjnych. Jednak Python jest dostarczany z modulem, który zapewnia infrastrukturę do definiowania abstrakcyjnych klas podstawowych (ABC). Moduł ten z oczywistych względów nosi nazwę `abc`. Poniższy kod Pythona używa modułu `abc` i definiuje abstrakcyjną klasę bazową:

```
from abc import ABC, abstractmethod

class AbstractClassExample(ABC):
    def __init__(self, value):
        self.value = value        super().__init__()

    @abstractmethod    def do_something(self):
        pass
```

Zdefiniujemy teraz podklasę przy użyciu wcześniej zdefiniowanej klasy abstrakcyjnej.

```
class DoAddTen(AbstractClassExample):
    pass

x = DoAddTen(1.23)
```

Zauważmy, że nie zaimplementowaliśmy metody `do_something`, mimo że musimy ją zaimplementować, ponieważ metoda ta jest udekorowana jako metoda abstrakcyjna z dekoratorem „`abstractmethod`”. Otrzymujemy wyjątek, że nie można utworzyć instancji `DoAddTen`:

Traceback (most recent call last):

File ".\tktest.py", line 15, in <module>

x = DoAddTen(1.23)

TypeError: Can't instantiate abstract class DoAddTen with abstract methods do_something

Zrobimy to poprawnie w poniższym przykładzie, w którym definiujemy dwie klasy dziedziczące po naszej klasie abstrakcyjnej:

```
class DoAddTen(AbstractClassExample):
    def do_something(self):
        return self.value + 10

class DoMulTen(AbstractClassExample):
    def do_something(self):
        return self.value * 10

x = DoAddTen(1.23)
y = DoMulTen(1.23)

print(x.do_something()) # wynik 11.23
print(y.do_something()) # wynik 12.3
```

Nie można utworzyć obiektu klasy, która pochodzi z klasy abstrakcyjnej, dopóki nie zostaną zdefiniowane wszystkie jej metody abstrakcyjne.

Metoda abstrakcyjna może mieć też implementację w klasie abstrakcyjnej, ale nawet jeśli zostanie tam zaimplementowana, w podklasach trzeba będzie nadpisać implementację. Podobnie jak w innych przypadkach „normalnego” dziedziczenia, metodę abstrakcyjną można wywołać za pomocą mechanizmu wywołania `super()`. Pozwala to na udostępnienie podstawowych funkcjonalności w metodzie abstrakcyjnej, które można wzbogacić implementacją podklasy.

```
from abc import ABC, abstractmethod

class AbstractClassExample(ABC):
    @abstractmethod    def do_something(self):
        print("Działanie ogólne!")
```

```
class AnotherSubclass(AbstractClassExample):
    def do_something(self):
        super().do_something()
        print("Działanie szczegółowe!")

x = AnotherSubclass()
x.do_something()
```

Innymi praktycznie używanymi dekoratorami, są `@classmethod` i `@staticmethod`. Przykładowo, zacznijmy od klasy, z której można stworzyć obiekt poprzez `__init__`

```
class Student(object):
    def __init__(self, first_name, last_name):
        self.first_name = first_name
        self.last_name = last_name

jnowak = Student('Jan', 'Nowak')
```

Do klasy można dodać teraz udekorowaną metodę, która stanie się metodą klasy – pierwszy argument metody to będzie `cls` (odniesienie do obiektu klasy – nazwa, podobnie jak w przypadku `self`, jest arbitralna). Ten parametr `cls` jest obiektem klasy, który pozwala metodom `@classmethod` na łatwe tworzenie instancji klasy, niezależnie od dziedziczenia.

```
class Student(object):
    def __init__(self, first_name, last_name):
        self.first_name = first_name
        self.last_name = last_name

    @classmethod
    def from_string(cls, name_str):
        first_name, last_name = map(str, name_str.split(' '))
        student = cls(first_name, last_name)
        return student

jnowak = Student.from_string('Jan Nowak')
print(jnowak.first_name, jnowak.last_name)
```

Dzięki takiej metodzie można utworzyć obiekt klasy, tak jak konstruktorem. Powyższy przykład jest bardzo prosty, ale można sobie wyobrazić bardziej skomplikowane sytuacje, gdy obiekt `Student` można pozyskiwać w wielu różnych formatach, i za pomocą tej strategii nadal budować obiekt klasy `Student`.

```
class Student(object):

    @classmethod
    def from_string(cls, name_str):
        first_name, last_name = map(str, name_str.split(' '))
        student = cls(first_name, last_name)
        return student

    @classmethod
    def from_json(cls, json_obj):
        return student

    @classmethod
    def from_pickle(cls, pickle_file):
        return student
```

Dekorator `@staticmethod` jest podobny do `@classmethod` w tym sensie, że można go wywołać bez instancji obiektu klasy, chociaż w tym przypadku nie ma parametru `cls` przekazanego do jego metody. Przykład:

```
@staticmethod
def is_full_name(name_str):
    names = name_str.split(' ')
    return len(names) > 1
```

```
Student.is_full_name('Adam Robak') # True  
Student.is_full_name('Robak')      # False
```

Ponieważ nie jest przekazywany żaden obiekt `self`, oznacza to, że nie mamy również dostępu do żadnych danych instancji, a zatem ta metoda nie może być wywołana również dla obiektu, którego instancja została utworzona. Brak parametru `cls` w metodach `@staticmethod` czyni je prawdziwymi metodami statycznymi w tradycyjnym sensie. Ich głównym celem jest zawarcie logiki odnoszącej się do klasy, ale ta logika nie powinna potrzebować konkretnych danych instancji klasy.

9. DODATKI

PRINT

Funkcja biblioteczna `print` jest zapewne jedną z pierwszych rzeczy, które poznaje się w języku Python (i nie tylko). Możemy sprawdzić, co o funkcji mówi metoda `type`:

```
>>> type(print)
<class 'builtin_function_or_method'>
```

Przyjrzyjmy się kilku mniej znanym faktom na jej temat. Specyfikacja funkcji oraz opis:

```
print(*objects, sep=' ', end='\n', file=None, flush=False)
```

Funkcja wysyła argumenty pozycyjne (`objects`) do pliku strumienia tekstowego, rozdzielone przez `sep` i zakończone `end`. `sep`, `end`, `file` oraz `flush` są argumentami słów kluczowych i posiadają domyślne wartości (jak powyżej). Zarówno `sep`, jak i `end` muszą być ciągami znaków. Podanie wartości `None` (czyli `sep=None`, `end=None`) oznacza użycie wartości domyślnych. Jeśli nie podano żadnych obiektów, `print()` wyśle do strumienia tylko `end`, zatem domyślnie – przejdzie do nowej linii. Wszystkie argumenty niebędące słowami kluczowymi są konwertowane na ciągi znaków w taki sposób, jak robi to `str()`, i wysyłane do strumienia, rozdzielone przez `sep` i zakończone `end`. Argument `file` musi być obiektem z metodą `write(string)`, jeśli go nie ma lub jest `None`, zostanie użyte `sys.stdout`. Ponieważ drukowane argumenty są konwertowane na ciągi tekstowe, `print()` nie może być używane z obiektami plików w trybie binarnym. Tyle oficjalnej dokumentacji.

Pierwszą interesującą rzeczą jest kreatywne użycie argumentów `sep` i `end`. W szczególności, `sep` i `end` mogą zawierać znaki sterujące, wymieńmy te, które mogą dać jakiś efekt: `\n` nowa linia, `\t` separator w postaci tabulacji, `\r` powrót kursora na początek bieżącej linii, `\b` powrót kursora o jeden znak wstecz, `\a` prawdopodobnie spowoduje wydanie dźwięku. Pamiętajmy, że argumenty `sep` i `end` mogą zawierać dowolny łańcuch znakowy, zatem także kilka znaków sterujących. Argument `flush` (domyślna wartość `False`) reguluje to, czy strumień zostanie najpierw skompletowany, zanim zostanie pokazany, zaś w przypadku wartości `True`, strumień jest niezwłocznie przekazywany (np. wyświetlany na ekranie lub zapisywany w pliku).

Wykonajmy kilka testów za pomocą poniższego kodu.

```
from time import sleep
SEP = ' '
END = '\n'
FLUSH = False
for i in range(10):
    print(i, sep = SEP, end = END, flush = FLUSH)
    sleep(0.5)
```

Proponuję zacząć od testu, gdy `FLUSH = False`, a następnie zamienić na `True`. `END` powinno być różne od `\n` (czyli na przykład spacja, jak wyżej), bo dla `\n` nie zobaczymy żadnego efektu z `flush`. W kolejnych testach proponuję sprawdzić dowolne kombinacje, np. `SEP = '\t'` żeby zobaczyć przeskoki o tabulację, `SEP = '\r'`, żeby drukować od początku linii, albo `\b` czyli powrót o jeden znak.

Używając sekwencji znaków `| / - \` można zasymulować „efekt” rotacji ASCII (ang. *spinner*, albo *throbber*). Podobnie w przypadku sekwencji znaków `. o O o`. Przykładowy kod (przebiegamy po elementach łańcucha 10 razy powielonego):

```
import time

# wybierz jeden z wariantow ponizej
for char in "|/-\\"*10:
# for char in ".oOo"*10:
    print(' ', char, ' ', end='', flush=True)
    time.sleep(0.2)
    print('\r', end='', flush=True)
```

Kilka przykładów, kreatywnie prezentujących możliwości drukowania w jednej linii.

Emluator pisania na maszynie. Kod ASCII numer 9619 oznacza symbol . Funkcja `chr()` zwraca znak ASCII podany numerem argumentu, funkcja `ord()` zwraca numer ASCII dla symbolu będącego argumentem.

```
import time

text = "Tekst pisany na maszynie!"
for char in text:
    print('\b', chr(9619), end='', sep='', flush=True)
    time.sleep(0.1)
    print('', end='\b', sep='', flush=True)
    print(char, ' ', end='', sep='', flush=True)
    time.sleep(0.3)
print('\b ')
```

Odbijająca się piłeczka. Trik polega na przerysowaniu kolejnych położów znaku 'O', aż do osiągnięcia prawej granicy (dla ruchu w prawo) lub lewej (dla ruchu w lewo).

```
import time

czas = 10
szerokosc = 30
pozycja = 0
kierunek = 1
koniec = time.time() + czas

while time.time() < koniec:
    print('\r' + ' ' * pozycja + 'O' + ' ' * (szerokosc - pozycja), end='', flush=True)
    pozycja += kierunek

    if pozycja == szerokosc or pozycja == 0:
        kierunek *= -1

    time.sleep(0.05)
print()
```

Powyższe przykłady operują na możliwościach funkcji `print()` w ramach jednej linii. Nie da się bowiem cofnąć kursora do poprzedniej linii, tylko do początku bieżącej, a przejście do kolejnej linii jest nieodwracalne. Można jednak wykonać trik z czyszczeniem całego ekranu, wtedy kursor

ładuje na początku u góry i można wtedy przerysować zawartość więcej niż jednej linii. Wykonując takie przerysowania odpowiednio często, otrzymamy wrażenie animacji, choć raczej niezbyt płynnej. Czyszczenie ekranu jest operacją zależną od systemu operacyjnego, ale z pomocą modułu `os`, odpytanie atrybutu `os.name` powinno być wystarczające, aby wysłać odpowiednią komendę. Pole `os.name` dla systemu Windows będzie zawierać `'nt'` (i wtedy czyścimy terminal instrukcją `cls`), dla Linux i macOS `'posix'` (czyszczenie instrukcją `clear`). Zatem odpowiedni fragment kodu, to:

```
os.system('cls' if os.name == 'nt' else 'clear')
```

Zwróćmy uwagę na formę wyrażenia warunkowego (zapisanego w nawiasie). Jeśli spełniony jest warunek „if”, zwracana jest wartość po lewej, jeśli nie jest spełniony warunek „if”, zwracana jest wartość występująca po „else”. Wykonawszy czyszczenie ekranu, możemy przystąpić do dowolnego przerysowania zawartości, zwykle kilku-kilkunastu linii, korzystając z wyżej ilustrowanych technik.

Przykładowo, **ruchu piłeczki w pionie**, można emulować takim programem:

```
import os
import time

czas = 10
wysokosc = 20
pozycja = 0
kierunek = 1
koniec = time.time() + czas

while time.time() < koniec:
    os.system('cls' if os.name == 'nt' else 'clear')
    print('\n' * pozycja + 'O')
    pozycja += kierunek

    if pozycja == wysokosc or pozycja == 0:
        kierunek *= -1

    time.sleep(0.05)
```

Bardziej zaawansowany program rysuje **dynamiczny kwadrat** (złożony ze znaków `chr(9608)` , `chr(9604)` , `chr(9600)` ) , którego wielkość „pulsuje” (powiększa się i pomniejsza) w czasie.

```
import time
import os

znak = chr(9608)
gorny_znak = chr(9600)
dolny_znak = chr(9604)
rozmiar = 10
czas = 10
kierunek = 1
biezacy_rozmiar = 1
koniec = time.time() + czas

while time.time() < koniec:
```

```

os.system('cls' if os.name == 'nt' else 'clear')
print(dolny_znak + dolny_znak*2 * biezacy_rozmiar + dolny_znak)
for _ in range(biezacy_rozmiar):
    print(znak + ' ' * biezacy_rozmiar*2 + znak)
print(gorny_znak + gorny_znak*2 * biezacy_rozmiar + gorny_znak)

biezacy_rozmiar += kierunek
if biezacy_rozmiar == rozmiar or biezacy_rozmiar == 1:
    kierunek *= -1
time.sleep(0.1)

```

Omówmy teraz ostatni z argumentów funkcji `print` wraz z jego wartością domyślną, czyli `file = sys.stdout`. W Pythonie, `sys.stdout` to standardowy strumień wyjściowy, którym zazwyczaj jest terminal lub konsola, w której uruchomiony jest skrypt. Działa on podobnie jak strumień wyjściowy w wielu innych językach programowania. Funkcje `print()` oraz tekst zachęty z funkcji `input()`, domyślnie przekazują dane do `sys.stdout`. Istnieją różne rodzaje buforowania strumieni, takich jak `stdout` (standardowe wyjście). W interaktywnych aplikacjach lub skryptach, gdzie oczekuje się natychmiastowego wyświetlenia danych po każdym wierszu, `stdout` używa buforowania wierszowe (line-buffered). W tym trybie buforowania dane są zapisywane do rzeczywistego miejsca docelowego (np. konsoli, pliku) wtedy, gdy napotykaną jest znak nowej linii (np. `\n`) w strumieniu. Dzięki temu użytkownik widzi każdy wiersz informacji natychmiast po jego wygenerowaniu. W nieinteraktywnych aplikacjach, strumień `stdout` jest buforowany blokowo (block-buffered). W tym trybie dane są gromadzone w buforze do momentu, gdy bufor osiągnie pewien rozmiar (np. 4096 bajtów) zanim zostaną zapisane do miejsca docelowego. Jest to bardziej wydajne, gdy dane są zapisywane w dużych ilościach, ponieważ zmniejsza to liczbę operacji zapisu. W niektórych sytuacjach może to jednak prowadzić do opóźnień w wyświetlaniu danych, ponieważ muszą one najpierw wypełnić bufor przed ich przekazaniem dalej.

Podsumowując, buforowanie wierszowe jest bardziej interaktywne i odpowiednie dla sytuacji, gdy chcemy natychmiastowe odpowiedzi (np. komunikaty na konsoli). Buforowanie blokowe jest bardziej wydajne dla operacji, które przetwarzają duże ilości danych, ale może prowadzić do opóźnień w wyświetlaniu danych. W kontekście strumienia `stdout`, może on działać w jednym z tych trybów w zależności od sytuacji. Buforowanie strumienia można wyłączyć, przekazując opcję wiersza poleceń `-u` lub ustawiając zmienną środowiskową `PYTHONUNBUFFERED`.

W jaki sposób ustawienie trybu buforowania wpływa na opcję `flush` w funkcji `print()`? Ustawienie `PYTHONUNBUFFERED` jako zmienną środowiskową na wartość `"TRUE"` (a nawet samo zdefiniowanie tej wielkości), nadpisuje ustawienie `flush=False`, nadpisuje również `os.unsetenv('PYTHONUNBUFFERED')` jeśli takie jest w kodzie. Podpowiedź: jeśli działamy pod Windows, dla bieżącej sesji PowerShell, ustawienie zmiennej:

➤ `$env:PYTHONUNBUFFERED = "TRUE"`

zaś skasowanie zmiennej to

➤ `Remove-Item Env:PYTHONUNBUFFERED`

Użycie modułu `os` oraz ustawienia:

```
os.environ['PYTHONUNBUFFERED'] = "TRUE"
```

nie nadpisuje ustawienia `flush=False`. Polecam wykonać kilka prostych testów sprawdzających powyższe informacje. Pamiętajmy, że jeżeli uruchamiamy przykłady w środowisku IDE, to zachowanie typu `flush=True` może być wymuszone. Testy należy zatem przeprowadzać w terminalu (xterm, PowerShell itp.).

Wreszcie, argument `file` pozwala oczywiście na wskazanie strumienia, który może być strumieniem plikowym, ale ogólniej - argument `file` musi otrzymać obiekt posiadający metodę `write(string)`. Zobaczmy dwa przykłady.

Pierwszy, klasyczny przykład **otwarcia pliku do zapisu** i ustawienia go jako argumentu w funkcji `print` (nie wchodzimy tu w szczegóły składni języka Python, więc jeśli coś jest niejasne, to trzeba wspomóc się podpowiedzią – może np. zaznaczyć fragment kodu i z pomocą GitHub Copilot Chat przeczytać wyjaśnienie):

```
# Otwieramy plik "output.txt" do zapisu
with open('output.txt', 'w') as file:
    # Użyj argumentu file w funkcji print(), aby przekierować wyjście do pliku
    print("Tekst zapisany w pliku, a nie wydrukowany w konsoli.", file=file)
```

Można też ustawić `os.stdout` na plik:

```
import sys

# Otwieramy plik "output.txt" do zapisu
with open('output.txt', 'w') as file:
    # Przekierowujemy sys.stdout do pliku
    sys.stdout = file
    # Używamy funkcji print(), aby zapisać tekst w pliku "output.txt"
    print("Tekst zostanie zapisany w output.txt, a nie wydrukowany w konsoli.")

# Przywracamy oryginalną wartość sys.stdout
sys.stdout = sys.__stdout__

# Ten tekst zostanie wydrukowany w konsoli
print("Ten tekst zostanie wydrukowany w konsoli.")
```

Drugi przykład demonstruje, że można zdefiniować **obiekt, który posiada wymaganą metodę `write()`** i wtedy może on być podany jako wartość argumentu `file` w funkcji `print()`. Poniższy kod zakłada podstawową znajomość pisania klas w języku Python, więc nie jest to konieczne na początku nauki, raczej jako ciekawostka uzupełniająca naszą wiedzę.

```
class Pisarz:
    def __init__(self):
        self.dane = []

    # tu jest wymagana metoda
    def write(self, tekst):
        self.dane.append(tekst)
```

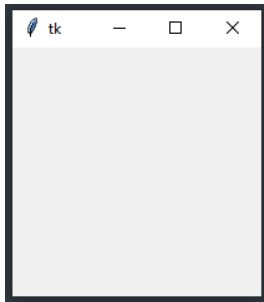
```
def czytajDane(self):  
    return ''.join(self.dane)  
  
# tworzymy obiekt  
pisarz = Pisarz()  
  
print("Pierwszy tekst do magazynu. ", end="", file=pisarz)  
print("Druga porcja. Potem wszystko zostanie wyswietlone!", file=pisarz)  
  
print(pisarz.czytajDane())
```

Buforowanie wyjścia jest zazwyczaj określone przez plik. Jednakże, jeśli wartość `flush=True`, to opróżnianie strumienia jest wymuszone.

W powyższym materiale nie zostały w ogóle poruszone kwestie formatowania, które często towarzyszą użyciu funkcji `print()`. Nie omówiono również użycia wzbogaconej wersji funkcji `pprint` (z modułu `pprint`), dedykowanej do bardziej czytelnego drukowania struktur danych. A zatem – na pewno poruszony tu materiał można będzie rozbudować o kolejne wątki w przyszłości!

TKINTER

Tkinter to biblioteka, która umożliwia tworzenie interfejsu graficznego użytkownika (GUI) w Pythonie. Tkinter jest zintegrowana z systemem Tk, biblioteką napisaną w języku C, która jest używana do tworzenia interfejsu graficznego użytkownika na różnych platformach. Dzięki Tkinter można tworzyć okna, przyciski, menu i inne elementy interfejsu graficznego, a także obsługiwać zdarzenia i wykonywać inne operacje związane z interfejsem graficznym. Tkinter jest dostępny w standardowym pakiecie Python, więc nie trzeba go instalować osobno.



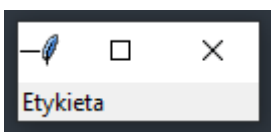
Takie proste okienko, skalowalne i z domyślnym tytułem na belce, otrzymamy już za pomocą trzech linii kodu:

```
import tkinter as tk
win = tk.Tk()
win.mainloop()
```

Możemy dodać do tego tytuł oraz zablokować skalowanie:

```
import tkinter as tk
win = tk.Tk()
win.title("Python GUI")
win.resizable(False, False) # albo True, ale X lub Y muszą być podane
win.mainloop()
```

Dodajmy pierwszy element, etykietę, która jest widżetem (ang. widget). Widżety są częścią interfejsu graficznego użytkownika (GUI), służącymi do wyświetlania informacji lub umożliwienia użytkownikowi wprowadzenia danych. W Tkinter, widżetami są elementy takie jak okna, przyciski, pola tekstowe, listy i inne. Widżety są tworzone za pomocą odpowiednich klas z biblioteki Tkinter i są umieszczane w oknie (ramce) za pomocą metody "pack" lub "grid". Można zmieniać wygląd widżetów za pomocą ich atrybutów, takich jak rozmiar, kolor tła, czcionka i inne. Można też obsługiwać różne zdarzenia związane z widżetami, takie jak kliknięcie przycisku lub wprowadzenie tekstu do pola tekstowego.



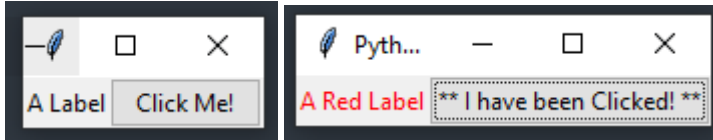
```
import tkinter as tk
from tkinter import ttk
win = tk.Tk()
win.title("Python GUI")
ttk.Label(win, text="Etykieta").grid(column=0, row=0)
win.mainloop()
```

Użycie widżetu Label spowodowało minimalizację obszaru potrzebnego na wyświetlenie zawartości (efektywnie: mniejsze okienko). Widzimy tu użycie ttk. Themed Tk to biblioteka, która umożliwia zmianę wyglądu interfejsu graficznego użytkownika (GUI) utworzonego za pomocą biblioteki Tkinter. Themed Tk dostarcza różnych motywów (skórek), które mogą być użyte do zmiany wyglądu elementów interfejsu graficznego, takich jak okna, przyciski, pola tekstowe i inne. Można użyć Themed Tk, aby dostosować wygląd interfejsu graficznego do

indywidualnych preferencji lub wymagań projektowych 

<https://docs.python.org/3/library/tkinter.ttk.html>

Dodajmy przycisk (Button):



```
import tkinter as tk
from tkinter import ttk

win = tk.Tk()
win.title("Python GUI")

a_label = ttk.Label(win, text="A Label")
a_label.grid(column=0, row=0)

def click_me():
    action.configure(text="** I have been Clicked! **")
    a_label.configure(foreground='red')
    a_label.configure(text='A Red Label')

action = ttk.Button(win, text="Click Me!", command=click_me)
action.grid(column=1, row=0)

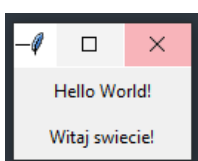
win.mainloop()
```

Obiekty ttk mają nieco inne opcje niż obiekty tk. Atrybuty widżetów modyfikuje się przez funkcję configure. Tło – foreground (w obiekcie tk jest to fg). Przycisk (Button) pozycjonujemy na prawo od etykiety (Label) poprzez podanie column=1. Jak widać, akcja (funkcja o nazwie click_me) wykonana przez kliknięcie Button jest przekazana przez argument command=click_me.

Sposoby rozmieszczenia elementów w nadrzędnym oknie

- pack – orientowanie elementów w relacji do wolnego miejsca, wraz ze wskazaniem preferowanej pozycji (side – LEFT, RIGHT, TOP, BOTTOM), (fill – X, Y, BOTH), (expand – True, False)
- grid – traktuje nadrzędne okno jako podzielone na komórki (wiersze, kolumny) (row=0, column=0), columnspan, rowspan – łączenie, domyślnie widżet siedzi w środku komórki, chyba że użyjemy argumentu sticky (N,S,E,W); NE – top right, jeśli chcemy zająć całą wysokość: NS, szerokość: EW, całość: NSEW
- place – precyzyjna lokacja (x,y w pikselach) można też podać względne położenie relx, rely, a także skalowanie relwidth, relheight. place można mieszać w jednym nadrzędnym widżecie albo z pack, albo z grid

Trochę bardziej obiektowy przykład, z klasą zawierającą dwa widżety Label:




```
import tkinter as tk

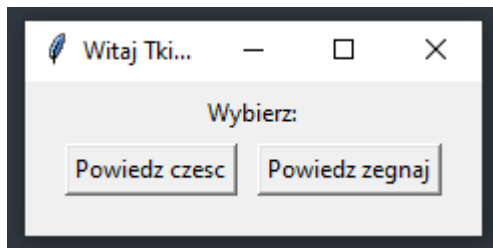
class Window(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Witaj Tkinter")

        label = tk.Label(self, text="Hello World!")
        label2 = tk.Label(self, text="Witaj świecie!")
        label.pack(fill=tk.BOTH, expand=1, padx=10, pady=5)
        label2.pack(fill=tk.BOTH, expand=1, padx=10, pady=5)

if __name__ == "__main__":
    window = Window()
    window.mainloop()
```

Dziedziczymy z głównego okna – widżetu Tk, dzięki czemu możemy ustawić np. tytuł. Tekst wyświetlami za pomocą widżetu Label, definiując widżet zawsze pierwszy argument wskazuje na rodzica (tutaj: self, wskazuje na Window). Po stworzeniu obiektu, wywołujemy mainloop() i widzimy okienko. Jeśli do padx i pady przekazujemy jedną liczbę, to jest to margines z obu stron. Można też przekazać krotkę, wtedy jest to (lewy, prawy) albo (góra, dół).

Dodamy teraz widżety Button, które wykonają wskazaną przez argument command funkcję.



```
import tkinter as tk

class Window(tk.Tk):
    def __init__(self):
        super().__init__()

        self.title("Witaj Tkinter")
        self.label = tk.Label(self, text="Wybierz:")
        self.label.pack(fill=tk.BOTH, expand=1, padx=10, pady=5)
        hello_button = tk.Button(self, text="Powiedz czesc", command=self.say_hello)
        hello_button.pack(side=tk.LEFT, padx=(20, 5), pady=(0, 20))
        goodbye_button = tk.Button(self, text="Powiedz zegnaj", command=self.say_goodbye)
        goodbye_button.pack(side=tk.RIGHT, padx=(5, 20), pady=(0, 20))

    def say_hello(self):
        self.label.configure(text="Czesc!")

    def say_goodbye(self):
        self.label.configure(text="Zegnaj po 2 sek!")
        #self.after(2000, self.destroy)

if __name__ == "__main__":
    window = Window()
    window.mainloop()
```

Przyciski ulokowane są za pomocą pack oraz side=tk.LEFT i RIGHT, odpowiednio. Dodatkowo, w funkcji say_goodbye wywołać możemy funkcję after (proszę odkomentować), która po zadanim czasie (ms) wykona jakąś inną funkcję, np. destroy zamknie okienko.

Gdybyśmy chcieli, zamiast wołać `configure`, zastosować lokalną zmienną z tekstem, do której byśmy wpisywali nowy ciąg znaków (w funkcjach `say_hello` i `say_goodbye`), to nic nie pokaże się. Tkinter wymaga zastosowania specjalnych zmiennych: `StringVar`, `IntVar`, `DoubleVar`, `BooleanVar`. Zmienną danego typu tworzymy tak jak obiekt klasy:

```
label_text = tk.StringVar()
```

Ustawienie lub pobranie ich zawartości realizuje się przez `set()` i `get()`. Nasz kod będzie wymagał takich zmian:

```
self.label_text = tk.StringVar()
self.label_text.set("Choose One")
self.label = tk.Label(self, textvar=self.label_text)
```

oraz

```
def say_hello(self):
    self.label_text.set("Czesc!")

def say_goodbye(self):
    self.label_text.set("Zegnaj po 2 sek!")
```

Zwróćmy uwagę na zmienną klucz `textvar` (a nie `text`) w obiekcie `Label`. Możemy też wprowadzić pop-up windows.

```
import tkinter.messagebox as messagebox
```

oraz

```
def say_hello(self):
    self.label_text.set("Czesc!")
    messagebox.showinfo("Tytuł", "To jest treść!")

def say_goodbye(self):
    self.label_text.set("Zegnaj po 2 sek!")
    messagebox.showinfo("Belka tytułowa", "Trochę tekstu na do widzenia!")
    self.after(2000, self.destroy)
```

Wyskakujące okienko zatrzymuje, aż do kliknięcia „OK” dalsze wykonanie. Podobnie można wyemitować ostrzeżenie lub komunikat błędu (inna ikona, ew. dźwięk), np. zmieńmy na:

```
msgbox.showwarning("Tytuł", "To jest treść!")
```

```
msgbox.showerror("Belka tytułowa", "Trochę tekstu na do widzenia!")
```

Do informacji zwrotnej od użytkownika przewidziane są okienka: **askquestion** (dowolne pytanie, zwraca ciąg „yes” lub „no”), **askyesno** (podobnie, zwraca 1 dla tak, None dla nie), **askokcancel** (zwraca 1 dla OK, None dla Cancel), **askretrycancel** (Retry zwraca 1, Cancel zwraca None). Zmieńmy zawartość funkcji `say_goodbye`

```
def say_goodbye(self):
    if msgbox.askyesno("Zamknąć?", "Czy na pewno chcesz zamknąć to okno?"):
        self.label_text.set("Okno zniknie za 2 sekundy.")
        self.after(2000, self.destroy)
    else:
        msgbox.showinfo("Super!", "A jednak nie! Zatem okno pozostanie.")
```

Można oczywiście warunek – okno rozpisać:

```
close = msgbox.askyesno("Close Window?", "Would you like to close this window?")
if close:
    self.close()
```

Do wprowadzenia dowolnego ciągu można użyć widżet Entry. Kluczowe elementy kodu to:

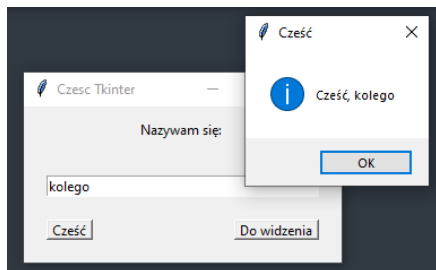
```
self.name_text = tk.StringVar()

self.name_entry = tk.Entry(self, textvar=self.name_text)
self.name_entry.pack(fill=tk.BOTH, expand=1, padx=20, pady=20)
```

Wprowadzoną zawartość można odczytać metodą get()

```
self.name_entry.get()
```

Przykład i cały kod:



```
import tkinter as tk
import tkinter.messagebox as msgbox

class Window(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Czesc Tkinter")
        self.label_text = tk.StringVar()
        self.label_text.set("Nazywam się: ")

        self.name_text = tk.StringVar()

        self.label = tk.Label(self, textvar=self.label_text)
        self.label.pack(fill=tk.BOTH, expand=1, padx=100, pady=10)

        self.name_entry = tk.Entry(self, textvar=self.name_text)
        self.name_entry.pack(fill=tk.BOTH, expand=1, padx=20, pady=20)

        hello_button = tk.Button(self, text="Cześć", command=self.say_hello)
        hello_button.pack(side=tk.LEFT, padx=(20, 0), pady=(0, 20))

        goodbye_button = tk.Button(self, text="Do widzenia", command=self.say_goodbye)
        goodbye_button.pack(side=tk.RIGHT, padx=(0, 20), pady=(0, 20))

    def say_hello(self):
        message = "Cześć, " + self.name_entry.get()
        msgbox.showinfo("Cześć", message)

    def say_goodbye(self):
        if msgbox.askyesno("Zamknąć?", "Na pewno zamknąć okno?"):
            message = "Okno zamknie się za 2 sekundy, " + self.name_entry.get()
            self.label_text.set(message)
```

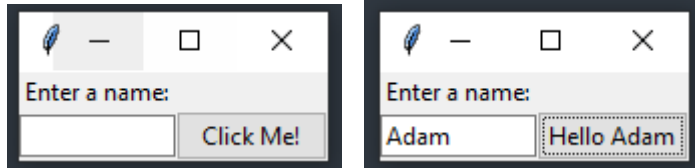
```

        self.after(2000, self.destroy)
    else:
        msgbox.showinfo("NIE", "A jednak nie, " + self.name_entry.get())

if __name__ == "__main__":
    window = Window()
    window.mainloop()

```

Jeszcze jeden przykład z polem Entry. Zwróćmy uwagę na rozmieszczenie elementów za pomocą grid. Również klucz (textvar) może się też nazywać textvariable. Szerokość pola Entry ustawiona jest na width=12



```

import tkinter as tk
from tkinter import ttk

win = tk.Tk()
win.title("Python GUI")

a_label = ttk.Label(win, text="A Label")
a_label.grid(column=0, row=0)

def click_me():
    action.configure(text='Hello ' + name.get())

ttk.Label(win, text="Enter a name:").grid(column=0, row=0)

name = tk.StringVar()
name_entered = ttk.Entry(win, width=12, textvariable=name)
name_entered.grid(column=0, row=1)

action = ttk.Button(win, text="Click Me!", command=click_me)
action.grid(column=1, row=1)

win.mainloop()

```

Teraz dodamy wywołanie na widżecie Entry funkcji focus(), dzięki której kursor będzie od razu w pozycji gotowej do wpisania zawartości.

```
name_entered.focus() # tutaj Focus - wpisać przed główną pętlą
```

Innym atrybutem można z kolei dezaktywować dany element. Np. button, wpisując state='disabled'

```
action.configure(state='disabled')
```

Rozszerzymy teraz okienko o widżet z rozwijaną listą wartości do wyboru. W tym celu:

- przesuniemy Button w prawo (będzie on trzecim elementem), więc column=2

```
action = ttk.Button(win, text="Click Me!", command=click_me)
action.grid(column=2, row=1)
```

- dodamy nową etykietę

```
ttk.Label(win, text="Choose a number:").grid(column=1, row=0)
```

- następnie tworzymy rozwijaną listę Combobox

```
number = tk.StringVar()
number_chosen = ttk.Combobox(win, width=12, textvariable=number)
number_chosen['values'] = (1, 2, 4, 42, 100) # to jest rzutowane na sting, bo taki jest typ number
number_chosen.grid(column=1, row=1)
number_chosen.current(0)
```

Liczby z listy przekazywane są do atrybutu 'values' w postaci krotki. Ustawienie na domyślne pole za pomocą funkcji current(). W obecnej postaci użytkownik może wpisać w polu z domyślną wartością dowolną swoją. Jeśli chcemy to zablokować, to w konstruktorze Combobox dodajemy argument state='readonly'

```
number_chosen = ttk.Combobox(win, width=12, textvariable=number, state='readonly')
```

Oczywiście, pobrać wybraną pozycję można poprzez akcję callback function wskazaną w opcjach Button. W tym przypadku number.get() lub number_chosen.get(). Teraz rozszerzymy okienko o kolejne elementy – pole wyboru (zaznacz / odznacz), które jest widżetem Checkbutton. Dodamy trzy takie elementy w dolnym rzędzie, pierwszy będzie nieaktywny, drugi będzie aktywny i niezaznaczony, trzeci będzie aktywny i domyślnie zaznaczony.

Przed linią name_entered.focus() wstawiamy:

```
chVarDis = tk.IntVar()
check1 = tk.Checkbutton(win, text="Disabled", variable=chVarDis, state='disabled')
check1.select()
check1.grid(column=0, row=4, sticky=tk.W)

chVarUn = tk.IntVar()
check2 = tk.Checkbutton(win, text="UnChecked", variable=chVarUn)
check2.deselect()
check2.grid(column=1, row=4, sticky=tk.W)

chVarEn = tk.IntVar()
check3 = tk.Checkbutton(win, text="Enabled", variable=chVarEn)
check3.select()
check3.grid(column=2, row=4, sticky=tk.W)
```

W ramach każdej komórki wskazana jest orientacja względna (na lewo): sticky=tk.W. Teraz dodamy trzy obiekty „Radiobutton”, których kliknięcie będzie wywoływać funkcję radCall(), w której sprawdzenie parametru

```
radVar = tk.IntVar()
```

będzie powodowało ustawienie (poprzez win.configure) koloru tła. W tym celu definiujemy nazwy dla kolorów:

```
COLOR1 = "Blue"
COLOR2 = "Gold"
COLOR3 = "Red"
```

zmienną, która będzie przetrzymywać stan danego Radiobutton

```
radVar = tk.IntVar()
```

i funkcję zwrotną, która będzie podpięta do obsługi po kliknięciu Radiobutton

```
def radCall():
    radSel=radVar.get()
    if radSel == 1: win.configure(background=COLOR1)
    elif radSel == 2: win.configure(background=COLOR2)
    elif radSel == 3: win.configure(background=COLOR3)
```

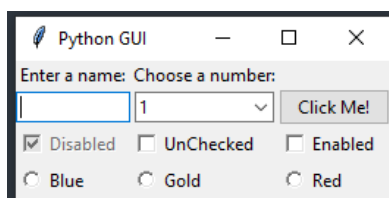
Kolejne elementy Radiobuttons utworzone oraz umieszczone (za pomocą grid) są następująco:

```
rad1 = tk.Radiobutton(win, text=COLOR1, variable=radVar, value=1, command=radCall)
rad1.grid(column=0, row=5, sticky=tk.W, columnspan=3)

rad2 = tk.Radiobutton(win, text=COLOR2, variable=radVar, value=2, command=radCall)
rad2.grid(column=1, row=5, sticky=tk.W, columnspan=3)

rad3 = tk.Radiobutton(win, text=COLOR3, variable=radVar, value=3, command=radCall)
rad3.grid(column=2, row=5, sticky=tk.W, columnspan=3)
```

Przykład i cały kod:



```
import tkinter as tk from tkinter import ttk

win = tk.Tk()
win.title("Python GUI")

a_label = ttk.Label(win, text="A Label")
a_label.grid(column=0, row=0)

def click_me():
    action.configure(text='Hello ' + name.get() + ' ' + number_chosen.get())

ttk.Label(win, text="Enter a name:").grid(column=0, row=0)

name = tk.StringVar()
name_entered = ttk.Entry(win, width=12, textvariable=name)
name_entered.grid(column=0, row=1)

action = ttk.Button(win, text="Click Me!", command=click_me)
action.grid(column=2, row=1) # <= change column to 2

# trzy checkbuttons
ttk.Label(win, text="Choose a number:").grid(column=1, row=0)
number = tk.StringVar()
number_chosen = ttk.Combobox(win, width=12, textvariable=number, state='readonly')
number_chosen['values'] = (1, 2, 4, 42, 100)
number_chosen.grid(column=1, row=1)
number_chosen.current(0)

chVarDis = tk.IntVar()
check1 = tk.Checkbutton(win, text="Disabled", variable=chVarDis, state='disabled')
check1.select()
check1.grid(column=0, row=4, sticky=tk.W)
```

```

chVarUn = tk.IntVar()
check2 = tk.Checkbutton(win, text="UnChecked", variable=chVarUn)
check2.deselect()
check2.grid(column=1, row=4, sticky=tk.W)

chVarEn = tk.IntVar()
check3 = tk.Checkbutton(win, text="Enabled", variable=chVarEn)
check3.deselect()
check3.grid(column=2, row=4, sticky=tk.W)

# GUI Callback function
def checkCallback(*ignoredArgs):
    # only enable one checkbutton
    if chVarUn.get(): check3.configure(state='disabled')
    else: check3.configure(state='normal')
    if chVarEn.get(): check2.configure(state='disabled')
    else: check2.configure(state='normal')

# trace the state of the two checkbuttons
chVarUn.trace('w', lambda unused0, unused1, unused2 : checkCallback())
chVarEn.trace('w', lambda unused0, unused1, unused2 : checkCallback())

# Radiobutton Globals
COLOR1 = "Blue"
COLOR2 = "Gold"
COLOR3 = "Red"

# Radiobutton Callback
def radCall():
    radSel=radVar.get()
    if radSel == 1: win.configure(background=COLOR1)
    elif radSel == 2: win.configure(background=COLOR2)
    elif radSel == 3: win.configure(background=COLOR3)

# create three Radiobuttons using one variable
radVar = tk.IntVar()

rad1 = tk.Radiobutton(win, text=COLOR1, variable=radVar, value=1, command=radCall)
rad1.grid(column=0, row=5, sticky=tk.W, columnspan=3)

rad2 = tk.Radiobutton(win, text=COLOR2, variable=radVar, value=2, command=radCall)
rad2.grid(column=1, row=5, sticky=tk.W, columnspan=3)

rad3 = tk.Radiobutton(win, text=COLOR3, variable=radVar, value=3, command=radCall)
rad3.grid(column=2, row=5, sticky=tk.W, columnspan=3)

name_entered.focus()
win.mainloop()

```

Przyjrzymy się ciekawej technice obserwowania stanu danej zmiennej, który może się zmienić. W Pythonie nie ma wbudowanego sposobu śledzenia zmiennych, jednak tkinter obsługuje tworzenie opakowań zmiennych, których można użyć w tym celu, dołączając wywołanie zwrotne „obserwatora” do zmiennej. Klasa tkinter.Variable zawiera konstruktory, takie jak BooleanVar, DoubleVar, IntVar i StringVar dla wartości logicznych, zmiennoprzecinkowych o podwójnej precyzji, liczb całkowitych i ciągów znaków, z których wszystkie mogą być zarejestrowane u obserwatora, który jest wyzwalany za każdym razem, gdy zmieni się wartość zmiennej. Obserwator pozostaje aktywny, dopóki nie zostanie usunięty. Funkcja wywołania zwrotnego powiązana z obserwatorem domyślnie pobiera trzy argumenty, tj. nazwę zmiennej Tkinter, indeks zmiennej tkinter w przypadku, gdy jest to tablica, w przeciwnym razie jest to pusty ciąg, oraz tryb dostępu. Obecnie w Pythonie są trzy funkcje:

- trace_add() służy do dodawania obserwatora do zmiennej i zwraca nazwę funkcji zwrotnej za każdym razem, gdy uzyskuje się dostęp do wartości.

Syntax : trace_add(self, mode, callback_name)

Parametry:

mode: "array", "read", "write", "unset", lub lista lub tuple takich rzeczy

callback_name: nazwa funkcji callback zarejestrowana dla zmiennej tkinter

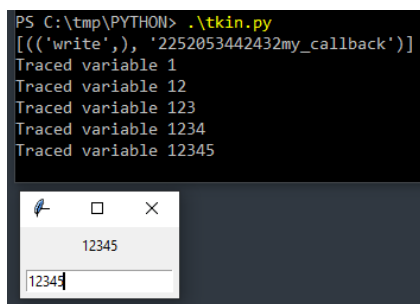
- trace_remove() służy do wyrejestrowania obserwatora. Zwraca nazwę wywołania zwrótnego używanego podczas początkowej rejestracji obserwatora za pomocą metody trace_add()

Syntax : trace_remove(self, mode, callback_name) – jak poprzednio

- trace_info() - zwraca nazwę wywołania zwrótnego, zwykle używana do znalezienia nazwy wywołania zwrótnego, które ma zostać usunięte. Nie przyjmuje żadnego argumentu poza zmienną tkinter.

Syntax : trace_info(self)

Przykład:



```
from tkinter import *

root = Tk()
my_var = StringVar()

# callback function (observer)
def my_callback(var, indx, mode):
    print("Traced variable {}".format(my_var.get()))

my_var.trace_add('write', my_callback)

Label(root, textvariable=my_var).pack(padx=5, pady=5)
Entry(root, textvariable=my_var).pack(padx=5, pady=5)

print(my_var.trace_info())
root.mainloop()
```

Przełożmy tę wiedzę na nasz przykład z widżetem Checkbox. Można, za pomocą takich obserwatorów i funkcji zwrótnych, kontrolować stan konkretnych pól wyboru i np. je aktywować / dezaktywować.

```
def checkCallback(*_):
    # tylko jeden checkbutton aktywny
    if chVarUn.get(): check3.configure(state='disabled')
    else: check3.configure(state='normal')
    if chVarEn.get(): check2.configure(state='disabled')
    else: check2.configure(state='normal')

# śledzenie stanu dwóch pól
chVarUn.trace_add('write', checkCallback)
chVarEn.trace_add('write', checkCallback)
```


Uwaga: zapis argumentów `*_` to nic innego jak inny zapis `*args`, przy czym konwencja tu jest taka, że argumenty oznaczone jako `_` są traktowane jako ignorowane (są to trzy, wyżej wymienione, argumenty).

Na koniec tej części dodamy obszar tekstowy z przewijaniem, czyli element `ScrolledText`. Widżety `ScrolledText` są znacznie większe niż widżety `Entry` i obejmują wiele wierszy. Przypominają Notatnik i zawijanie linii, automatycznie włączające pionowe paski przewijania, gdy tekst jest większy niż wysokość widżetu `ScrolledText`. Musimy zaimportować:

```
from tkinter import scrolledtext
```

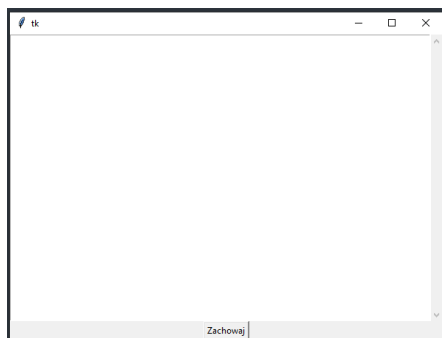
oraz definiujemy i pozycjonujemy:

```
scrol_w = 30
scrol_h = 3
scr = scrolledtext.ScrolledText(win, width=scrol_w, height=scrol_h, wrap=tk.WORD)
scr.grid(column=0, columnspan=3) # columnspan zależy od wcześniejszego podziału
```

Można poeksperymentować z ostatnią linią, dodając np. takie argumenty:

```
scr.grid(column=0, row=5, sticky='WE', columnspan=3)
```

Gotowy przykład, w którym odczytamy również zawartość widżetu `ScrolledText`:



```
import tkinter as tk
from tkinter.scrolledtext import ScrolledText

class SaveProject:
    def __init__(self, master):
        self.master = master
        self.textFrame = ScrolledText(self.master, width=80)
        self.textFrame.pack()

        self.save_btn = tk.Button(self.master, text='Zachowaj', command=self.save)
        self.save_btn.pack()

    def save(self):
        self.saveText = self.textFrame.get('1.0', tk.END)
        print('self.saveText:', self.saveText)

root = tk.Tk()
project = SaveProject(root)
root.mainloop()
```

Dodamy pewne elementy dekoracyjne – etykiety w ramce.

```
# Ramka
buttons_frame = ttk.LabelFrame(win, text=' Labels in a Frame ')
buttons_frame.grid(column=0, row=7)
# buttons_frame.grid(column=1, row=7)

# Etykiety w obiekcie ramki
ttk.Label(buttons_frame, text="Label1").grid(column=0, row=0, sticky=tk.W)
ttk.Label(buttons_frame, text="Label2").grid(column=1, row=0, sticky=tk.W)
ttk.Label(buttons_frame, text="Label3").grid(column=2, row=0, sticky=tk.W)
```

Można sprawdzić (zamieniając `column=0` na 1 lub 2 dla `buttons`), jak się cała grupa pozycjonuje. Podobnie, można zmienić ułożenie etykiet na `column=0` oraz `row=0, 1, 2`. Możemy również dodać marginesy (`padding`) do ramki:

```
buttons_frame.grid(column=0, row=7, padx=20, pady=40) # padx, pady
```

A teraz możemy dodać marginesy wokół każdej z etykiet, za pomocą pętli obiektów-potomków względem `buttons_frame`:

```
for child in buttons_frame.winfo_children():
    child.grid_configure(padx=8, pady=4)
```

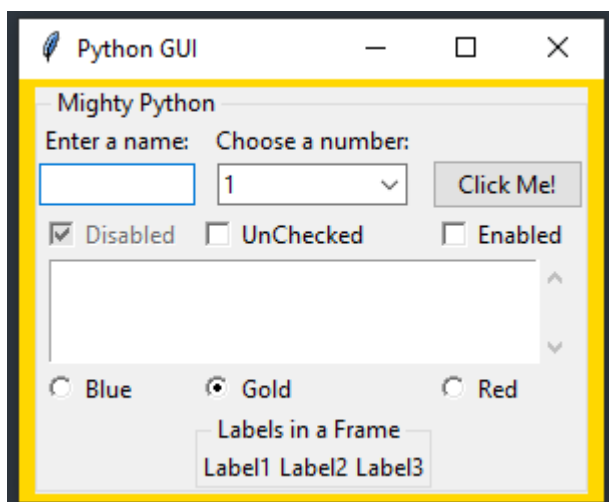
Funkcja `winfo_children()` zwraca listę wszystkich potomków należących do zmiennej `buttons_frame`. To pozwala nam przeglądać je w pętli i przypisywać marginesy do każdej etykiety. Funkcja `grid_configure()` pozwala modyfikować elementy interfejsu użytkownika, zanim główna pętla je wyświetli.

W kolejnym kroku zagnieźdźmy ramki w ramach. Konkretnie, tylko `LabelFrame` będzie zakotwiczone w głównym oknie (`win`), reszta obiektów zaś w nim.

Zaraz za oknem głównym stworzymy:

```
mighty = ttk.LabelFrame(win, text=' Mighty Python ')
mighty.grid(column=0, row=0, padx=8, pady=4)
```

Wszystkie kolejne elementy będą osadzone w „mighty”, cały kod:



```

import tkinter as tk
from tkinter import ttk

from tkinter import scrolledtext

win = tk.Tk()
win.title("Python GUI")

mighty = ttk.LabelFrame(win, text=' Mighty Python ')
mighty.grid(column=0, row=0, padx=8, pady=4)

a_label = ttk.Label(mighty, text="Enter a name:")
a_label.grid(column=0, row=0)

def click_me():
    action.configure(text='Hello ' + name.get() + ' ' + number_chosen.get())
    name = tk.StringVar()
name_entered = ttk.Entry(mighty, width=12, textvariable=name)
name_entered.grid(column=0, row=1)

action = ttk.Button(mighty, text="Click Me!", command=click_me)
action.grid(column=2, row=1) # <= change column to 2

ttk.Label(mighty, text="Choose a number:").grid(column=1, row=0)
number = tk.StringVar()
number_chosen = ttk.Combobox(mighty, width=12, textvariable=number, state='readonly')
number_chosen['values'] = (1, 2, 4, 42, 100)
number_chosen.grid(column=1, row=1)
number_chosen.current(0)

chVarDis = tk.IntVar()
check1 = tk.Checkbutton(mighty, text="Disabled", variable=chVarDis, state='disabled')
check1.select()
check1.grid(column=0, row=4, sticky=tk.W)

chVarUn = tk.IntVar()
check2 = tk.Checkbutton(mighty, text="UnChecked", variable=chVarUn)
check2.deselect()
check2.grid(column=1, row=4, sticky=tk.W)

chVarEn = tk.IntVar()
check3 = tk.Checkbutton(mighty, text="Enabled", variable=chVarEn)
check3.deselect()
check3.grid(column=2, row=4, sticky=tk.W)

def checkCallback(*ignoredArgs):
    if chVarUn.get():
        check3.configure(state='disabled')
    else:
        check3.configure(state='normal')
    if chVarEn.get():
        check2.configure(state='disabled')
    else:
        check2.configure(state='normal')

chVarUn.trace('w', lambda unused0, unused1, unused2: checkCallback())
chVarEn.trace('w', lambda unused0, unused1, unused2: checkCallback())

scrol_w = 30 scrol_h = 3 scr = scrolledtext.ScrolledText(mighty, width=scrol_w, height=scrol_h,
wrap=tk.WORD)
scr.grid(column=0, row=5, columnspan=3)

colors = ["Blue", "Gold", "Red"]

def radCall():
    radSel = radVar.get()

```

```

if radSel == 0:
    win.configure(background=colors[0]) # zero-based
elif radSel == 1:
    win.configure(background=colors[1]) # using list
elif radSel == 2:
    win.configure(background=colors[2])

radVar = tk.IntVar()
radVar.set(99)

for col in range(3):
    curRad = tk.Radiobutton(mighty, text=colors[col], variable=radVar, value=col,
command=radCall)
    curRad.grid(column=col, row=6, sticky=tk.W) # row=6

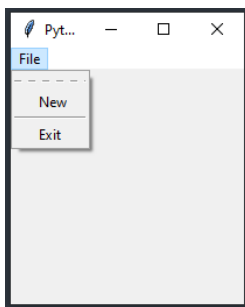
buttons_frame = ttk.LabelFrame(mighty, text=' Labels in a Frame ')
buttons_frame.grid(column=1, row=7) # now col 1

ttk.Label(buttons_frame, text="Label1").grid(column=0, row=0, sticky=tk.W)
ttk.Label(buttons_frame, text="Label2").grid(column=1, row=0, sticky=tk.W)
ttk.Label(buttons_frame, text="Label3").grid(column=2, row=0, sticky=tk.W)

name_entered.focus()
win.mainloop()

```

Dodamy proste menu do okienka:



```

import tkinter as tk
from tkinter import Menu

win = tk.Tk()
win.title("Python GUI")

menu_bar = Menu(win)
win.config(menu=menu_bar)

file_menu = Menu(menu_bar) # create File menu
file_menu.add_command(label="New") # add File menu item
file_menu.add_separator() # pozioma kreska
file_menu.add_command(label="Exit") # jeszcze jedn a pozycja
menu_bar.add_cascade(label="File", menu=file_menu) # add File menu to menu bar and give it a label

win.mainloop()

```

Zapewne nie podoba się nam przerywana linia na górze – kiedy się ją kliknie, można oddzielić menu co jest raczej niepraktyczne. Można ją usunąć:

```
file_menu = Menu(menu_bar, tearoff=0)
```

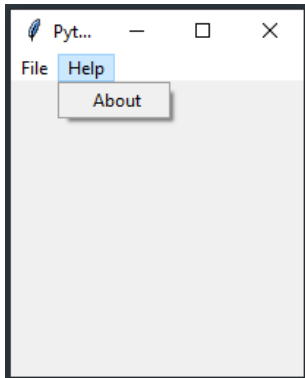
Dodajmy drugą pozycję menu obok File, będzie to Help (z About):

```
help_menu = Menu(menu_bar, tearoff=0)
menu_bar.add_cascade(label="Help", menu=help_menu)
help_menu.add_command(label="About")
```

Teraz trzeba podpiąć jakieś funkcje zwrotne.

```
def _quit():
    win.quit()
    win.destroy()
    exit()
```

Przykład i cały kod:



```
import tkinter as tk
from tkinter import Menu

win = tk.Tk()
win.title("Python GUI")

menu_bar = Menu(win)
win.config(menu=menu_bar)

def _quit():
    win.quit()
    win.destroy()
    exit()

file_menu = Menu(menu_bar, tearoff=0)
file_menu.add_command(label="New") # add File menu item
file_menu.add_separator() # pozioma kreska
file_menu.add_command(label="Exit", command=_quit)
menu_bar.add_cascade(label="File", menu=file_menu) # add File menu to menu bar and give it a label

help_menu = Menu(menu_bar, tearoff=0)
menu_bar.add_cascade(label="Help", menu=help_menu)
help_menu.add_command(label="About")

win.mainloop()
```

Wprowadzimy teraz zakładki. Sam kod z pojedynczą zakładką będzie taki:

```
import tkinter as tk from tkinter import ttk

win = tk.Tk() # Create instance
win.title("Python GUI") # Add a title
tabControl = ttk.Notebook(win) # Create Tab Control
tab1 = ttk.Frame(tabControl) # Create a tab
tabControl.add(tab1, text='Tab 1') # Add the tab
```

```
tabControl.pack(expand=1, fill="both") # Pack to make visible
win.mainloop()
```

Bardzo łatwo dodać drugą zakładkę:

```
tab2 = ttk.Frame(tabControl) # Add a second tab
tabControl.add(tab2, text='Tab 2') # Add second tab
```

Dodajmy do pierwszej zakładki ramkę LabelFrame z etykietą Label:

```
# LabelFrame
mighty = ttk.LabelFrame(tab1, text='Mighty Python ')
mighty.grid(column=0, row=0, padx=8, pady=4)

# Label osadzony w mighty
a_label = ttk.Label(mighty, text="Enter a name:")
a_label.grid(column=0, row=0, sticky='W')
```

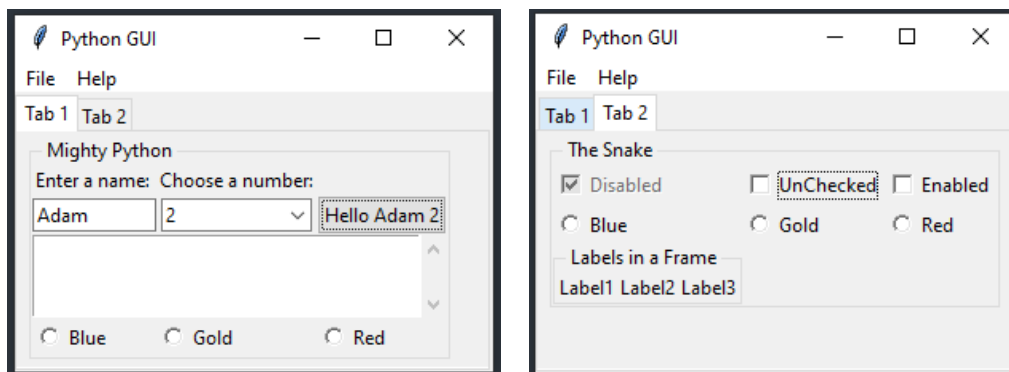
Dodamy jeszcze jedną etykietę plus pętlę po obiektach, żeby dodać marginesy:

```
ttk.Label(mighty, text="Choose a number:").grid(column=1, row=0)
```

oraz

```
for child in mighty.winfo_children():
    child.grid_configure(padx=8)
```

Teraz umieścimy z powrotem poprzednie widżety plus kilka dodatkowych, porozkładane pomiędzy obie zakładki. Przykład i kompletny kod:



```
import tkinter as tk
from tkinter import ttk
from tkinter import scrolledtext
from tkinter import Menu

win = tk.Tk()
win.title("Python GUI")

tabControl = ttk.Notebook(win) # Tab Control
tab1 = ttk.Frame(tabControl) # pierwszy tab
tabControl.add(tab1, text='Tab 1') # dodaj pierwszy tab do kontrolera
tab2 = ttk.Frame(tabControl) # drugi tab
tabControl.add(tab2, text='Tab 2') # dodaj drugi tab do kontrolera
tabControl.pack(expand=1, fill="both") # umiejscowienie

# LabelFrame wewnątrz tab1
mighty = ttk.LabelFrame(tab1, text='Mighty Python ')
mighty.grid(column=0, row=0, padx=8, pady=4)
```

```

# Label wewnątrz mighty
a_label = ttk.Label(mighty, text="Enter a name:")
a_label.grid(column=0, row=0, sticky='W')

# funkcja zwrotna po kliknięciu Button
def click_me():
    action.configure(text='Hello ' + name.get() + ' ' + number_chosen.get())

# Textbox Entry
name = tk.StringVar()
name_entered = ttk.Entry(mighty, width=12, textvariable=name)
name_entered.grid(column=0, row=1, sticky='W')

# Button
action = ttk.Button(mighty, text="Click Me!", command=click_me)
action.grid(column=2, row=1)

ttk.Label(mighty, text="Choose a number:").grid(column=1, row=0)
number = tk.StringVar()
number_chosen = ttk.Combobox(mighty, width=12, textvariable=number, state='readonly')
number_chosen['values'] = (1, 2, 4, 42, 100)
number_chosen.grid(column=1, row=1)
number_chosen.current(0)

# scrolled Text
scrol_w = 30
scrol_h = 3
scr = scrolledtext.ScrolledText(mighty, width=scrol_w, height=scrol_h, wrap=tk.WORD)
scr.grid(column=0, row=2, sticky='WE', columnspan=3)

# Tab Control 2 -----
mighty2 = ttk.LabelFrame(tab2, text=' The Snake ')
mighty2.grid(column=0, row=0, padx=8, pady=4)

# 3 checkbuttons
chVarDis = tk.IntVar()
check1 = tk.Checkbutton(mighty2, text="Disabled", variable=chVarDis, state='disabled')

check1.select()
check1.grid(column=0, row=4, sticky=tk.W)

chVarUn = tk.IntVar()
check2 = tk.Checkbutton(mighty2, text="Unchecked", variable=chVarUn)
check2.deselect()
check2.grid(column=1, row=4, sticky=tk.W)

chVarEn = tk.IntVar()
check3 = tk.Checkbutton(mighty2, text="Enabled", variable=chVarEn)
check3.deselect()
check3.grid(column=2, row=4, sticky=tk.W)

def checkCallback(*ignoredArgs):
    if chVarUn.get():
        check3.configure(state='disabled')
    else:
        check3.configure(state='normal')
    if chVarEn.get():
        check2.configure(state='disabled')
    else:
        check2.configure(state='normal')

chVarUn.trace_add('write', checkCallback)
chVarEn.trace_add('write', checkCallback)
colors = ["Blue", "Gold", "Red"]

def radCall():
    radSel = radVar.get()

```

```

    if radSel == 0:
        mighty2.configure(text='Blue')
    elif radSel == 1:
        mighty2.configure(text='Gold')
    elif radSel == 2:
        mighty2.configure(text='Red')

radVar = tk.IntVar()
radVar.set(99)

for col in range(3):
    curRad = tk.Radiobutton(mighty2, text=colors[col], variable=radVar, value=col,
command=radCall)
    curRad.grid(column=col, row=6, sticky=tk.W) # row=6

for col in range(3):
    curRad = tk.Radiobutton(mighty, text=colors[col], variable=radVar, value=col,
command=radCall)
    curRad.grid(column=col, row=3, sticky=tk.W) #row=3

# kontener na etykiety
buttons_frame = ttk.LabelFrame(mighty2, text=' Labels in a Frame ')
buttons_frame.grid(column=0, row=7)

# etykiety w kontenerze
ttk.Label(buttons_frame, text="Label1").grid(column=0, row=0, sticky=tk.W)
ttk.Label(buttons_frame, text="Label2").grid(column=1, row=0, sticky=tk.W)
ttk.Label(buttons_frame, text="Label3").grid(column=2, row=0, sticky=tk.W)

# Exit GUI cleanly
def _quit():
    win.quit()
    win.destroy()
    exit()

# Menu Bar
menu_bar = Menu(win)
win.config(menu=menu_bar)

# MENU
file_menu = Menu(menu_bar, tearoff=0)
file_menu.add_command(label="New")
file_menu.add_separator()
file_menu.add_command(label="Exit", command=_quit)
menu_bar.add_cascade(label="File", menu=file_menu)

# MENU 2
help_menu = Menu(menu_bar, tearoff=0)
help_menu.add_command(label="About")
menu_bar.add_cascade(label="Help", menu=help_menu)

name_entered.focus()
win.mainloop()

```

Wykonamy serię modyfikacji dodając jakieś funkcjonalności do bieżącego okienka, czyli funkcje callback. Przed MENU 2 dodamy (pamiętając, że na początku from tkinter import messagebox as msg)

```

# Message Box
def _msgBox():
    msg.showinfo('Message Info Box', 'Fragment okna informacyjnego,\ndwie linijki.')

```

Tu można zbadać warianty:

```

msg.showwarning('Message Warning Box', 'Fragment okna ostrzegawczego,\nmamy rok 2021.')

```


oraz

```
msg.showerror('', '')
```

Oraz:

```
help_menu.add_command(label="About", command=_msgBox)
```

Możemy nieco zmodyfikować funkcję, aby zobaczyć okienko pop-up z trzema opcjami wyboru (Tak, Nie, Anuluj)

```
def _msgBox():
    answer = msg.asksyesnocancel("Pytanie", "Na pewno?")
    if answer:
        msg.showinfo("Informacja", "Kliknales w menu\nzatem sie pojawiam.")
```

Note: gdybyśmy chcieli schować jakieś okienko, to służy do tego funkcja `.withdraw()`
Kolejne modyfikacje. Dodajmy ikonkę dekorującą główne okno aplikacji (można samemu narysować <https://www.favicon.cc/>)

```
win.iconbitmap('favicon.ico')
```

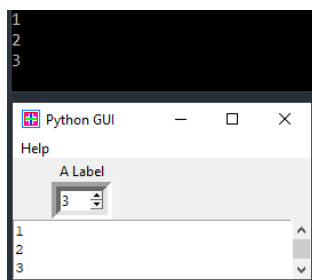
Dodajmy inny widżet, Spinbox. Pozwala on, za pomocą małych strzałek, wybrać wartość:

```
spin = tk.Spinbox(win, from_=0, to=10)
spin.grid(column=0, row=2)
```

W powyższym Spinbox można podawać różne parametry np. szerokość, ramkę:

```
spin = tk.Spinbox(win, from_=0, to=10, width=4, bd=8)
```

Dodamy teraz funkcję zwrotną do Spinbox, która wypisze informację w poniższym ScrolledText oraz na konsoli (cały kod):



```
import tkinter as tk
from tkinter import ttk
from tkinter import messagebox as msg
from tkinter import Menu
from tkinter import scrolledtext

win = tk.Tk()
win.title("Python GUI")
win.iconbitmap('favicon2.ico')

a_label = ttk.Label(win, text="A Label")
```

```

a_label.grid(column=0, row=0)

def _spin():
    value = spin.get()
    print(value)
    scrol.insert(tk.INSERT, value + '\n')

spin = tk.Spinbox(win, from_=0, to=10, width=4, bd=8, command=_spin)
spin.grid(column=0, row=2)

scrol_w = 30
scrol_h = 3
scrol = scrolledtext.ScrolledText(win, width=scrol_w, height=scrol_h, wrap=tk.WORD)
scrol.grid(column=0, row=3, sticky='WE', columnspan=3)

def msgBox():
    answer = msg.askyesnocancel("Pytanie", "Na pewno?")
    if answer:
        msg.showinfo("Informacja", "Kliknales w menu\nzatem sie pojawiam.")

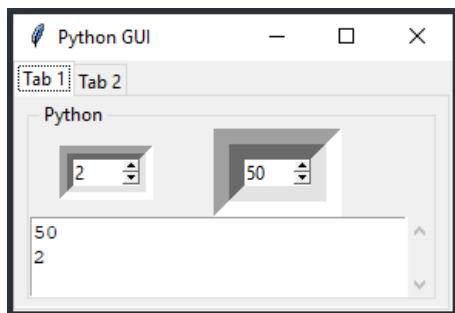
menu_bar = Menu(win)
win.config(menu=menu_bar)

help_menu = Menu(menu_bar, tearoff=0)
help_menu.add_command(label="About", command=msgBox)
menu_bar.add_cascade(label="Help", menu=help_menu)

win.mainloop()

```

Możemy przyrzeć się różnym opcjom wklęsłości / płaskości / wypukłości na przykładzie spinboxu (jeszcze jednego):



```

import tkinter as tk
from tkinter import ttk
from tkinter import scrolledtext
from tkinter import Menu
from tkinter import messagebox as msg
from tkinter import Spinbox

win = tk.Tk()
win.title("Python GUI")

tabControl = ttk.Notebook(win)

tab1 = ttk.Frame(tabControl)
tabControl.add(tab1, text='Tab 1')
tab2 = ttk.Frame(tabControl)
tabControl.add(tab2, text='Tab 2')

tabControl.pack(expand=1, fill="both")

mighty = ttk.LabelFrame(tab1, text=' Python ')
mighty.grid(column=0, row=0, padx=8, pady=4)

```

```

# Spinbox callback
def _spin():
    value = spin.get()
    print(value)
    scrol.insert(tk.INSERT, value + '\n')

# Spinbox widget
spin = Spinbox(mighty, values=(1, 2, 4, 42, 100), width=5, bd=9, command=_spin) # using range
spin.grid(column=0, row=2)

# Spinbox2 callback
def _spin2():
    value = spin2.get()
    print(value)
    scrol.insert(tk.INSERT, value + '\n')

spin2 = Spinbox(mighty, values=(0, 50, 100), width=5, bd=20, command=_spin2) # default value is: tk.SUNKEN
# spin2 = Spinbox(mighty, values=(0, 50, 100), width=5, bd=20, command=_spin2, relief=tk.FLAT)
# spin2 = Spinbox(mighty, values=(0, 50, 100), width=5, bd=20, command=_spin2, relief=tk.RAISED)
# spin2 = Spinbox(mighty, values=(0, 50, 100), width=5, bd=20, command=_spin2, relief=tk.SUNKEN)
# default
# spin2 = Spinbox(mighty, values=(0, 50, 100), width=5, bd=20, command=_spin2, relief=tk.GROOVE)
# spin2 = Spinbox(mighty, values=(0, 50, 100), width=5, bd=20, command=_spin2, relief=tk.RIDGE)

spin2.grid(column=1, row=2)

scrol_w = 30 scrol_h = 3 scrol = scrolledtext.ScrolledText(mighty, width=scrol_w,
height=scrol_h, wrap=tk.WORD)
scrol.grid(column=0, row=3, sticky='WE', columnspan=3)

# Tab Control 2
win.mainloop()

```

Do każdego element składowego okna można dodać „dymek informacyjny”, wykonuje się to poprzez przypisanie odpowiednich akcji związanych z położeniem myszy. Przykład z definicją klasy realizującą tego typu zachowanie:

```

class Tooltip(object):
    def __init__(self, widget, tip_text=None):
        self.widget = widget          self.tip_text = tip_text          widget.bind('<Enter>',
self.mouse_enter) # bind mouse events
        widget.bind('<Leave>', self.mouse_leave)

    def mouse_enter(self, _event):
        self.show_tooltip()

    def mouse_leave(self, _event):
        self.hide_tooltip()

    def show_tooltip(self):
        if self.tip_text:
            x_left = self.widget.winfo_rootx() # get widget top-left coordinates
            y_top = self.widget.winfo_rooty() - 18 # place tooltip above widget or it flickers

            self.tip_window = tk.Toplevel(self.widget) # create Toplevel window; parent=widget
            self.tip_window.overridedirect(True) # remove surrounding toolbar window
            self.tip_window.geometry("+%d+%d" % (x_left, y_top)) # position tooltip

            label = tk.Label(self.tip_window, text=self.tip_text, justify=tk.LEFT,
                             background="ffffe0", relief=tk.SOLID, borderwidth=1,
                             font=("tahoma", "8", "normal"))
            label.pack(ipadx=1)

    def hide_tooltip(self):

```

```
if self.tip_window:
    self.tip_window.destroy()
```

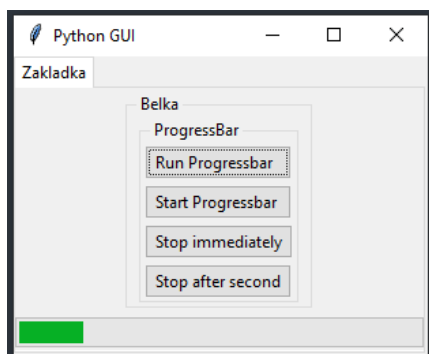
I teraz np. można zwi zać tak  akcj  plus wy wietlany tekst:

```
ToolTip(spin, 'To jest kontrolka Spin')
ToolTip(scrol, 'To jest opis wid etu \nScrolledText - druga linijka')
```

albo dla obiektu Button (przypisanego do zmiennej action):

```
ToolTip(action, 'Jak mnie klikniesz to zobaczysz')
```

Jeszcze jeden, nietypowy element: pasek post pu (Progressbar) w takim przyk adzie (wi cej szczeg lowych informacji na stronie <https://docs.python.org/3/library/tkinter.ttk.html#progressbar>):



```
import tkinter as tk from tkinter import ttk from time import sleep

win = tk.Tk()
win.title("Python GUI")

tabControl = ttk.Notebook(win)
tab = ttk.Frame(tabControl)
tabControl.add(tab, text='Zakladka')

tabControl.pack(expand=1, fill="both")

# Tab Control -----
mighty = ttk.LabelFrame(tab, text=' Belka ')
mighty.grid(column=0, row=0, padx=8, pady=4)

# Progressbar
progress_bar = ttk.Progressbar(tab, orient='horizontal', length=300, mode='determinate')
progress_bar.grid(column=0, row=3, pady=2)

def run_progressbar():
    progress_bar["maximum"] = 100    for i in range(101):
        sleep(0.05)
        progress_bar["value"] = i # increment
        progress_bar.update() # update() in loop
        progress_bar["value"] = 0 # reset/clear progressbar

def start_progressbar():
    progress_bar.start()

def stop_progressbar():
    progress_bar.stop()

def progressbar_stop_after(wait_ms=1000):
    win.after(wait_ms, progress_bar.stop)
```

```
# Create a container to hold buttons
buttons_frame = ttk.LabelFrame(mighty, text=' ProgressBar ')
buttons_frame.grid(column=0, row=2, sticky='W', columnspan=2)

# Add Buttons for Progressbar commands
ttk.Button(buttons_frame, text=" Run Progressbar ", command=run_progressbar).grid(column=0,
row=0, sticky='W')
ttk.Button(buttons_frame, text=" Start Progressbar ", command=start_progressbar).grid(column=0,
row=1, sticky='W')
ttk.Button(buttons_frame, text=" Stop immediately ", command=stop_progressbar).grid(column=0,
row=2, sticky='W')
ttk.Button(buttons_frame, text=" Stop after second ",
command=progressbar_stop_after).grid(column=0, row=3, sticky='W')

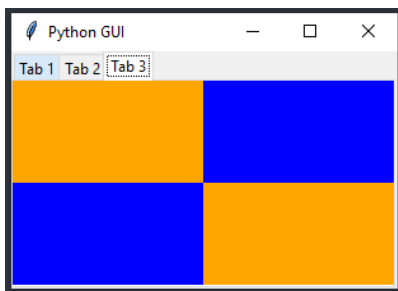
for child in buttons_frame.winfo_children():
    child.grid_configure(padx=2, pady=2)

for child in mighty.winfo_children():
    child.grid_configure(padx=8, pady=2)

win.mainloop()
```

I na koniec – Canvas, bardzo ciekawy i kreatywny element!

Najpierw przykład, w którym wyrysowana (na zakładce nr 3) zostaje niebiesko-pomarańczowa szachownica:



```
import tkinter as tk from tkinter import ttk

win = tk.Tk()
win.title("Python GUI")

tabControl = ttk.Notebook(win)

tab1 = ttk.Frame(tabControl)
tabControl.add(tab1, text='Tab 1')
tab2 = ttk.Frame(tabControl)
tabControl.add(tab2, text='Tab 2')
tab3 = ttk.Frame(tabControl)
tabControl.add(tab3, text='Tab 3')

tabControl.pack(expand=1, fill="both")

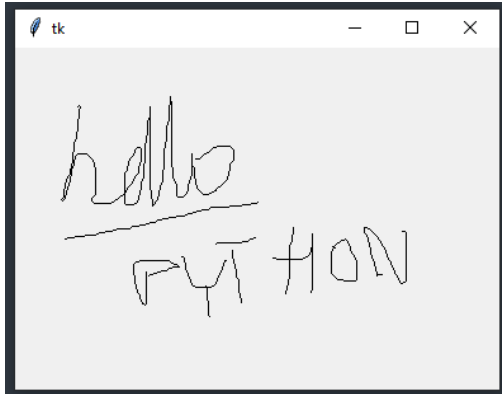
# Tab Control 3 -----
tab3_frame = tk.Frame(tab3, bg='blue')
tab3_frame.pack()
for orange_color in range(2):
    canvas = tk.Canvas(tab3_frame, width=150, height=80, highlightthickness=0, bg='orange')
    canvas.grid(row=orange_color, column=orange_color)

win.mainloop()
```

A na koniec demonstracja możliwości związana z powiązaniem akcji (bind) oraz prostych operacji z Canvas (takich, jak rysowanie linii). Akcje to np. kliknięcie myszą, albo przesuwanie

myszki w stanie wciśniętego przycisku. Pierwsza z nich odczytuje pozycję, druga tworzy linię i zachowuje punkt.

A na koniec demonstracja możliwości związana z powiązaniem akcji (bind) oraz prostych operacji z Canvas (takich, jak rysowanie linii). Akcje to np. kliknięcie myszą, albo przesuwanie myszy w stanie wciśniętego przycisku. Pierwsza z nich odczytuje pozycję, druga tworzy linię i zachowuje punkt.



```
from tkinter import *

def savePosn(event):
    global lastx, lasty
    lastx, lasty = event.x, event.y

def addLine(event):
    canvas.create_line((lastx, lasty, event.x, event.y))
    savePosn(event)

root = Tk()
root.columnconfigure(0, weight=1)
root.rowconfigure(0, weight=1)

canvas = Canvas(root)
canvas.grid(column=0, row=0, sticky=(N, W, E, S))
canvas.bind("<Button-1>", savePosn)
canvas.bind("<B1-Motion>", addLine)

root.mainloop()
```

To tylko część możliwości interfejsu tkinter. Osoby zainteresowane z łatwością odnajdą literaturę, bądź szereg kursów na YT pokazujących możliwości i zastosowania.